

ENEL712 Embedded Systems Design

Course Overview

Generated learning resource

May 25, 2026

Contents

How to use this resource	12
1 Course Introduction & Embedded Systems Foundations	13
1.1 MCU Architecture Fundamentals	13
1.1.1 RISC (Reduced Instruction Set Computer)	13
1.1.2 Two-Stage Pipelining	13
1.1.3 Harvard vs. Von Neumann	14
1.2 Memory Types and Constraints	14
1.3 Communication Protocols	14
1.3.1 UART (Asynchronous)	14
1.3.2 SPI (Synchronous)	14
1.3.3 I ² C (Synchronous)	14
1.4 Advanced Concepts and Resilience	15
1.5 Use Cases & Applications	15
1.5.1 When NOT to use an 8-bit AVR	15
1.6 Worked Examples	15
1.6.1 Pipeline flush cost	15
1.6.2 EEPROM endurance	15
1.7 Code Snippets	16
1.7.1 Storing a constant in Flash with PROGMEM	16
1.7.2 Petting the watchdog	16
2 Microcontroller Operation & Architecture	18
2.1 Overview of a Microcontroller	18
2.2 Microcontroller Operation: Fetch–Decode–Execute	18
2.3 Core CPU Components	19
2.4 Architecture and Performance Concepts	19
2.4.1 Harvard Architecture	19
2.4.2 Pipelining	19
2.5 RISC vs. CISC Comparison	19
2.6 Use Cases & Applications	19
2.7 Worked Example: Trace of a Simple Instruction	20
2.8 Practical Notes	20
3 MCU Memory Architecture & Clocking	22
3.1 Architectural Fundamentals	22
3.2 Microcontroller Memory Types	23
3.3 SRAM Usage, Memory Map, and the Stack	23
3.4 Bus System	23
3.5 System Clock Architecture	23

3.5.1	Clock Accuracy: The UART Constraint	24
3.6	Use Cases & Applications	24
3.7	Formulas & Calculations	24
3.7.1	UART baud-rate accuracy budget	24
3.7.2	EEPROM lifetime with wear leveling	24
3.8	Memory Map Cheat Sheet (AT90USB128)	25
4	Advanced MCUs and Microprocessors (MPUs)	27
4.1	Transitioning Beyond 8-bit MCUs	27
4.1.1	Atmel XMEGA MCUs	27
4.1.2	Hybrid DSP/MCUs	27
4.2	The ARM Architecture	28
4.2.1	Cortex-M Hierarchy	28
4.3	MCUs vs. MPUs	28
4.4	Selection Advice for Advanced Systems	28
4.5	Use Cases & Applications	29
4.5.1	Where each device family lives	29
4.6	Decision Framework	29
4.7	Worked Example: Picking a Chip for a Quadcopter Flight Controller	29
5	Embedded Digital Communications A	31
5.1	Introduction to Embedded Communication	31
5.2	Parallel vs. Serial Communication	31
5.2.1	Legacy parallel interfaces	31
5.3	Asynchronous Serial (RS-232 / RS-422 / RS-485)	32
5.3.1	RS-232	32
5.3.2	RS-422 / RS-485	32
5.4	Synchronous Serial (SPI, I ² C)	32
5.4.1	SPI (Serial Peripheral Interface)	32
5.4.2	I ² C (Inter-Integrated Circuit)	32
5.5	Other Notable Protocols	32
5.6	Use Cases & When to Pick What	33
5.7	Formulas & Calculations	33
5.7.1	Effective vs. raw UART byte rate	33
5.7.2	I ² C ACK timing	33
5.8	Worked Example: SPI Mode Picking	33
5.9	Code Snippets	33
5.9.1	AVR UART transmit one byte	33
5.9.2	Reading an I ² C sensor	33
6	Embedded Digital Communications B	35
6.1	Beyond Basic Serial Protocols	35
6.2	Single-Ended vs Differential Signalling	35
6.2.1	Single-ended	35
6.2.2	Differential	35
6.3	Cable Construction and Distance	36
6.4	Universal Serial Bus (USB)	36
6.4.1	Microcontroller implementation	36
6.5	Ethernet on Microcontrollers	36
6.6	Use Cases & Applications	36
6.7	Formulas & Rules of Thumb	37
6.8	Worked Example: Why Long Single-Ended Cables Fail	37

6.9	Practical Notes	37
7	Embedded Hardware Design	39
7.1	The Hardware Design Process	39
7.1.1	Vineyard weather station block components	39
7.2	I/O Device Selection	40
7.2.1	Wireless module requirements	40
7.3	Microcontroller Selection	40
7.3.1	I/O Tally	40
7.3.2	Weather station tally example	40
7.3.3	Selected MCU example	41
7.4	Power Supply Design	41
7.4.1	Current tally example	41
7.5	Switching vs Linear Regulators	41
7.6	Next Steps	41
7.7	Use Cases & Applications	41
7.8	Formulas & Calculations	42
7.8.1	Battery life from current tally	42
7.8.2	Worked example: vineyard station	42
7.8.3	Sizing a 20 % current margin	42
7.9	Design Checklist	42
8	Embedded Software Design A — Software Engineering	44
8.1	Fundamentals of Software Engineering	44
8.2	Software Process Activities	44
8.3	Requirements Engineering	45
8.3.1	Functional vs non-functional	45
8.4	System Modelling and Abstraction	45
8.5	Software Testing	45
8.6	Use Cases & Applications	45
8.7	Worked Example: A Tiny Specification	45
8.8	State Diagram in Words	46
8.9	Cost-of-Defect Curve	46
9	Embedded Software Design B — Architectural Design	48
9.1	Embedded Software Development Fundamentals	48
9.2	Development Approaches	49
9.3	Software Architectures	49
9.3.1	Polling Loop	49
9.3.2	Event-Driven	49
9.3.3	Finite State Machine (FSM)	49
9.3.4	Operating System (OS)	49
9.3.5	Real-Time Operating System (RTOS)	49
9.4	Hardware/Software Partitioning	49
9.5	Fault Tolerance and Defensive Design	50
9.5.1	Fault tolerance techniques	50
9.5.2	Defensive software	50
9.6	Software Documentation	50
9.7	Use Cases & Applications	50
9.8	Worked Examples	50
9.8.1	Polling vs interrupts for a button	50
9.8.2	Watchdog timer setup	51

9.9	Code Snippets	51
9.9.1	Switch-case FSM	51
9.9.2	POST + checksum on settings	51
10	Modularizing Embedded Projects	53
10.1	Core Concepts of Modularization	53
10.2	Structuring Modular Software	54
10.3	Advantages and Disadvantages	54
10.4	Specialized Modular Techniques	54
10.4.1	Driver levels	54
10.4.2	Wrapper functions	54
10.5	Six-Step Modular Design Recipe	54
10.6	Encapsulation and Scoping in C	55
10.7	Performance Implications	55
10.8	Use Cases & Applications	55
10.9	Worked Example: Layered SD-Card Driver	55
10.10	Code Snippets	55
10.10.1	Header file (public API)	55
10.10.2	Source file with private statics	56
10.10.3	Wrapper function example	56
11	Advanced C Programming Techniques	58
11.1	Switch-Case State Machines	58
11.2	Advanced Datatypes	59
11.3	Pointer Concepts and Arithmetic	59
11.4	Standard C Library Functions	59
11.4.1	<stdio.h>	59
11.4.2	<stdlib.h>	59
11.4.3	<string.h>	59
11.5	Variable Access and Scope	60
11.6	Embedded Risks with Strings and Arrays	60
11.7	Use Cases & Applications	60
11.8	Code Snippets	60
11.8.1	Function pointer / callback	60
11.8.2	Static local for state	61
11.8.3	sprintf-into-buffer for UART output	61
11.9	Common Pitfalls	61
12	Software Architecture Case Study — Vineyard Weather Station	63
12.1	Project Specification	63
12.2	Hardware Block Diagram Overview	63
12.3	Software Architecture and Modularization	64
12.3.1	1. Defining modules	64
12.3.2	2. Multiple source files	64
12.3.3	3. Functions and logic	64
12.4	Final Modular Structure for the Weather Station	64
12.5	Driver Abstraction Levels	65
12.6	State-Machine + Event-Driven Architecture	65
12.7	Use Cases & Applications	65
12.8	Worked Example: Time Budget per Wake	65
12.9	State-Machine Skeleton (main.c)	65
12.10	Failure Modes & Mitigations	66

13 Cost, Memory & Power Estimation	68
13.1 Cost Estimation	68
13.2 Memory Estimation	68
13.2.1 C program storage and overhead	68
13.2.2 Estimation challenges and risks	69
13.3 Power Estimation	69
13.3.1 Constant loads	69
13.3.2 Varying loads	69
13.4 Power-Saving Techniques	69
13.5 Use Cases & Applications	69
13.6 Formulas & Calculations	69
13.6.1 Battery life (constant load)	69
13.6.2 Battery life (duty-cycled load)	70
13.6.3 Memory rule of thumb	70
13.6.4 Volume cost reduction	70
13.7 Worked Example: Sensor Node Battery Life	70
13.8 Memory Budget Worksheet	70
 14 Introduction to OOP and C#	 72
14.1 OOP Overview	72
14.1.1 In-class framing: “why this feels weird at first”	72
14.1.2 The four pillars in plain words	73
14.2 Classes, Objects, Properties, Methods	73
14.2.1 “Class oriented programming”	73
14.2.2 What is a “module” here?	73
14.3 Encapsulation and Access Control	74
14.3.1 Two parts of encapsulation: grouping + gateway	74
14.3.2 Why <code>public</code> is a global variable in disguise	74
14.4 C# Memory and Initialization Mechanics	74
14.5 Introduction to C#	74
14.6 Capabilities and Use Cases	74
14.7 Why Use C# (and Why Not)	75
14.7.1 Advantages	75
14.7.2 Disadvantages	75
14.8 Use Cases & Applications	75
14.9 Worked Example: A Tiny Account Class	75
14.10 C# vs C++ vs C — Quick Compare	76
14.11 Intuition — Why OOP Feels Strange at First	76
14.12 Value Types vs Reference Types — In Practice	76
14.13 Mutable vs Immutable Types	77
14.14 Picking a Type — <code>var</code> , <code>decimal</code> , <code>double</code> , <code>int</code>	77
14.15 Methods vs Properties — Reading the Brackets	77
14.15.1 The in-class walkthrough — “brackets mean it does something”	78
14.15.2 Why IntelliSense is your first OOP debugger	78
14.16 Abstraction — The House / Street / Neighbourhood Analogy	78
14.16.1 Why this matters for .NET	79
14.17 Namespaces — The Same-Realm Rule	79
14.17.1 The header-file analogy from class	79
14.18 Encapsulation — The Rainbow’s End Analogy	79
14.18.1 Why this analogy is worth memorising	80
14.19 The Building-After-Hours Analogy	80

15 OOP in C# — Class Mechanics & Encapsulation	83
15.1 Introduction to OOP	83
15.2 Encapsulation in Detail	83
15.3 Class Mechanics in C#	84
15.3.1 In-class walkthrough: declaring vs instantiating	84
15.3.2 Naming convention: capital letters announce a class	84
15.3.3 Access modifiers	84
15.4 Constructors and Overloading	84
15.5 Properties: Get and Set	85
15.6 Code Example: Circle Class Evolution	85
15.6.1 The bank-balance question that motivates encapsulation	85
15.7 Use Cases & Applications	85
15.8 Worked Example: Circle Class Evolution	85
15.8.1 Bad — public field	85
15.8.2 Better — private field	86
15.8.3 Best — encapsulated with property and validation	86
15.9 Stack vs Heap for a Class Instance	86
15.10 Naming Conventions — Read at a Glance	87
15.11 Constructors — Default, Explicit, Overloaded	87
15.11.1 The in-class confusion: “what is the area going to be?”	87
15.11.2 “Constructor shares its name with the class”	88
15.11.3 Overloading the constructor: what changes for the caller	88
15.12 The Circle Class — Three-Step Evolution	88
15.12.1 Step 1: public field — too open	88
15.12.2 Step 2: private field + constructor — too closed	89
15.12.3 Step 3: property with get/set — the gateway	89
15.13 Properties as Gateways — Why Bother	89
15.13.1 The honest in-class objection — “why bother, this is just an int?”	90
15.13.2 The role of the implicit <code>value</code> keyword	90
15.14 Method Overloading — Same Name, Different Signature	90
16 OOP Part B — Static, Inheritance, Polymorphism	93
16.1 C# Naming Conventions	93
16.1.1 PascalCase — types and public members	93
16.1.2 camelCase — locals, parameters, private fields	94
16.1.3 What <i>not</i> to use	94
16.2 Static Methods and Data Members	94
16.2.1 The transcript framing — “called by the class, not the instance”	95
16.2.2 Two everyday static classes you have already used	95
16.3 Inheritance	95
16.3.1 Why inheritance exists: avoiding duplication	95
16.3.2 “You cannot derive from two base classes”	96
16.3.3 Windows Forms — inheritance you have already seen	96
16.3.4 Calling base-class constructors	96
16.4 Polymorphism and Overloading	96
16.5 Enumerations (enum)	97
16.5.1 “Not actually an OOP concept”	97
16.6 Coaxial Cable Class Exercise	97
16.7 Use Cases & Applications	97
16.8 Worked Example: Employee Hierarchy	98
16.9 Worked Example: Coaxial Cable Model	98

16.10	Operator Overloading Sketch	98
17	OOP Part C — Virtual, Override, Abstract, Sealed	100
17.1	Virtual Methods and the override Keyword	100
17.1.1	The lecturer’s framing: “a placeholder for derived classes”	100
17.1.2	<code>ToString()</code> — the virtual method you inherit for free	101
17.1.3	Rules for using virtual methods (don’t get any of these wrong)	101
17.2	Polymorphism (Many Forms)	101
17.3	Protected Access Modifier	102
17.4	Abstract Methods and Classes	102
17.4.1	Virtual vs abstract — one-line difference	102
17.5	Sealed Classes and Methods	102
17.5.1	In the lecture: “not often used — FYI only”	102
17.6	Use Cases & Applications	103
17.7	Worked Example: Shape Hierarchy with Polymorphism	103
17.8	Common Pitfalls	104
18	.NET Framework & Automatic Memory Management	106
18.1	Introduction to .NET	106
18.1.1	.NET Framework vs .NET — the distinction that matters in this course	106
18.1.2	NuGet — “it is like pip”	107
18.2	Core .NET Components	107
18.3	Common Language Runtime (CLR)	107
18.3.1	How the CLR works	107
18.3.2	Native vs JIT vs interpreted — where C# sits	108
18.3.3	Common Type System — “write once in any .NET language”	108
18.3.4	Pros & cons	108
18.4	.NET Class Library and Data Types	108
18.5	Namespaces	109
18.6	Automatic Memory Management	109
18.6.1	Allocation on the managed heap	109
18.6.2	In-class framing: “the heap has a shape”	109
18.6.3	Garbage Collection (GC) algorithm	109
18.6.4	In-class walkthrough: “assume everything is garbage”	109
18.6.5	Pointers and <code>unsafe</code> — and why you almost never need them	110
18.6.6	Threading inside the CLR (and the timer-vs-comm bug)	110
18.6.7	Optimisations and unsafe code	110
18.7	Use Cases & Applications	110
18.8	Worked Example: Tracing a GC Pass	111
18.9	Code Snippet: Pinning Memory for Native Interop	111
18.10	Practical Notes	111
19	C# Fundamentals — Syntax, Types, Methods, Arrays	113
19.1	C# Fundamentals and Syntax	113
19.1.1	The keyword that should usually scare you: <code>goto</code>	113
19.2	Variable Naming	114
19.3	Data Types and Conversion	114
19.3.1	Float to double vs double to float — “you’re losing fidelity”	114
19.3.2	<code>Convert.ToInt32</code> vs <code>int.Parse</code> — the exception-vs-zero choice	114
19.4	Methods and Parameters	115
19.4.1	Why <code>ref</code> exists — “behaves like a pointer”	115
19.4.2	<code>out</code> — the deliberate “not initialised yet” contract	115

19.5	Operators and Mathematical Functions	116
19.5.1	Operator precedence (high $\rightarrow$ low)	116
19.5.2	Math, randoms, dates	116
19.6	Arrays	116
19.6.1	Declare vs initialise — two separate steps	117
19.6.2	for with <code>.Length</code> vs <code>foreach</code>	117
19.6.3	Multidimensional and jagged arrays	117
19.6.4	<code>params</code> — not the same as overloading	117
19.7	Exception Handling	118
19.7.1	Multiple <code>catch</code> blocks — “you can overload your catches”	118
19.8	Use Cases & Applications	119
19.9	Worked Example: TryParse Pattern	119
19.10	Worked Example: Copy-by-Reference Pitfall	119
19.11	Operator Precedence Cheat Sheet	119
20	C# Strings, Structures & Collections (with Embedded Review)	121
20.1	C# Strings	121
20.1.1	“Strings are completely immutable”	122
20.1.2	Three kinds of string literal	122
20.1.3	String interpolation — the <code>\$</code> prefix	123
20.1.4	Quick reference for the main string methods	123
20.2	C# Structures (struct)	123
20.2.1	Struct vs array — “different data types in one place”	123
20.2.2	The all-or-nothing constructor rule	124
20.3	Collection Classes	124
20.3.1	<code>ArrayList</code> — “a more flexible array”	124
20.3.2	<code>Dictionary</code> — “it’s like a bunch of tuples”	124
20.4	Embedded Review: MCU Architecture & Operation	125
20.5	Embedded Review: MCU Memory Types	125
20.6	Embedded Review: Digital Communications	125
20.7	Embedded Review: Software Design and Estimation	126
20.8	Use Cases & Applications	126
20.9	Worked Example: Word Counter	126
20.10	Struct vs Class	126
20.11	Embedded Cross-Reference	126
21	GUI Design & Event-Driven Programming (Windows Forms)	129
21.1	Introduction to the GUI	129
21.2	Visual Studio Toolbox & Common Controls	130
21.3	Containers and Non-Visual Controls	130
21.3.1	Common container controls — when to reach for each	130
21.3.2	Non-visual controls — “things that happen in the background”	130
21.4	Data Handling and Conversion	130
21.5	Event-Driven Programming	131
21.5.1	“Windows doesn’t know when things are going to happen”	131
21.5.2	Name the controls <i>before</i> you generate handlers	131
21.6	Error Handling and Dialogs	131
21.6.1	Calculator evolution — the lecture’s three-step refactor	131
21.7	Advanced: Multiple Forms & Encapsulation	133
21.8	Use Cases & Applications	133
21.9	Worked Example: Two-Number Calculator	133
21.10	Encapsulating Cross-Form Access	134

21.11	Common Pitfalls	134
22	File Handling & Serial Ports	136
22.1	File Types and Streams	136
22.1.1	“You can’t just create a file — you need to create a stream first”	137
22.1.2	Text: <code>Write</code> vs <code>WriteLine</code>	137
22.1.3	Reading binary — and why audio is sampled at 44.1 kHz	137
22.2	C# Classes for File Handling	138
22.3	Windows Dialog Boxes	138
22.4	Directory and Path Operations	138
22.5	Exception Handling in File Operations	138
22.6	Serial Port Communication	138
22.6.1	Programmatic vs graphical serial port setup	139
22.6.2	Blocking vs the <code>DataReceived</code> event	139
22.6.3	<code>ReadExisting</code> — the buffer-flushing read	139
22.7	Endianness in Multi-Byte Data	139
22.8	Use Cases & Applications	139
22.9	Worked Example: Open + Read a Text File	140
22.10	Worked Example: Event-Driven Serial Reception	140
22.11	Endianness Conversion	140
22.12	FileMode Decision Table	140
22.13	Image Controls — the <code>PictureBox</code>	140
22.13.1	Picking a file format — GIF / JPEG / BMP / PNG	141
22.14	2D Graphics — the <code>System.Drawing</code> namespace	141
22.14.1	The coordinate system: top-left is (0,0); +Y is down	141
22.14.2	Pens and brushes — outlines vs fills	141
22.14.3	Paint events — persistent vs one-shot drawing	142
22.15	Debugging in Visual Studio	142
22.15.1	Three half-serious debugging attitudes (lecturer’s slide)	142
22.15.2	Breakpoints	142
22.15.3	Inspect: variables, locals, call stack	142
22.15.4	Step in / step over / step out	143
22.15.5	Conditional breakpoints — “best friend” for long loops	143
22.15.6	Logging to the Output window	143
23	Week 7 — Memory Layout & OOP Fundamentals	145
23.1	Why Memory Is the Heart of OOP	145
23.2	Program Memory Management	146
23.2.1	Memory segmentation	146
23.2.2	The stack	146
23.2.3	The heap	146
23.3	Stack vs Heap Comparison	146
23.4	OOP Variable Vernacular	146
23.4.1	Value vs reference types	146
23.4.2	Mutability	147
23.4.3	Type declaration	147
23.5	OOP Philosophy and Accessibility Modifiers	148
23.5.1	Accessibility modifiers	148
23.6	Logical vs Physical Organisation	148
23.7	Class Components in C#	148
23.8	Use Cases & Applications	149
23.9	Worked Example: Stack Overflow Demonstration	149

23.10	Mutability Cheat Sheet	149
23.11	Practical Notes	149
23.12	Lecture Notes: Memory Layout in Practice	150
23.13	Stack Mechanics — Push / Pop / Stack Pointer	150
23.13.1	In-class walkthrough: how the stack actually grows	150
23.13.2	Why the area between stack and heap is shared	151
23.13.3	Worked example: stack frames during nested calls	151
23.14	Recursion — Why Embedded Engineers Avoid It	151
23.14.1	In-class framing: “recursion is the bee’s knees — until it isn’t”	151
23.15	Heap Behaviour — Fragmentation, GC, malloc	152
23.15.1	In-class framing: “the heap is a free-for-all”	152
23.15.2	malloc vs the .NET runtime — “we don’t worry about malloc”	153
23.15.3	The garbage collector in one paragraph	153
23.15.4	Deconstructors — the other half of automatic cleanup	153
23.16	Threading and the Stack	153
23.16.1	In-class clarification: “interleaving, not parallel”	153
23.17	Stack vs Heap — One-Page Comparison	154
24	Week 9 — Advanced OOP & C# Conventions	157
24.1	Naming and Language Conventions	157
24.1.1	Pascal Case	157
24.1.2	Camel Case	157
24.1.3	Identifier rules	158
24.2	Static Behaviour	158
24.3	Inheritance and Reuse	158
24.4	Polymorphism and Method Overriding	158
24.5	Design Constraints and Encapsulation	158
24.6	Enumerations	158
24.7	Use Cases & Applications	159
24.8	Naming Convention Quick Reference	159
24.9	Worked Example: Method Overriding	159
24.10	When to Use What	160
25	Week 10 — .NET & C# Concepts	162
25.1	The .NET Framework	162
25.1.1	.NET Framework vs Modern .NET	163
25.2	Common Language Runtime	163
25.3	Automatic Memory Management & Generational GC	163
25.3.1	Generations	163
25.4	Namespaces and the Class Library	163
25.5	C# Concepts and Data Types	163
25.6	Structures and Collections	164
25.7	Graphical User Interfaces	164
25.8	Use Cases & Applications	164
25.9	Worked Example: Boxing Cost	164
25.10	Quick GC Mental Model	165
25.11	JIT vs AOT	165
26	Week 11 — File Handling, Serial Ports, Graphics & Debugging	167
26.1	File Handling and Serial Ports	167
26.1.1	SerialPort communication	167
26.2	Graphics and Plotting	168

26.3 Debugging in Visual Studio	168
26.4 OOP / C# Review Concepts	168
26.5 Use Cases & Applications	168
26.6 Worked Example: Logging Serial Data to a File	169
26.7 Plotting Live Data on a PictureBox	169
26.8 Debugging Cheat Sheet	170

How to use this resource

This document is the LaTeX-rendered companion to the Google Drive `COURSE OVERVIEW` folder. Each lecture follows the same layout:

- **Summary** — a one-paragraph orientation.
- **Topics Covered** — the headings within the lecture.
- **Sections** — the lecture content, organised into the same topical structure across every lecture.
- **Use Cases / Worked Examples / Formulas / Code** — the learning-resource extensions: when to apply each idea, with calculations and concrete C / C# snippets.
- **Key Definitions** — terminology you can revisit before exams.
- **Sources** — the original lecture PDFs.

Each lecture file under `lectures/` can be compiled standalone via ‘`pdflatex`’ thanks to the `subfiles` package, or rolled up here into a single PDF by compiling `course-overview.tex`.

Chapter 1

Course Introduction & Embedded Systems Foundations

ENEL712 Embedded Systems Design — Lecture 1

Summary

Foundational overview of embedded systems: AT90USB-style 8-bit AVR architecture, memory hierarchy, communication protocols, real-time/resilience principles, and hardware/software partitioning.

Topics Covered

- MCU architecture fundamentals (RISC, pipelining, Harvard vs Von Neumann)
- Memory types and constraints (Flash, SRAM, EEPROM)
- Communication protocols (UART, SPI, I2C)
- Real-time systems and defensive software
- Hardware/software partitioning

1.1 MCU Architecture Fundamentals

The Atmel AT90USB series uses the 8-bit AVR RISC architecture.

1.1.1 RISC (Reduced Instruction Set Computer)

- Large register file (32 general-purpose registers) and a simplified instruction set.
- Most instructions execute in a single clock cycle, enabling up to 16 MIPS at 16 MHz.
- Load-store architecture: operations occur only on registers, minimising slower SRAM traffic.

1.1.2 Two-Stage Pipelining

- While one instruction is executed, the next is fetched from Flash, allowing effective single-cycle execution.
- Branch instructions or interrupts flush the pipeline, costing 1–4 cycles.

- For time-critical code (e.g., ISRs), branching should be minimised.

1.1.3 Harvard vs. Von Neumann

- AVR uses a Harvard architecture: separate buses for program (Flash) and data (SRAM).
- Allows simultaneous instruction fetch and data access — avoids the Von Neumann bottleneck.
- Reason for using PROGMEM to keep constants in Flash and save SRAM.

1.2 Memory Types and Constraints

Embedded systems must operate within strict memory constraints, requiring careful management of three primary memory types.

Memory	Size	Volatile?	Use	Key constraint
Flash	128 KB	No	Program code & constants	100,000 write cycles; use PROGMEM for tables
SRAM	8 KB	Yes	Variables, stack, buffers	Stack grows down from 0x20FF; risk of overflow
EEPROM	4 KB	No	Settings & calibration	~3.4 ms/byte write; 100,000 cycles/byte

EEPROM is unsuitable for frequent logging or writes inside time-critical ISRs. For data written more than once a day, use external storage or wear leveling.

1.3 Communication Protocols

1.3.1 UART (Asynchronous)

- Simple point-to-point serial using two wires (TX/RX).
- Both sides agree on a baud rate; frame = START (0), 8 data, STOP (1).
- Requires <2% clock error to avoid corruption — timing errors accumulate.

1.3.2 SPI (Synchronous)

- Fast, full-duplex; minimum four wires (SCK, MOSI, MISO, CS).
- Ideal for SD cards/displays; needs a CS line per slave.
- Master and slave must match Clock Polarity (CPOL) and Clock Phase (CPHA) modes.

1.3.3 I²C (Synchronous)

- Two-wire bus (SCL/SDA) supporting up to 127 devices via addressing.
- Open-drain lines with pull-up resistors (e.g., 4.7 k Ω).
- Transaction = START → 7-bit address → R/W bit → ACK from slave.

1.4 Advanced Concepts and Resilience

- Real-Time Systems: deterministic, bounded execution. Hard real-time (e.g., automotive) must meet all deadlines; standard OSs introduce scheduler jitter.
- Defensive Software: checksums for data integrity, watchdog timer (WDT) to recover from runaway code.
- Hardware/Software Partitioning: prefer built-in hardware (Timer PWM, Input Capture, DMA) over software loops to maximise CPU throughput.

1.5 Use Cases & Applications

An 8-bit AVR like the AT90USB shines when you need deterministic timing, low cost, and tiny power budgets — not raw throughput. Typical embedded applications:

- Battery sensor nodes (e.g., the vineyard weather station): MCU sleeps for seconds, wakes via WDT, samples I²C sensors, sends a packet over UART/wireless, then sleeps.
- Real-time motor control: tight ISRs that must hit deadlines every PWM cycle — RISC + 2-stage pipeline gives predictable single-cycle execution.
- USB peripheral devices: AT90USB has a built-in USB peripheral, so it's a good fit for HID gadgets (keyboards, custom controllers).
- Safety-critical glue logic: a small MCU + watchdog timer monitoring a larger system, ready to reset or report on failure.

1.5.1 When NOT to use an 8-bit AVR

- FPU-heavy DSP, audio, or PID with float — pick a Cortex-M4 with FPU.
- More than ~128 KB code or ~8 KB RAM — pick an ARM Cortex-M with more memory.
- Full-speed USB / Ethernet / TCP-IP — pick an ARM Cortex-M or an MPU running Linux.

1.6 Worked Examples

1.6.1 Pipeline flush cost

An ISR services an event every 100 μ s on a 16 MHz AVR. The ISR contains 30 instructions, of which 4 are branches that each cost a 2-cycle pipeline flush.

- Base instructions: 30 cycles.
- Pipeline flush penalty: $4 \times 2 = 8$ extra cycles.
- Total: 38 cycles $\rightarrow 38 / 16e6 \approx 2.375 \mu$ s per ISR — about 2.4 % CPU load.
- Lesson: collapsing branches inside hot ISRs reclaims ~20 % of the routine.

1.6.2 EEPROM endurance

EEPROM cell endurance is 100,000 cycles per byte. Logging a 16-byte record once a minute would burn through that in ~70 days. Either drop to once an hour, write to external SPI flash, or implement wear leveling that rotates the write address across, say, 100 EEPROM positions for ~7,000 days.

1.7 Code Snippets

1.7.1 Storing a constant in Flash with PROGMEM

```
#include <avr/pgmspace.h>

const uint8_t sineLUT[256] PROGMEM = { /* ... */ };

uint8_t sample(uint8_t i) {
    return pgm_read_byte(&sineLUT[i]); // reads from Flash, not SRAM
}
```

1.7.2 Petting the watchdog

```
#include <avr/wdt.h>

int main(void) {
    wdt_enable(WDTO_500MS); // reset MCU if not petted within 500 ms
    for (;;) {
        do_work();
        wdt_reset(); // "pet the dog"
    }
}
```

Key Definitions

MIPS

Millions of Instructions Per Second; a measure of CPU throughput.

Pipeline Flush

Performance cost when a branch/interrupt forces the CPU to discard prefetched instructions (1–4 cycles).

Modified Harvard

Architecture that separates program and data buses but allows some cross-access (e.g., PROGMEM).

Open-Drain

Bus type used in I²C where devices only pull lines low; requires pull-up resistors.

Non-Volatile

Memory that retains data when power is removed (Flash, EEPROM).

Volatile

Memory that loses data when power is removed (SRAM).

PROGMEM

C directive that stores constant data in Flash instead of consuming SRAM.

Von Neumann Bottleneck

Limitation of single shared bus that blocks simultaneous fetch and data access.

Watchdog Timer (WDT)

Hardware timer that resets the MCU if not 'petted' periodically; recovers from runaway.

ISR

Interrupt Service Routine; function executed in response to a hardware signal.

Stack Overflow

Fault when downward-growing stack collides with upward-growing globals.

MIPS Throughput

Approximation of how many simple instructions a CPU finishes per second; AT90USB peaks around 16 MIPS at 16 MHz.

Wear Leveling

Spreading EEPROM/Flash writes across many cells so no single cell hits its endurance limit prematurely.

RTC

Real-Time Clock — low-frequency oscillator (often 32.768 kHz) used to keep wall-clock time and wake from deep sleep.

Sources

- Lecture 1.pdf

Chapter 2

Microcontroller Operation & Architecture

ENEL712 Embedded Systems Design — Lecture 2

Summary

How an MCU executes instructions: the fetch-decode-execute cycle, core CPU components, Harvard architecture, pipelining, and the RISC vs CISC trade-off.

Topics Covered

- Microcontroller overview & integrated components
- Fetch–Decode–Execute cycle
- Core CPU components
- Architecture and performance concepts
- RISC vs CISC comparison

2.1 Overview of a Microcontroller

A microcontroller (MCU) is a complete computer on a single IC, integrating the CPU core, memory (RAM, ROM, EEPROM), ADCs, timers, and I/O peripherals.

- Integration reduces external components and simplifies the power supply (single 3.3 V / 5 V rail).
- Enables deterministic real-time behaviour and low-power modes.

2.2 Microcontroller Operation: Fetch–Decode–Execute

- Power-Up: components disabled while the supply stabilises and the oscillator settles (milliseconds).
- Initialization: reset releases, CPU starts at the reset vector, firmware initialises peripherals and SFRs.

- Execution Cycle: PC starts at zero; instructions are fetched, decoded, executed; PC increments and the cycle repeats millions/billions of times per second.

2.3 Core CPU Components

Component	Role
Program Counter (PC)	Holds address of next instruction; increments after fetch, jumps on branches/calls/interrupts.
Instruction Register (IR)	Temporary register holding the fetched instruction during decode and execute.
Instruction Decoder (ID)	Translates instruction bits into control signals (registers, ALU operation).
Arithmetic Logic Unit (ALU)	Performs integer arithmetic (add/sub/multiply) and logic (shift/rotate) operations.
General Purpose Registers (GPRs)	32 fast 8-bit registers giving the ALU single-cycle access.
Status Register (SREG)	Flags (zero, carry, negative, overflow) and Global Interrupt Enable for conditional branches.

2.4 Architecture and Performance Concepts

2.4.1 Harvard Architecture

Used by the AT90 AVR. Separates program and data memory with dedicated buses, enabling simultaneous instruction fetch and data access.

2.4.2 Pipelining

- Two-stage AVR pipeline overlaps fetch and execute so most instructions complete every clock cycle.
- Branches invalidate the prefetched instruction, forcing a flush and refill that costs extra cycles.

2.5 RISC vs. CISC Comparison

Feature	RISC	CISC
Example	AVR, ARM	x86, 8051
Instruction set	Simple, uniform, mostly single-cycle	Complex, varied, often multi-cycle
Pipelining	Highly effective due to uniform timing	Harder; requires complex hardware
Power / hardware	Lower power, simpler hardware	Complex hardware, can have denser code

2.6 Use Cases & Applications

- Boot-from-reset behaviour: every embedded product walks through the same Power-Up → Initialization → Execution sequence on reset; understanding it helps debug 'why does my MCU appear stuck for 50 ms after power-on?' (oscillator stabilisation).
- Performance estimation: knowing the AVR is single-cycle per instruction lets you back-of-the-envelope timing for ISRs and tight loops in cycles, not seconds.

- Driver writing: SFRs are how you talk to peripherals — every write to a UART/SPI/I²C register is a side-effect on a memory-mapped SFR.
- Conditional code in C compiles to flag-based branches (zero, carry, overflow) read out of SREG — useful when reading disassembly.

2.7 Worked Example: Trace of a Simple Instruction

Consider ADD R16, R17 on an AT90 (8-bit AVR). Walk through one fetch–decode–execute cycle.

- Fetch: PC points at the instruction in Flash; the 16-bit opcode is loaded into the IR; the next instruction is prefetched (pipeline stage 1).
- Decode: ID interprets the opcode as 'ALU add, source R17, destination R16'.
- Execute: ALU reads R16 and R17, sums them, writes the result back to R16, and sets SREG flags (Z if zero, C if carry-out, N if MSB set).
- PC increments to the next instruction; total elapsed time at 16 MHz $\approx$ 62.5 ns.

2.8 Practical Notes

- Why startup takes time: the crystal oscillator typically needs several thousand cycles to stabilise — datasheets specify this as 'startup time from reset' (often 65 ms on AVRs).
- Why some 'simple' C statements compile to many instructions: 16- or 32-bit operations on an 8-bit ALU are decomposed into multiple 8-bit ADDs/SUBs with carry propagation through SREG.
- When SREG matters in C: the volatile keyword and the cli()/sei() pair around shared variables — they suppress interrupts so SREG.I doesn't change underneath you.

Key Definitions

Microcontroller (MCU)

Single IC integrating CPU, memory, and peripherals (timers, comm interfaces, ADC, GPIO).

Special Function Registers (SFRs)

Registers used to control and check the status of on-chip peripherals and I/O ports.

Harvard Architecture

Separate storage and buses for program and data memory.

Pipelining

Overlapping instruction fetch/decode/execute stages to increase throughput.

Program Counter (PC)

Register holding the address of the next instruction to fetch.

Instruction Register (IR)

Register holding a fetched instruction while it is being decoded.

Arithmetic Logic Unit (ALU)

CPU component executing arithmetic and logic operations.

Status Register (SREG)

Flag register storing results of the most recent operation (zero, carry, overflow, etc.).

Reset Vector

Fixed Flash address where the CPU starts executing after reset; usually 0x0000 on AVR.

Memory-Mapped Register

Hardware register that lives at a normal data address so it can be accessed with regular load/store instructions.

Cycle Budget

Number of clock cycles available for a periodic task; ISR length must be $< (\text{period} \times \text{clock frequency})$.

Sources

- Lecture 2.pdf

Chapter 3

MCU Memory Architecture & Clocking

ENEL712 Embedded Systems Design — Lecture 3

Summary

Detailed look at AVR memory architecture (Flash, SRAM, EEPROM), the SRAM memory map and stack, the bus system, and clock-source selection (with the UART accuracy constraint).

Topics Covered

- Architectural fundamentals (Harvard vs Von Neumann)
- Microcontroller memory types
- SRAM usage, memory map, and the stack
- Bus system
- System clock architecture & UART accuracy constraint

3.1 Architectural Fundamentals

- Harvard: physically separate program and data memory and buses, enabling concurrent access and pipelined fetch-execute.
- Constants must be placed in program memory using `PROGMEM` to avoid wasting SRAM.
- Von Neumann: shared memory and bus; introduces the Von Neumann bottleneck where the CPU must wait for memory.

3.2 Microcontroller Memory Types

Memory	Size (AT90USB128)	Purpose	Key characteristics
Flash	128 KB (64K × 16-bit)	Non-volatile program code and constants	≥100,000 erase/write cycles; use <code>PROGMEM</code> and <code>pgm_read_byte()</code> to save SRAM
SRAM	8 KB (0x0100–0x20FF)	Run-time variables, buffers, stack	Volatile; no refresh needed; preceded by GPRs (0x00–0x1F) and I/O regs (0x20–0xFF)
EEPROM	4 KB	Persistent settings & calibration	Byte-addressable; ~3.4 ms/byte write; ~100,000 cycles/byte; avoid frequent or ISR writes; use wear leveling

3.3 SRAM Usage, Memory Map, and the Stack

- Variables grow upward from 0x0100; the stack grows downward from 0x20FF.
- Stack: LIFO buffer for local variables and return addresses; the 16-bit Stack Pointer decrements on push, increments on pop.
- Avoid the C heap (`malloc`): leads to fragmentation, non-deterministic timing, and stack-overflow risk.

3.4 Bus System

Bus	Carries
Address bus	Memory address for instruction or data access
Data bus	Data (8-bit on AT90 series) between ALU, memory, and peripherals
Control bus	Read, write, and enable lines that control data flow

3.5 System Clock Architecture

Clock Source	Accuracy	External components	compo-	Use case
Low-Power Crystal Oscillator	≈ ±20–50 ppm	Crystal + 2 capacitors		Required for UART, USB, accurate timing/PWM (<2 % error)
Calibrated Internal RC	≈ ±10 %	None		Quick prototyping with no accurate timing/comms needs
Low-Frequency Crystal	≈ ±5–20 ppm	Tiny crystal		RTC and ultra-low power wakeup

3.5.1 Clock Accuracy: The UART Constraint

UART is sensitive because TX and RX have separate clocks and must agree on a baud rate. Internal RC oscillator drift ($\pm 10\%$) causes the receiver to sample data bits incorrectly after just a few bits — accurate UART requires a crystal so baud-rate error stays below $\approx 2\%$.

3.6 Use Cases & Applications

- Choosing a clock source: USB and UART force a crystal because they need $< 2\%$ timing error; idle prototyping can use the internal RC.
- Low-power design: the low-frequency 32.768 kHz crystal lets the MCU keep an RTC running on tens of nA in deep sleep.
- Calibration data storage: EEPROM holds factory trims (e.g., ADC offset, sensor gain) that survive reflashing.
- Stack analysis: knowing SRAM ends at 0x20FF lets you place a canary value below the maximum observed stack and detect overflow.

3.7 Formulas & Calculations

3.7.1 UART baud-rate accuracy budget

A UART frame is 10 bits (1 start + 8 data + 1 stop). The receiver samples each bit near its centre. After the start bit, accumulated timing error must stay below half a bit period at the last bit.

- Allowed error per bit $\approx 0.5 / 9 \approx 5.5\%$.
- Combined transmitter + receiver tolerance must fit inside this — so each side is typically held to $\leq 2\%$.
- An internal RC at $\pm 10\%$ drifts by far more than the budget; a crystal at ± 50 ppm is essentially perfect.

3.7.2 EEPROM lifetime with wear leveling

Endurance L cycles per cell, write rate W writes per second, N cells dedicated to the data $\rightarrow$ expected lifetime $\approx N \times L / W$ seconds.

3.8 Memory Map Cheat Sheet (AT90USB128)

Region	Address range	Notes
GPRs	0x0000–0x001F	32 × 8-bit registers visible as memory
I/O Registers	0x0020–0x00FF	SFRs accessed via IN/OUT or memory ops
Extended I/O	0x0100–0x01FF	Additional peripheral SFRs on bigger AVRs
SRAM (data)	0x0100–0x20FF	Globals grow up; stack grows down from 0x20FF
Flash (program)	0x0000–0x1FFFF	Word-addressed (16-bit), 64K words = 128 KB
EEPROM	0x0000–0x0FFF	Separate address space accessed via SFRs

Key Definitions

Harvard Architecture

Separate buses and memory spaces for program (Flash) and data (SRAM), allowing concurrent access.

Von Neumann Architecture

Single shared bus and memory for program and data; limited by the Von Neumann bottleneck.

Flash Memory

Non-volatile program storage with finite write endurance (~100k cycles).

SRAM

Volatile RAM (8 KB on AT90USB128) for runtime variables, buffers, and the stack.

EEPROM

Non-volatile, byte-addressable memory (4 KB) for persistent configuration.

Stack

LIFO SRAM region for return addresses and locals; grows downward from the top.

PROGMEM

AVR-GCC mechanism to force constants into Flash and save SRAM.

Wear Leveling

Strategy to rotate write addresses across EEPROM to avoid hitting endurance limits.

Stack Overflow

Downward-growing stack collides with upward-growing globals; corrupts data.

Von Neumann Bottleneck

Performance limit caused by a single shared memory bus.

Stack Canary

Sentinel byte placed below the deepest expected stack point; corruption signals stack overflow.

Crystal PPM

Parts-per-million frequency tolerance; 50 ppm = 0.005 %.

Heap Fragmentation

Free memory broken into many small unusable holes after repeated malloc/free cycles.

Sources

- Lecture 3.pdf

Chapter 4

Advanced MCUs and Microprocessors (MPUs)

ENEL712 Embedded Systems Design — Lecture 4

Summary

Moving beyond 8-bit MCUs: hybrid 8/16-bit XMEGAs, hybrid DSP/MCUs, the ARM Cortex hierarchy, and how MCUs differ from MPUs in integration, performance, and complexity.

Topics Covered

- Transitioning beyond 8-bit MCUs
- ARM architecture and Cortex hierarchy
- MCUs vs MPUs
- Selection advice for advanced systems

4.1 Transitioning Beyond 8-bit MCUs

When projects require higher raw throughput (trig, multiplication), better timing, or complex comms, 8-bit MCUs may not be enough.

4.1.1 Atmel XMEGA MCUs

- Hybrid 8/16-bit: 8-bit data bus with 16-bit ALU; better integer performance for values >255.
- Event System lets peripherals trigger each other directly, bypassing the CPU.
- DMA handles automatic data movement for ADC or comms, freeing the CPU.

4.1.2 Hybrid DSP/MCUs

Devices like TI's C2000 family combine DSP performance with MCU ease-of-use. 32-bit, optimised for high-speed real-time control (motor control, digital power supplies, solar).

4.2 The ARM Architecture

- ARM Holdings licenses its ISA and architecture; manufacturers integrate their own peripherals.
- RISC-based design uses fewer transistors → lower power and heat → dominant in mobile.

4.2.1 Cortex-M Hierarchy

Family	Notes
Cortex-M0+	Lowest power; simple IoT; replaces 8-bit MCUs
Cortex-M3	General embedded control; hardware divide
Cortex-M4	DSP and FPU; motor drives and signal processing
Cortex-M7	High-performance real-time; dual-issue pipeline
Cortex-A	MMU enables full OSs like Linux (e.g., Raspberry Pi)

Choose ARM over a simpler MCU (e.g., AT90USB) when you need FPU math, more than 128 KB Flash or 8 KB RAM, or faster comms (Full-Speed USB, Ethernet).

4.3 MCUs vs. MPUs

Feature	Microcontroller (MCU)	Microprocessor (MPU)
Integration	CPU, memory, I/O on one chip	CPU only; needs external DRAM/NAND/I/O/oscillator
Memory	Up to ~2 MB Flash, 8 KB RAM	External: GBs available
Performance	Real-time OS and simple UIs; deterministic IRQ latency	Full OS, Gigabit Ethernet, GUIs (e.g., 400 DMIPS for Linux)
Real-time	Sub- μ s interrupt latency	OS scheduler adds jitter; no hard real-time
Complexity	Single-rail supply; few external components	Multi-rail supplies; high PCB complexity (6+ layers, impedance control)
Power / cost	Sub- μ A sleep; \$1–\$10	Higher power; \$10–\$100+

4.4 Selection Advice for Advanced Systems

- Use what you know unless the project demands a new architecture.
- If code doesn't fit, buy a bigger MCU rather than fighting extreme assembly optimisations.
- Hybrid solutions (e.g., TI F28M3X dual-core: ARM Cortex-M3 + C2000) prevent the OS interfering with high-speed control loops.
- MPUs shine when an OS like Linux can supply built-in drivers (TCP/IP, USB, video); bare-metal MPU work is a huge undertaking.

4.5 Use Cases & Applications

4.5.1 Where each device family lives

Device family	Typical product
AT90USB / 8-bit AVR	Battery sensor nodes, simple appliances, hobby USB gadgets
XMEGA (8/16-bit hybrid)	Higher-precision sensor frontends with DMA-driven ADC
Cortex-M0+/M3	IoT modules, fitness trackers, mid-range consumer products
Cortex-M4 + FPU	Motor drives, drones, audio DSP
Cortex-M7	Advanced industrial controllers, image preprocessing
Cortex-A (Linux MPU)	Single-board computers, set-top boxes, 7" smart-home controllers
TI C2000 / DSP-MCU	Solar inverters, power converters, sensorless motor control

4.6 Decision Framework

- If your code fits in <128 KB Flash and 8 KB RAM, real-time deadlines exist, and you don't need TCP/IP → an 8-bit/16-bit MCU is usually cheapest.
- If you need FPU / DSP, USB, Ethernet, or larger memory but still want hard real-time → ARM Cortex-M.
- If you need a full operating system (Linux), gigabit Ethernet, or a rich GUI → MPU; accept added complexity, power, and PCB cost.
- If you need both real-time control AND an OS → hybrid devices like TI F28M3X (M3 + C2000) keep concerns separate.

4.7 Worked Example: Picking a Chip for a Quadcopter Flight Controller

- Sensor fusion (gyro + accel + magnetometer) requires float math and 1–2 kHz update rates → Cortex-M4 with FPU is ideal.
- Logging to SD card and a 100 Mbps Ethernet uplink would push toward Cortex-M7 or a dual-core part.
- If you also want a desktop-style GUI for live telemetry on the device itself, Linux on a Cortex-A is justified.

Key Definitions

MCU

Single-chip computer integrating CPU, memory (Flash/RAM), and I/O peripherals.

MPU

CPU-focused chip needing external memory, I/O, and support components.

ARM

Acorn RISC Machines; licensed ISA dominant in mobile due to low power per watt.

RISC

Reduced Instruction Set Computer; small, simple instruction set with low transistor count.

DMA

Direct Memory Access; hardware that moves data between peripherals and memory without CPU.

Event System

Peripheral-to-peripheral hardware triggering used in advanced MCUs (e.g., XMEGA).

DSP

Digital Signal Processor optimised for multiply-accumulate / signal processing.

FPU

Floating-Point Unit; coprocessor for high-speed floating-point arithmetic.

DMIPS

Dhrystone MIPS; CPU performance benchmark; Linux on ARM typically wants ≥ 400 DMIPS.

MMU

Memory Management Unit; required for general-purpose OSs (Linux/Windows) to provide virtual memory and process isolation.

Hybrid Core

Single chip combining a control-oriented core (e.g., Cortex-M3) with a real-time DSP core (e.g., C2000).

Sources

- Lecture 4.pdf

Chapter 5

Embedded Digital Communications A

ENEL712 Embedded Systems Design — Lecture 5

Summary

Introduction to embedded communications: parallel vs serial, asynchronous (RS-232/422/485), synchronous (SPI, I²C), and other notable protocols (CAN, PS/2).

Topics Covered

- Introduction to embedded communication
- Parallel vs serial communication
- Asynchronous serial protocols
- Synchronous serial protocols
- Other notable protocols

5.1 Introduction to Embedded Communication

Embedded systems frequently need MCUs to talk to other ICs, sensors, or PCs, transferring multiple bytes rather than single bits. Hundreds of protocols exist, but ~6 are common across most MCUs.

5.2 Parallel vs. Serial Communication

- Parallel: multiple bits transmitted concurrently across a bus.
- Serial: one bit at a time over a single wire.

5.2.1 Legacy parallel interfaces

- PC Parallel Port (DB-25): 8 data pins + control lines; superseded by USB and Ethernet.
- IDE (Parallel ATA): 40-pin cable, 16 parallel data bits; superseded by SATA.

- LCD parallel: 4-bit or 8-bit data with 3 control lines and timing requirements.

5.3 Asynchronous Serial (RS-232 / RS-422 / RS-485)

- TX and RX have separate clocks and must agree on a baud rate.
- Frames synchronised by start and stop bits.

5.3.1 RS-232

- Three wires for full-duplex (Tx, Rx, GND).
- True RS-232 uses ± 15 V; TTL RS-232 (common in MCUs) uses 0–3.3 V or 0–5 V.
- Optional parity bit for single-bit error detection.
- Unbalanced (single-ended) signalling — sensitive to noise; short distances/lower rates.

5.3.2 RS-422 / RS-485

- Differential signalling for noise immunity, range >1 km, and higher data rates.
- RS-485 is multi-point — up to 32 transceivers on a single bus.

5.4 Synchronous Serial (SPI, I²C)

5.4.1 SPI (Serial Peripheral Interface)

- Four wires: SCK, MOSI, MISO, SS.
- Full-duplex; master generates clock and initiates all transfers.
- Highly flexible/fast but interpretation is software-dependent.
- Four modes from CPOL/CPHA combinations (Mode 0–3) define when the master clocks and when data is sampled.

5.4.2 I²C (Inter-Integrated Circuit)

- Two wires: SCL and SDA.
- Multi-master with up to 1008 slaves via 7-bit or 10-bit addressing.
- Open-collector ports require pull-up resistors to define the high state.
- Half-duplex; mandatory ACK bit after every 8 bits ensures reliability.

5.5 Other Notable Protocols

- CAN (Controller Area Network): asynchronous, message-based, fault-tolerant; used in automotive/industrial. Differential signalling and non-destructive bitwise arbitration by message ID priority.
- PS/2: synchronous protocol similar to I²C; older keyboards/mice.

5.6 Use Cases & When to Pick What

Protocol	Pick when...	Typical example
UART/RS-232	Two devices, modest rate, simple framing	MCU ↔ PC debug terminal, GPS module
RS-485	Long bus, many drops, noisy environment	Industrial sensors on a shop floor
SPI	One master, fast point-to-point, full-duplex	SD card, OLED display, ADC
I ² C	Many small sensors on the same two pins	Temperature, humidity, EEPROM combo
CAN	Distributed control; messages, not addressing; tolerant of faults	Automotive ECUs, factory automation
PS/2	Legacy keyboard/mouse interfaces	Retro computing

5.7 Formulas & Calculations

5.7.1 Effective vs. raw UART byte rate

Each byte costs 10 bits on the wire (8 data + start + stop). At baud rate B bps, effective byte rate $\approx B / 10$. So a 115,200 baud link gives $\approx 11,520$ bytes per second.

5.7.2 I²C ACK timing

An I²C transaction is at least 1 (start) + 7 (addr) + 1 (R/W) + 1 (ACK) + 8 (data) + 1 (ACK) + 1 (stop) = 20 bit periods. At 100 kHz that is 200 μ s per byte payload — slower than SPI but acceptable for sub-Hz sensors.

5.8 Worked Example: SPI Mode Picking

An SD card datasheet calls out SPI Mode 0 (CPOL = 0, CPHA = 0). That means the clock idles low and data is sampled on the rising edge. Setting your SPI peripheral to Mode 3 instead (CPOL = 1, CPHA = 1) is a classic 'works in the lab, dies in the field' bug: the card will respond to some commands but garble multi-byte transfers because the sample point is shifted by half a bit.

5.9 Code Snippets

5.9.1 AVR UART transmit one byte

```
void uart_putc(uint8_t c) {
    while (!(UCSROA & (1<<UDRE0))) ; // wait until tx buffer is empty
    UDRO = c;                          // write triggers transmission
}
```

5.9.2 Reading an I²C sensor

```
// 7-bit address 0x48, register 0x00
uint8_t value;
i2c_start();
i2c_write((0x48 << 1) | 0); // address + write bit
```

```
i2c_write(0x00);          // register pointer
i2c_repeated_start();
i2c_write((0x48 << 1) | 1); // address + read bit
value = i2c_read(false);   // NACK = stop after this byte
i2c_stop();
```

Key Definitions

Baud Rate

Rate at which symbols (bits/signals) are transmitted per second on a serial link.

Full-Duplex

Channel that carries simultaneous, two-way traffic (e.g., SPI).

Half-Duplex

Channel that allows two-way comms one direction at a time (e.g., I²C, RS-485).

Parity Bit

Optional bit added to detect single-bit errors.

Differential Signalling

Two complementary lines whose difference carries the signal; rejects common-mode noise (RS-422/485, CAN).

Open-Collector

Output that pulls low or floats; needs a pull-up to define the high state (e.g., I²C).

Arbitration

Bus access process used in multi-master systems (like CAN) based on message priority.

Bit Period

Duration of one bit at the current baud/clock rate; $1 / B$ seconds.

Repeated Start

I²C primitive that issues a new START condition without an intervening STOP — used to switch from write to read on the same device.

Frame Overhead

Bits added to each character for synchronisation/error detection (e.g., start, parity, stop) that lower effective throughput.

Sources

- Lecture 5.pdf

Chapter 6

Embedded Digital Communications B

ENEL712 Embedded Systems Design — Lecture 6

Summary

Beyond simple serial: differential signalling, cable construction, USB topology and stacks, and Ethernet implementation on MCUs.

Topics Covered

- Beyond basic serial protocols
- Single-ended vs differential signalling
- Cable construction and distance
- Universal Serial Bus (USB)
- Ethernet on MCUs

6.1 Beyond Basic Serial Protocols

- SPI/TTL RS-232 typically max ~125 kB/s and rarely sustain that beyond 0.5 m.
- Wireless options (Bluetooth, ZigBee) help with distance but high-bandwidth/long-distance needs require advanced modes.

6.2 Single-Ended vs Differential Signalling

6.2.1 Single-ended

- One signal line + common ground (TTL RS-232, SPI, I²C, PS/2).
- Sensitive to noise, prone to crosstalk, generates EMI.

6.2.2 Differential

- Two complementary lines; receiver subtracts them (USB, Ethernet, HDMI, RS-422/485).
- Common-mode noise cancels at the receiver; reduced EMI; supports higher frequencies.

6.3 Cable Construction and Distance

- Twisted pairs cancel induced EMI and reduce crosstalk; foil shielding blocks external interference.
- Higher-bandwidth cables (Cat.5 → Cat.7a) typically use more shielding.
- Maximum reliable distance is inversely proportional to data rate.
- On long, high-speed lines (incl. USB 3.0), terminating resistors must match the cable's characteristic impedance to absorb signal reflections.

6.4 Universal Serial Bus (USB)

- Host-controlled, tiered-star topology; up to 127 devices across 7 tiers.
- USB 1.0: Low Speed 1.5 Mbit/s, Full Speed 12 Mbit/s. USB 2.0: High Speed 480 Mbit/s. USB 3.0/3.1: SuperSpeed 5/10 Gbit/s.
- Wires: 1.0/2.0 use 4; 3.0 uses 11; USB-C uses 24.
- Enumeration: host detects, addresses, and reads descriptors from a new device.
- Endpoints: unidirectional 'pipes' for Control, Bulk, Interrupt, or Isochronous transfers.

6.4.1 Microcontroller implementation

- AT90USB has a USB peripheral that can act as host or device, with default Endpoint 0 plus 6 programmable endpoints.
- Manufacturers (e.g., Atmel) provide free software stacks (ASF-USB) to handle low-level register and protocol details.

6.5 Ethernet on Microcontrollers

- Most 8/16-bit MCUs lack the RAM and speed for native Ethernet.
- Implementations typically use a dedicated controller (e.g., Microchip ENC28J60) over SPI plus magnetics (transformers) for the RJ-45 socket.
- Writing the TCP/IP stack from scratch is a huge effort; using Linux on ARM or pre-built modules (e.g., XPort) is often more practical.

6.6 Use Cases & Applications

- USB: connect an MCU-based product to PCs without designing a driver — the standard HID class is enough for keyboards, joysticks, and many measurement instruments.
- Ethernet on MCU: factory IoT gateways that aggregate Modbus/CAN sensors and forward them to a server.
- RS-485: multi-drop industrial buses (e.g., 32 sensors on a single twisted pair across a building).
- Differential signalling: HDMI, USB 3+, PCIe, and MIPI in consumer electronics — same idea as RS-422 scaled up to GHz rates.

6.7 Formulas & Rules of Thumb

- Cable bandwidth $\times$ length is roughly constant: doubling the data rate halves the achievable distance.
- RS-485 rule of thumb: 1.2 km at 100 kbps, ~12 m at 10 Mbps.
- Termination: place a resistor matching the cable's characteristic impedance (typically 100–120 Ω) at each end of a long differential bus.
- USB cable length max: 5 m for USB 2.0 High Speed without a hub or active cable; 3 m for USB 3.0.

6.8 Worked Example: Why Long Single-Ended Cables Fail

Imagine running 5 m of unshielded TTL UART (single-ended) past a switching power supply. Each switching pulse couples ~50 mV of noise onto the signal line. With a 3.3 V signal, that 50 mV may not corrupt a steady-state level, but at the edges (where the receiver decides 0 vs 1) it shifts the perceived crossing point and shows up as random framing errors. Switch to RS-485 at the same baud and the receiver sees the noise on both wires equally — common-mode rejection cancels it out.

6.9 Practical Notes

- Don't run differential pairs across split planes — that breaks return-current paths and reintroduces common-mode noise.
- When using an external Ethernet controller (e.g., ENC28J60 + magnetics), keep the SPI bus to the MCU short — SPI is single-ended and noisy.
- USB-C's 24 wires include CC (configuration), power, multiple high-speed pairs, and SBU lines for alternate modes (DisplayPort/Thunderbolt) — most embedded designs only wire the USB 2.0 subset.

Key Definitions

Differential Signalling

Two complementary lines whose difference carries the signal; rejects common-mode noise.

Common-Mode Noise

Interference appearing equally on both lines of a differential pair; cancelled by the receiver.

Twisted Pairs

Conductors physically twisted to cancel induced EMI and reduce crosstalk.

Crosstalk

Coupling between adjacent wires causing interference.

Enumeration

USB host's process of detecting, addressing, and configuring a new device.

Endpoint

Logical buffer/pipe in a USB device for one direction of a particular transfer type.

Magnetics

Transformers in Ethernet ports providing isolation and EMI filtering.

TCP/IP Stack

Set of software protocols managing network communication and data routing.

Software Stack

Manufacturer library handling low-level details of a complex protocol (USB, TCP/IP).

Ethernet Controller

Dedicated IC handling the MAC/PHY layers so an MCU can connect to Ethernet.

RS-485

Differential, multi-point standard supporting long distances (up to 1.2 km).

Common-Mode Rejection Ratio (CMRR)

Receiver's ability to ignore signals appearing equally on both differential lines.

Characteristic Impedance

Effective resistance a long cable presents to a fast edge; mismatched ends cause reflections.

Active Cable

Cable with built-in repeaters/equalisers used to extend high-speed links beyond passive limits.

Sources

- Lecture 6.pdf

Chapter 7

Embedded Hardware Design

ENEL712 Embedded Systems Design — Lecture 7

Summary

End-to-end hardware design process for an embedded product, illustrated by a NZ vineyard remote weather station: I/O selection, MCU selection, and power-supply design.

Topics Covered

- Hardware design process
- I/O device selection
- Microcontroller selection
- Power supply design
- Switching vs linear regulators
- Next steps after hardware selection

7.1 The Hardware Design Process

Briefs are usually short and lacking technical detail (e.g., 'Design a remote weather station for NZ vineyards'). Designers must address MCU selection, power supply, components (SMD vs through-hole), PCB design, and interfacing.

First step: draw a block diagram on paper to visualise system components and their connections.

7.1.1 Vineyard weather station block components

- MCU (μC)
- Power supply and battery (LiPO)
- Loads: temperature, humidity, rainfall, sunlight sensors
- Wireless transceiver (T/R)
- Memory (e.g., EEPROM for settings)

- Display (optional, for debugging or local status)
- JTAG/ISP header for programming and debugging

7.2 I/O Device Selection

I/O devices are typically chosen before the MCU so the processor can support all required interfaces.

- Sensors (temperature, light, movement)
- Actuators (motors, solenoids, speakers — less relevant for the weather station)
- User input (buttons, keypads)
- Displays (LCDs, LEDs)
- Communications/storage (SPI, UART, external memory/SD cards)

7.2.1 Wireless module requirements

- Communication range ≥ 1 km
- Data rate ≥ 100 kbps
- Operating in the licence-free ISM band
- UART or SPI interface preferred
- Use a pre-configured module (e.g., ZigBee over UART) over a transceiver IC to avoid antenna/impedance design.
- Prefer digital sensor outputs over analogue for easier accurate reading.

7.3 Microcontroller Selection

7.3.1 I/O Tally

- # digital inputs/outputs
- # analogue inputs/outputs (ADC, DAC)
- # communication interfaces (UARTs, SPIs, I²Cs)
- # timers (PWM and Input Capture channels)

7.3.2 Weather station tally example

- 5× digital outputs (e.g., power-down lines, rainfall container empty signal, battery charger status)
- 3× analogue inputs (rainfall, sunshine, battery voltage)
- 1× UART (wireless ZigBee module)
- 2× I²C (temperature and humidity sensors, possibly shared bus)
- 1× SPI (external EEPROM)

7.3.3 Selected MCU example

ATmega 328P (picoPower): 32 KB Flash, 2 KB RAM, 1.8 V – 5 V operation, picoPower mode for long battery life. Verify all required pins (including ISP) are available on the chosen package.

7.4 Power Supply Design

- DC-to-DC supply must consider input/output voltages, noise sensitivity, current, and efficiency.
- Sleep modes are critical for the battery system — wake on WDT or RTC alarm.
- Build a current tally for the supply, regulators, and all loads.

7.4.1 Current tally example

Rail / total	Voltage	Current
5 V rail total	5 V	101 mA
3.3 V rail total	3.3 V	216 mA
Total (calculated)	—	317 mA
Total (with 20 % margin)	—	380 mA

3.3 V rail: MCU (10 mA), wireless T/R (205 mA), I²C sensors. 5 V rail: rainfall sensor and sunshine sensor.

7.5 Switching vs Linear Regulators

Regulator	Pros	Cons
Switching	High efficiency (>80 %); good for large Vin/Vout differences	Complex to design; introduces electrical noise
Linear	Simple and cheap	Inefficient; dissipates excess as heat

A common hybrid approach uses a switching regulator first (efficiency) followed by a linear regulator and decoupling caps (clean output).

7.6 Next Steps

After hardware selection, move to schematic design (e.g., Altium) following datasheets for tips and required external components. Software planning can begin in parallel — hardware is not required to start coding.

7.7 Use Cases & Applications

- Iterating from brief to schematic: the I/O tally → MCU choice → power tally pipeline applies to almost every battery-powered embedded product.
- Pre-configured wireless modules let small teams ship a wireless product without an RF specialist — the trade-off is per-unit cost.

- Hybrid switching + linear supplies are common in audio, sensor frontends, and any analog circuitry that is sensitive to switching ripple.

7.8 Formulas & Calculations

7.8.1 Battery life from current tally

Lifetime (h) $\approx$ Battery capacity (mAh) $\div$ Average current (mA). For varying loads, average current is the time-weighted mean across all operating modes.

7.8.2 Worked example: vineyard station

If the device wakes for 1 s every 60 s drawing 380 mA, and sleeps the rest at 50 μ A on a 500 mAh battery:

- Active duty cycle: $1 / 60 \approx 1.67\%$.
- $I_{avg} = 0.0167 \times 380 \text{ mA} + 0.9833 \times 0.05 \text{ mA} \approx 6.39 \text{ mA}$.
- Lifetime $\approx 500 / 6.39 \approx 78 \text{ h} \dots$ so we'd need to either cut active time, lower active current, drop the wake rate, or move to a bigger battery.

7.8.3 Sizing a 20 % current margin

$I_{target} = 1.20 \times \Sigma I_{max}(\text{component})$. Margin covers component tolerance, transient peaks (e.g., Wi-Fi TX bursts), and inrush.

7.9 Design Checklist

- Block diagram drawn before any chip is selected.
- I/O tally signed off against a candidate MCU pinout (and ISP/JTAG header verified).
- Current tally per voltage rail, with margin.
- Sleep state path: which oscillator is running? what wakes us? what is the residual current?
- Schematic captures all manufacturer-recommended decoupling, pull-ups, and crystal load capacitors.

Key Definitions

ADC

Analog-to-Digital Converter; turns continuous voltage into discrete digital values.

Current Tally

Sum of every component's max current (with margin) used to size each voltage rail.

I/O Tally

Comprehensive count of digital, analogue, communication, and timer peripherals required.

ISM Band

Industrial, Scientific, Medical bands (e.g., 900 MHz, 2.4 GHz) — licence-free for embedded wireless.

ISP

In-System Programming header for programming/debugging the MCU on-board.

Linear Regulator

Voltage regulator that drops excess voltage as heat; simple but inefficient.

Sleep Modes

Low-power MCU states that shut down peripherals to drastically reduce current.

Switching Regulator

Inductor-based regulator with high efficiency but more complexity and noise.

picoPower

Atmel marketing term for AVRs with very low active and sleep currents (down to $<1 \mu\text{A}$ in power-down).

LDO Dropout

Minimum $V_{in} - V_{out}$ the regulator needs to keep regulating; below this the output sags.

Decoupling Capacitor

Small capacitor near a chip's power pin that supplies transient currents and filters noise.

Sources

- Lecture 7.pdf

Chapter 8

Embedded Software Design A — Software Engineering

ENEL712 Embedded Systems Design — Lecture 8

Summary

Software engineering applied to embedded systems: the software lifecycle, requirements, abstraction and modelling (UML, FSMs), and testing.

Topics Covered

- Fundamentals of software engineering
- Software process activities
- Requirements engineering
- System modelling and abstraction
- Software testing

8.1 Fundamentals of Software Engineering

Software engineering covers all aspects of software production, from specification to maintenance. In modern embedded systems, software often dominates development cost and is more expensive to maintain than to build.

8.2 Software Process Activities

- Specification: define what the software must do and its constraints, with the customer.
- Development: design and program the software.
- Validation: confirm the software meets customer requirements.
- Evolution: modify the software to reflect market or customer change.

8.3 Requirements Engineering

8.3.1 Functional vs non-functional

Type	Describes
Functional	Services the system provides, reactions to inputs, and behaviour in particular situations.
Non-functional	System properties and constraints: timing, reliability, response time, storage, etc.

About 56 % of software defects originate in requirements. Fixing a scope change post-launch can cost up to 200× more than during initial analysis.

8.4 System Modelling and Abstraction

- Abstraction: ignore some properties to focus on the most important — top-down (add detail) or bottom-up (combine solved parts).
- UML: graphical notation for representing different system views to aid analyst/customer communication.
- State machine models: states as nodes, events as arcs; superstates expand into sub-state diagrams for complex behaviour.

8.5 Software Testing

Test type	Goal
Validation testing	Show the system meets requirements; every requirement should have ≥ 1 test.
Defect testing	Make the system perform incorrectly to expose bugs.

Testing reveals the presence of errors, but cannot prove their absence.

8.6 Use Cases & Applications

- Medical/automotive/aerospace embedded products legally must follow software-engineering processes (IEC 62304, ISO 26262, DO-178C).
- Even one-person hobby projects benefit from short specifications and state diagrams — they catch contradictions before code is written.
- Test-driven development on PC code can run in CI even if the final binary targets an MCU; mock hardware modules behind interfaces.

8.7 Worked Example: A Tiny Specification

Spec: 'door access controller'. Functional and non-functional requirements look like this:

- FR1: When a valid card is presented, the controller shall unlock the door for 5 s.
- FR2: After 3 invalid attempts within 60 s, the controller shall lock out for 5 minutes.

- FR3: All access events shall be logged to flash with a timestamp.
- NFR1: Card-to-unlock latency shall be < 500 ms.
- NFR2: Battery backup shall sustain operation for ≥ 24 h.

Each requirement maps to one or more tests (validation tests) and a state in the FSM. Defect testing then attacks the edges: bad card data, near-simultaneous reads, brown-out during a write.

8.8 State Diagram in Words

Sketch of the door controller as a hierarchical FSM:

- States: LOCKED, UNLOCKING, UNLOCKED, LOCKOUT.
- Events: VALID_CARD, INVALID_CARD, TIMER_EXPIRES.
- Transitions: LOCKED + VALID_CARD $\rightarrow$ UNLOCKING (start motor); UNLOCKING + TIMER_EXPIRES $\rightarrow$ UNLOCKED; UNLOCKED + TIMER_EXPIRES $\rightarrow$ LOCKED; LOCKED + 3 $\times$ INVALID_CARD $\rightarrow$ LOCKOUT.
- Superstate ANY + brown-out $\rightarrow$ SAFE (all locks engaged, log, halt).

8.9 Cost-of-Defect Curve

Industry data shows the cost of fixing a defect rises roughly an order of magnitude per phase: $\sim 1\times$ in requirements, $\sim 5\text{--}10\times$ in design, $\sim 10\text{--}40\times$ in code, $\sim 100\times$ in test, $\sim 200\times$ post-launch. Catching ambiguity in requirements is therefore by far the highest leverage activity.

Key Definitions

Software Engineering

Discipline covering all aspects of software production from spec to maintenance.

Abstraction

Ignoring some properties to focus on the essential parts of a problem.

Functional Requirement

Service or behaviour the system must provide.

Non-Functional Requirement

Constraint or property such as timing, reliability, or storage.

Validation Testing

Tests intended to demonstrate that the software meets its requirements.

Defect Testing

Tests intended to expose incorrect behaviour or bugs.

Requirement Traceability

Practice of linking every test back to one or more requirements (and vice versa).

Verification vs Validation

Verification: 'are we building the product right?' Validation: 'are we building the right product?'

Mock / Stub

Stand-in module that simulates a hardware peripheral or external service for desk-side testing.

Sources

- Lecture 8.pdf

Chapter 9

Embedded Software Design B — Architectural Design

ENEL712 Embedded Systems Design — Lecture 9

Summary

Software architectures for embedded systems: bottom-up vs top-down development, polling vs event-driven vs state machines vs RTOS, hardware/software partitioning, fault tolerance, and documentation.

Topics Covered

- Embedded software development fundamentals
- Development approaches (bottom-up vs top-down)
- Software architectures (polling, event-driven, FSM, OS, RTOS)
- Hardware/software partitioning
- Fault tolerance and defensive design
- Software documentation

9.1 Embedded Software Development Fundamentals

- Iterative process — designs are rarely correct first time, regardless of experience.
- Skills accrue through projects: C/C++ proficiency, peripheral knowledge, reusable templates.
- Tool selection depends on project requirements and engineer experience — no universal solution.
- Dominant languages: C (~82 %), C++ (~45 %), Assembly (~35 %).

9.2 Development Approaches

Approach	Pros	Cons
Bottom-Up	Quick coding; immediate hardware testing	No clear system picture; undefined interfaces; complexity creep
Top-Down	Architecture is defined first; fewer late changes; low-level modules become clear	Testing only starts mid-project

Starting any project without a clear architectural design is poor form and risky.

9.3 Software Architectures

9.3.1 Polling Loop

- Simplest strategy — loop checks all inputs and handles all actions.
- Delays stall the processor; wasted cycles, poor power efficiency.

9.3.2 Event-Driven

- Internal data events or external events (interrupts) trigger processing.
- CPU is free between events — can do other tasks or sleep.
- Hybrid systems often use interrupts for high-priority events plus polling for low-priority ones.

9.3.3 Finite State Machine (FSM)

- Program divided into a finite set of states; transitions on internal/external events.
- Widely used; typically implemented with a switch-case statement.

9.3.4 Operating System (OS)

- Manages resources and provides services for applications; embedded OSs are often cut-down PC OSs.
- Kernel is the core of the OS controlling networking, file systems, etc.
- Provides apparent parallelism via time-slicing; standard OSs do not guarantee timing.

9.3.5 Real-Time Operating System (RTOS)

- Designed for time-critical apps with deterministic execution.
- Soft real-time: timing not critical; Hard real-time: must strictly meet timing.
- Examples: FreeRTOS, SYS/BIOS. Standard Windows/Linux are not RTOSs.

9.4 Hardware/Software Partitioning

- Hardware: faster development for performance tasks; runs concurrently with the CPU; use built-in peripherals (DMA, Event System, comms ports).
- Software: lower per-device manufacturing cost; reproduction cost is just the memory used.

- Examples: switch debouncing, comms protocols (HW peripheral or SW bit-banging), encoding/decoding, encryption/decryption.

9.5 Fault Tolerance and Defensive Design

9.5.1 Fault tolerance techniques

- Power-On Self-Test (POST): verify components and sensible memory at startup.
- Checksums: detect corruption in stored data (e.g., EEPROM settings).
- Hardware initialisation: explicitly initialise all ports — never rely on datasheet defaults.

9.5.2 Defensive software

- Watchdog Timer to recover from runaway code.
- Boundary checks on variables.
- Fill unused memory with a safe routine (NOP).
- Define unused interrupt vectors to point to a safe routine.
- Disable interrupts while modifying hardware registers.

9.6 Software Documentation

- Critical for maintainability and often a legal requirement; do it as the project progresses.
- Sensible naming conventions; thorough comments.
- Graphical aids: state diagrams, flow charts, UML.
- Mathematical equations and technical manuals.

9.7 Use Cases & Applications

Architecture	Choose when...	Example
Polling loop	Tiny system; few inputs; latency » loop period	Bench LED + button demo
Event-driven (interrupts)	Latency-critical events; CPU should sleep	Battery sensor node, button debouncer
FSM (switch-case)	Discrete process with clear states & transitions	USB enumeration, vending machine controller
RTOS	Multiple concurrent activities with soft/hard deadlines	Drone flight stack, multi-sensor logger
Full OS (Linux)	Networking, storage, GUI, lots of throughput	Smart home gateway, automotive infotainment

9.8 Worked Examples

9.8.1 Polling vs interrupts for a button

A debounced button press takes ~10 ms. A polling loop running every 100 ms can miss a quick press. An interrupt-driven design starts a debounce timer on the GPIO change and reliably

catches every press while letting the CPU sleep between events.

9.8.2 Watchdog timer setup

Set the watchdog timeout $> 2 \times$ longest expected loop. If the main loop usually completes in 50 ms, a 250 ms watchdog gives margin for slow paths but still resets within a quarter-second of a runaway.

9.9 Code Snippets

9.9.1 Switch-case FSM

```
typedef enum { IDLE, ARMED, FIRING, COOLDOWN } state_t;
static state_t state = IDLE;

void step(event_t e) {
    switch (state) {
        case IDLE:    if (e == ARM)    state = ARMED;    break;
        case ARMED:   if (e == TRIGGER) { fire();    state = FIRING;    }
                    else if (e == DISARM) state = IDLE;    break;
        case FIRING:   if (e == DONE)    state = COOLDOWN; break;
        case COOLDOWN: if (e == COOL)    state = IDLE;    break;
    }
}
```

9.9.2 POST + checksum on settings

```
// Verify that EEPROM-stored settings are valid before using them.
uint8_t computed = crc8(settings, sizeof(settings) - 1);
if (computed != settings.crc) {
    load_factory_defaults();
    save_settings();
}
```

Key Definitions

Bottom-Up Development

Start with low-level modules; quick to test but lacks system picture.

Top-Down Development

Start with high-level architecture; fewer late surprises but slower to test.

Polling Loop

Repeatedly check inputs and run actions; wastes cycles when delays are needed.

Event-Driven Architecture

Triggered by interrupts/events; CPU can sleep or do other work in between.

FSM

Finite State Machine; states and transitions, often a switch-case implementation.

RTOS

Real-Time Operating System; deterministic, soft or hard real-time.

Fault Tolerance

Ability to recover automatically from errors (e.g., power spikes).

Defensive Design

Techniques limiting damage from interference (boundary checks, WDT, safe vectors).

Watchdog Timer

Resets the MCU if the program fails to pet it periodically.

POST

Power-On Self-Test; checks components and memory sanity at startup.

Soft Real-Time

Timing requirements where missing a deadline degrades quality (e.g., audio glitches) but isn't catastrophic.

Hard Real-Time

Timing requirements where a missed deadline is a system failure (e.g., engine ignition timing).

Brown-Out Detection

MCU feature that holds the CPU in reset when supply voltage dips below a threshold, preventing erratic behaviour.

Re-entrant Function

Function that can be safely interrupted and called again from the ISR without corrupting state (no static variables, no shared globals).

Sources

- Lecture 9.pdf

Chapter 10

Modularizing Embedded Projects

ENEL712 Embedded Systems Design — Lecture 10

Summary

Modular design principles in embedded C: how to split a project into modules, source files, and functions; driver levels; encapsulation in C; and a 6-step modular design recipe.

Topics Covered

- Core concepts of modularization
- Structuring modular software
- Advantages and disadvantages
- Specialized modular techniques
- Six-step modular design recipe
- Encapsulation and scoping in C
- Performance implications

10.1 Core Concepts of Modularization

- Modularization splits a large program into manageable components: source files, classes, and functions.
- Group common functionality into modules with focused requirements.
- Modules can be unique or interchangeable; modular programming is a key part of architectural design.
- Applies across software levels (drivers to apps) and even hardware.

10.2 Structuring Modular Software

Level	Description
Modules	High-level abstract groupings of functionality (e.g., Communications, Controller, UI). Typically 3–5.
Source files	Each module = one or more source files (one per module is often easiest).
Functions	Each source file = one or more focused functions (recommended 3–6).

10.3 Advantages and Disadvantages

Advantages	Disadvantages
Promotes top-down architecture and good design	More files create clutter
Easier to develop, debug, test, and maintain	Finding functionality across many files can be hard
Enables isolation for testing	Header files needed to link functions across files
Provides basic encapsulation by controlling access	Function call overhead (push/pop on stack)
Enables code reuse across projects	—

10.4 Specialized Modular Techniques

10.4.1 Driver levels

Level	Role
Low-level	Operates at the register level (most hardware-dependent).
Mid-level	Basic operations like sending/receiving individual bytes.
High-level	Specific complex functionality calling mid-level functions.

10.4.2 Wrapper functions

Provide a simpler calling interface for complex functions, often setting default values or converting argument formats without changing core behaviour.

10.5 Six-Step Modular Design Recipe

- Module identification: divide specs into 3–5 main groups (e.g., physical devices, user I/O, algorithms, data).
- Source file identification: usually one per module unless complexity demands more.
- Function identification: split file requirements into focused functions; avoid duplication; use wrappers.

- Data identification: decide how data is passed (consider structures and pointers for many variables).
- Interrupt identification: pick events for ISRs based on timing or risk of missing data.
- Code reuse: write modules (e.g., SPI) to be platform/application-independent where possible.

10.6 Encapsulation and Scoping in C

- Header (.h): public API — declarations of functions and types other modules use; extern declares variables/functions defined elsewhere.
- Source (.c): implementation; static gives functions/variables internal linkage and hides them from other modules.
- Layered driver approach simplifies application code (e.g., readSPIReg(device, address) hides chip select and register details).

10.7 Performance Implications

Function calls require pushing parameters/return address/registers onto the stack and popping them on return. Lots of jumping = lots of push/pop. For short, time-critical code this overhead can matter.

10.8 Use Cases & Applications

- Driver reuse: an SPI module written against a generic 'send byte / receive byte' interface drops into the next project on a different MCU with only the lowest layer rewritten.
- Team scaling: separate source files let two engineers work on Comms and UI independently with minimal merge conflicts.
- Unit testing: modules with narrow header APIs can be linked against a PC build harness and exercised without hardware.

10.9 Worked Example: Layered SD-Card Driver

Layer	Functions	What it knows about
Low (mcu_spi.c)	spi_init(), spi_xfer(byte)	AVR SPI registers
Mid (sd_block.c)	sd_read_block(lba, *buf), sd_write_block(lba, *buf)	SD command set, CRCs
High (fs_fat.c)	fopen(), fread(), fwrite()	File system semantics
App (logger.c)	log_event(struct e)	Business logic only

Swapping the MCU only changes mcu_spi.c — the SD command logic, FAT layer, and application stay identical.

10.10 Code Snippets

10.10.1 Header file (public API)

```
// uart.h
#ifndef UART_H
#define UART_H

#include <stdint.h>

void uart_init(uint32_t baud);
void uart_putc(uint8_t c);
uint8_t uart_getc(void);

#endif
```

10.10.2 Source file with private statics

```
// uart.c
#include "uart.h"
#include <avr/io.h>

static volatile uint8_t rx_buf[64]; // private to this file
static volatile uint8_t rx_head, rx_tail;

void uart_init(uint32_t baud) { /* ... */ }
void uart_putc(uint8_t c)      { /* ... */ }
uint8_t uart_getc(void)       { /* ... */ }
```

10.10.3 Wrapper function example

```
// Convenience wrapper that hides the SPI mode and chip-select details.
uint8_t adc_read_channel(uint8_t ch) {
    uint8_t cmd[3] = { 0x06 | (ch >> 2), ch << 6, 0x00 };
    spi_select(ADC_CS);
    spi_xfer_buf(cmd, 3);
    spi_deselect(ADC_CS);
    return ((cmd[1] & 0x0F) << 8) | cmd[2];
}
```

Key Definitions

Module

High-level abstract grouping of related functionality (e.g., UI, Comms, Controller).

Source File

Concrete .c file implementing a module's functions and private data.

Wrapper Function

Function simplifying the calling interface for a more complex underlying function.

Low-Level Driver

Code operating directly on hardware registers.

High-Level Driver

Code performing complex device functionality via mid-/low-level calls.

Encapsulation (in C)

Hiding internals using headers for the public API and static for private members.

Function Call Overhead

Cost of pushing/popping context on the stack at every call/return.

Top-Down Design

Plan architecture (modules, files, functions) before implementation.

ISR

Interrupt Service Routine; runs in response to a hardware signal.

Translation Unit

A .c source file plus everything it #includes after preprocessing — the unit the compiler turns into one .o.

API (Application Programming Interface)

Set of declarations a module exposes for callers to use; in C, this lives in the header file.

Volatile Keyword

Tells the compiler not to optimise away accesses to a variable; required for hardware registers and ISR-shared state.

Sources

- Lecture 10.pdf

Chapter 11

Advanced C Programming Techniques

ENEL712 Embedded Systems Design — Lecture 11

Summary

Advanced C techniques for embedded systems: switch-case state machines, advanced datatypes (arrays, strings, pointers, structures, function pointers), standard library helpers, and variable scope.

Topics Covered

- Switch-case state machines
- Advanced datatypes
- Pointer concepts and arithmetic
- Standard C library functions
- Variable access and scope
- Embedded risks with strings and arrays

11.1 Switch-Case State Machines

- FSM is the standard pattern for discrete processes (multi-stage flows, comms, UI, simple control).
- In C, naturally implemented using switch-case.
- Design with pen and paper or a graphics tool before coding.
- Always end each case with break to avoid case fall-through.

```
typedef enum { IDLE, MEASURE, SEND } state_t;
state_t state = IDLE;

for(;;) {
    switch(state) {
        case IDLE:    if (timer_flag) state = MEASURE; break;
```

```

    case MEASURE: sample = read_sensor(); state = SEND; break;
    case SEND:    uart_send(sample); state = IDLE; break;
}
}

```

11.2 Advanced Datatypes

Datatype	Description	Key features
Arrays	Data stored in consecutive memory locations	Pointer arithmetic; same-type elements; usually 1D/2D
Strings	Char arrays terminated with <code>'\{0'</code>	Not a native type; ASCII; required for std-lib string funcs
Pointers	Variables holding a memory address	Access without copying; allow funcs to modify args
Structures	Group multiple variables of different types under one name	. for direct access, -> for pointers
Function pointers	Pointers that point to functions	Useful for callbacks and search algorithms

11.3 Pointer Concepts and Arithmetic

- Reference (&): returns the address of a variable.
- Dereference (*): accesses the value at an address.
- Pointers are integer addresses, so arithmetic is allowed (e.g., incrementing an array pointer moves to the next element).
- Dangling pointers (uninitialised) are bad practice.
- In embedded C, pointers access hardware registers (peripherals at fixed addresses), pass large structs efficiently, and implement callbacks/jump tables.

11.4 Standard C Library Functions

11.4.1 <stdio.h>

- `printf` — formatted string to stdout.
- `sprintf` — like `printf` but writes to a char array (very useful in embedded).
- `scanf` — read formatted data from a string.

11.4.2 <stdlib.h>

- `atoi`, `atof`, `atol` — quickly convert strings to int/float/long (no error reporting).

11.4.3 <string.h>

- `memcpy` — copy N bytes between memory locations.
- `memset` — set an entire array to a specified value (e.g., zero-init).
- `strcpy` — copy null-terminated character arrays.

11.5 Variable Access and Scope

- extern: share a global variable across files — define in one .c, declare extern in a header.
- Local static: a local variable that retains its value between calls.
- Global static: restricts a global variable or function to its source file (internal linkage).
- Risk: static-local variables make a function non re-entrant — unsafe in interrupts/threads.

11.6 Embedded Risks with Strings and Arrays

- Buffer overflow: writing past an allocated buffer corrupts adjacent data and risks security.
- Missing null terminator: standard library reads past the intended end.
- printf/sprintf are large in code and RAM — use carefully on resource-constrained devices.

```
typedef struct {
    int16_t  temp_c_x100;
    uint32_t timestamp_ms;
    uint8_t  status;
} temp_record_t;
```

11.7 Use Cases & Applications

- Function pointers as callbacks: a generic timer driver can take 'on tick' as a function pointer so different modules wire up their own logic without modifying the driver.
- Structs as records: a single struct passed by pointer keeps a complex measurement (timestamp + temp + humidity + ...) coherent across function calls.
- Pointer-based parsing: an incoming UART buffer is parsed by walking a pointer through it, picking out fields without copying.
- FSMs implemented with switch-case scale to dozens of states — when they reach hundreds, prefer state-tables of function pointers.

11.8 Code Snippets

11.8.1 Function pointer / callback

```
typedef void (*tick_handler_t)(void);
static tick_handler_t on_tick;

void timer_set_handler(tick_handler_t fn) {
    on_tick = fn;
}

ISR(TIMER1_OVF_vect) {
    if (on_tick) on_tick();
}
```

11.8.2 Static local for state

```
uint16_t debounce(uint8_t pin) {  
    static uint16_t shift = 0;           // retains value between calls  
    shift = (shift << 1) | pin;  
    return shift == 0xFFFF;             // 16 stable highs  
}
```

11.8.3 sprintf-into-buffer for UART output

```
char buf[40];  
int n = snprintf(buf, sizeof(buf),  
                 "T=%d.%02d C, RH=%d %%\r\n",  
                 t / 100, t % 100, rh);  
uart_write(buf, n);
```

11.9 Common Pitfalls

- Buffer overflow: `snprintf` is safer than `sprintf` because it takes the buffer length.
- Forgetting the null terminator when manually building strings — string functions read past the intended end.
- Using an interrupt-shared variable without `volatile` causes the compiler to cache it in a register and miss updates from the ISR.
- Calling `printf` inside an ISR can blow the stack; ISRs should be short and only set flags or push to a queue.

Key Definitions

FSM

Finite State Machine; pattern for multi-stage processes implemented with switch-case in C.

Dangling Pointer

Pointer that has been declared but does not point to a valid memory address.

extern

Declares a variable/function defined in another translation unit.

static (local)

Local variable with static storage; retains value between calls.

static (global/file scope)

Restricts a variable/function to its own source file (internal linkage).

Function Pointer

Variable storing the address of a function; used for callbacks and configurable strategies.

Pointer Arithmetic

Mathematical operations on pointers that move them by the size of the pointed-to type.

Null Terminator

`'\0'` character marking the end of a C string.

Callback

Function pointer passed to another module so that module can 'call back' into yours when an event occurs.

Buffer Overflow

Writing past the end of an allocated buffer; corrupts adjacent memory and is a classic security and reliability bug.

Re-entrancy

Whether a function can be safely called recursively or from interrupts; static locals and shared globals break re-entrancy.

snprintf

Length-bounded variant of sprintf that truncates rather than overrunning the buffer.

Sources

- Lecture 11.pdf

Chapter 12

Software Architecture Case Study — Vineyard Weather Station

ENEL712 Embedded Systems Design — Lecture 12

Summary

Designing modular embedded software using the NZ vineyard weather station as a case study: hardware block diagram, modules, source files, functions, and an event-driven state-machine architecture.

Topics Covered

- Project specification
- Hardware block diagram overview
- Software architecture and modularization
- Final modular structure
- Driver abstraction levels
- State-machine + event-driven architecture

12.1 Project Specification

- Capabilities: monitor and record temperature, humidity, rainfall, sunlight hours.
- Power: self-powered or designed to minimise battery replacement.
- Connectivity: wireless link to the owner's house.
- Initial briefs are typically short — MCU choice, wireless tech, sample rate, and physical size are unspecified.

12.2 Hardware Block Diagram Overview

- MCU: ATmega 328P.

- Sensors: TCN75A temperature (I²C), HTS221 humidity (I²C), Hydreon RG-11 rainfall (pulse counter), Excelitas VT90N1 LDR sunlight.
- Power: 7.4 V 500 mAh Li-Po → 3.3 V via Microchip MCP1824T LDO.
- Storage/comms: Microchip 25AA1024 EEPROM (SPI), Digi XBee Pro ZigBee (UART).
- Other: JTAG interface for debugging, system clock.

12.3 Software Architecture and Modularization

12.3.1 1. Defining modules

- Modules are abstract groupings (Communications, Controller, UI).
- First step from spec to design — usually 3–5 modules.

12.3.2 2. Multiple source files

- Each module = one or more .c files; files allow independent build/test.
- Encapsulation: internal data/funcs not declared in the header stay local.
- Example: calcs.c, gps.c, lcd.c, setup.c.

12.3.3 3. Functions and logic

- Each file holds many small functions, each doing one job.
- Granularity: don't write one giant readGPS — split into decodeTime, decodeLatLong, etc.
- Aim for 3–6 functions per source file.

12.4 Final Modular Structure for the Weather Station

Module	Source file	High-level functions	Low-level / driver functions
Sensors	sensors.c	readAllSensors(data)	readTemp(), readHum(), readLight(), readRain(), readADC(channel), I2CRead/Write
Communications	comms.c	txMeasmnt(data)	txChecksum(), readyToSend(), UART_TxTime(), UART_TxInt(), UART_Tx()
Memory	memory.c	writeMeasmnt(data), readMeasmnt()	getCapacity(), freeMemoryLeft(), clearMemory(), writeByte(), readByte(), SPIExchange()
Main	main.c	main() state machine	ISR, setup, UART_Rx

Central data structure: timestamp, temperature, humidity, rainfall, sunshine.

12.5 Driver Abstraction Levels

Level	Example
Low-level	Direct register access, e.g., SPDR = byte
Mid-level	spi_transfer(byte) hides register details
High-level	readTemp() calls mid-level I ² C functions and applies calibration

12.6 State-Machine + Event-Driven Architecture

- Battery-powered → must spend most time asleep.
- main() implements the core loop as a state machine (e.g., SLEEP/SENSE/TX/SLEEP).
- ISRs handle WDT wakeup, UART data ready, etc., so the CPU can sleep otherwise.

12.7 Use Cases & Applications

- The vineyard station pattern (Sleep → Sense → Process → Transmit → Sleep) generalises to any battery-powered logger: livestock trackers, weather stations, environmental loggers.
- I²C for slow sensors + SPI for high-throughput storage is a typical low-power node split.
- WDT-only wakeups + an event-driven main FSM is the dominant architecture for sub-100 μ A average current designs.

12.8 Worked Example: Time Budget per Wake

Suppose the station wakes once every 5 minutes for a 2 s active window:

- Time online per day: $2 \text{ s} \times (24 \times 60 / 5) = 576 \text{ s} \approx 9.6 \text{ min}$ — under 0.7 % duty cycle.
- If active current is 200 mA and sleep current 30 μ A, $I_{\text{avg}} \approx 0.0067 \times 200 + 0.9933 \times 0.03 \approx 1.37 \text{ mA}$.
- On a 2,000 mAh Li-Po, lifetime $\approx 2,000 / 1.37 \approx 1,460 \text{ h} \approx 60 \text{ days}$.
- Levers to extend: cut active time (process less locally), drop wake rate, switch to a lower-current radio.

12.9 State-Machine Skeleton (main.c)

```
typedef enum { S_SLEEP, S_SENSE, S_LOG, S_TX } state_t;

int main(void) {
    setup();
    state_t s = S_SLEEP;
    measurement_t m;

    for (;;) {
        wdt_reset();
        switch (s) {
            case S_SLEEP: enter_power_down(); /* WDT will wake */ s = S_SENSE; break;
            case S_SENSE: m = readAllSensors();                      s = S_LOG;
        }
        break;
    }
}
```

```

        case S_LOG:    writeMeasmnt(&m);                s = S_TX;
    break;
        case S_TX:     txMeasmnt(&m);                s = S_SLEEP;
    break;
    }
}
}

```

12.10 Failure Modes & Mitigations

- Wireless TX never returns ACK → bound the retry loop and drop to log-only mode.
- EEPROM write failure → checksum each record; if the checksum reads back wrong, retry on a different cell.
- Sensor I²C lockup → re-init the bus periodically; if a transaction times out, reset the affected device line.
- Brown-out mid-write → store settings in two slots and pick the higher-versioned valid one on boot.

Key Definitions

MCU

Single-chip computer integrating CPU, memory, and peripherals for real-time control.

I²C

Synchronous two-wire bus (SCL/SDA) supporting multiple devices via addressing.

SPI

Synchronous, full-duplex four-wire serial protocol with master/slave roles.

UART

Asynchronous serial protocol relying on agreed baud rate and frame format.

Encapsulation

Public declarations live in headers; private details in .c — restricts access.

Modularization

Splitting software into manageable abstract components.

FSM

Finite State Machine architecture pattern.

Non-Functional Requirements

Constraints on how well the system performs (timing, reliability, power).

Duty Cycle

Fraction of time spent in the active state vs total period.

Periodic Wake

Pattern of using a low-power timer (WDT or RTC) to wake the MCU at a fixed cadence.

Power Budget

Allocation of average current per subsystem against the total battery capacity.

Sources

- Lecture 12.pdf

Chapter 13

Cost, Memory & Power Estimation

ENEL712 Embedded Systems Design — Lecture 13

Summary

Estimating costs, memory, and power for embedded products: manufacturing economics, RAM/Flash budgets and risks, and battery-life estimation with sleep modes.

Topics Covered

- Cost estimation
- Memory estimation (Flash, RAM, EEPROM)
- Power estimation methods
- Power-saving techniques

13.1 Cost Estimation

- Manufacturing costs = components + PCB fab + assembly + testing + distribution.
- Component selection (e.g., LDO): balance price, availability, performance (dropout, current), and supply risk.
- Economies of scale: a regulator at \$0.509 per unit can fall to \$0.125 at >20,000 units.
- Sustainability and RoHS: pick RoHS-compliant components — limits on lead, mercury, etc.
- PCB fab/assembly: local (e.g., Circuit Labs, PCB Zone) vs cheaper overseas; full-service options (e.g., Triode) cover programming and testing.

13.2 Memory Estimation

13.2.1 C program storage and overhead

- Even an empty while(1) consumes Flash (e.g., 176 bytes) due to initialisation code and interrupt vectors.
- Local variables in small programs may live in GPRs to save RAM; globals always live in RAM.

- Initialised globals require space in BOTH Flash and RAM — Flash holds the constant, the startup code copies it into RAM. For estimation, assume initialised vars consume double memory.

13.2.2 Estimation challenges and risks

- Flash usage depends on code structure, language, libraries, optimisation flags, and floating point.
- Avoid recursion: rapidly consumes stack and is invisible to static memory estimates — risk of unhandled overflow.

13.3 Power Estimation

13.3.1 Constant loads

Lifetime = battery capacity (mAh) $\div$ total current (mA). Capacity rating depends on C-rate, age, cycles, and cutoff voltage.

13.3.2 Varying loads

- Rough method: schedule of operating modes per hour; integrate current over each state's duration.
- Accurate method: measure current over time and numerically integrate.

13.4 Power-Saving Techniques

- Power scales with supply voltage and clock frequency — slower and lower means less.
- Sleep modes (Idle, Power-down, etc.) shut down unused internal modules.
- Power Reduction Registers (PRR) gate the clock to specific peripherals.
- Duty cycling — minimise time in active state, maximise time in sleep — is the dominant lever for battery life.

13.5 Use Cases & Applications

- Costing a product variant: redoing the BOM with cheaper alternates and re-running the current tally tells you how much margin you can shave.
- Memory budgeting before code: catches projects that won't fit on the chosen MCU before sinking weeks into firmware.
- Battery sizing: the same energy estimation underpins every IoT sensor, wearable, and remote control.

13.6 Formulas & Calculations

13.6.1 Battery life (constant load)

$T_hours \approx Capacity_mAh / I_mA$ (true only at the rated C-rate and temperature).

13.6.2 Battery life (duty-cycled load)

$I_{avg} = D \cdot I_{active} + (1 - D) \cdot I_{sleep}$, where $D = t_{active} / (t_{active} + t_{sleep})$. Then $T_{hours} \approx \text{Capacity_mAh} / I_{avg_mA}$.

13.6.3 Memory rule of thumb

Initialised global variables eat both Flash and RAM. For estimation, treat them as costing double until the linker map says otherwise.

13.6.4 Volume cost reduction

Component price typically follows 'price $\propto 1 / \text{volume}^k$ ' with $k \approx 0.2\text{--}0.4$ across normal cut tape vs reel breakpoints. A $1 \rightarrow 1,000$ jump can drop unit price $3\text{--}5\times$.

13.7 Worked Example: Sensor Node Battery Life

Spec: 1-year service life, AA battery ($\sim 2,000$ mAh).

- Required $I_{avg} \leq 2,000 \text{ mAh} / (365 \times 24 \text{ h}) \approx 0.228 \text{ mA} = 228 \mu\text{A}$.
- If $I_{active} = 50 \text{ mA}$ (radio TX) and $I_{sleep} = 4 \mu\text{A}$, allowable $D \leq (228 - 4) / (50,000 - 4) \approx 0.0045 = 0.45 \%$.
- On a 60-second wake cadence, $t_{active} \leq 0.27 \text{ s}$ — pushes you to fast-startup radios and pre-computed payloads.

13.8 Memory Budget Worksheet

- Estimate Flash: empty firmware + libraries + features. Add 20 % margin for late changes.
- Estimate RAM: globals + biggest realistic stack depth + buffers (UART/SPI). Forbid the heap to keep this deterministic.
- If Flash is tight, audit floating-point usage — `printf("%f")` drags in the entire fp library.
- If RAM is tight, move large constants into Flash with `PROGMEM/const`.

Key Definitions

RoHS

Restriction of Hazardous Substances directive limiting materials like lead and mercury.

Program Memory (Flash)

Non-volatile firmware storage; persists across power cycles.

Data Memory (RAM)

Volatile memory for runtime variables, stack, and buffers.

EEPROM

Byte-addressable non-volatile memory for calibration/settings.

Recursion

A function calling itself; risky in embedded systems due to stack consumption.

Duty Cycling

Toggling between high-current active and low-current sleep states to save energy.

Power Reduction Registers (PRR)

Registers that disable the clock to specific peripherals.

LDO Regulator

Linear regulator with very small required V_{in} – V_{out} dropout.

Economies of Scale

Per-unit cost reduction as production volume rises.

GPRs

General Purpose Registers; fast CPU storage that can hold local variables.

BOM (Bill of Materials)

The complete list of components in a product, with quantities and per-unit costs.

C-Rate

Charge/discharge rate expressed as a multiple of capacity per hour (1C = full discharge in 1 h).

Linker Map

File produced by the linker showing exactly how much Flash and RAM each module/symbol consumes.

Duty Cycle (power)

Fraction of time spent in the high-current active state; the dominant lever for battery life.

Sources

- Lecture 13.pdf

Chapter 14

Introduction to OOP and C#

ENEL712 Embedded Systems Design — Lecture 14

Summary

Foundational OOP concepts (classes/objects, encapsulation, get/set, constructors, overloading, reference types) and an introduction to C# and the .NET platform.

Topics Covered

- OOP overview
- Classes, objects, properties, methods
- Encapsulation and access control
- C# memory and initialization mechanics
- Introduction to C#
- Capabilities and use cases
- Why use C# (and why not)

14.1 OOP Overview

- OOP organises programs around objects and data rather than actions and logic.
- Modularity, reusability, encapsulation, and hierarchy.
- Other OOP languages: C++, Java, MATLAB, Python, PHP.

14.1.1 In-class framing: “why this feels weird at first”

Dr. Banner opened the topic by being honest about his own first encounter with OOP, and the description is worth keeping because it matches what most students feel:

- Coming from **structured** / **functional** programming (microcontroller C), every program had “a nice flowing logical order” — a sequence of statements you could read top to bottom.
- His first impression of OOP was the opposite: “getting a ball of clay and just throwing at the wall and everything going *splat*.”

- The chaos is only on the surface. OOP is in fact **deliberately organised** — but the organisation is around *data ownership and lifetime* (where things live in stack vs heap, who is allowed to touch them), not around the order of statements.
- Once every class decision is read as “where does this live in memory, and who is allowed to touch it?”, the apparent disorder turns into structure.

14.1.2 The four pillars in plain words

- **Modularity** — break the program into independent pieces that can be written, debugged and tested separately.
- **Reusability** — “this should be set up everywhere, even in structured programming.” If a block of code is used more than once, lift it into a function/method.
- **Encapsulation** — the OOP-specific pillar. All the data and the operations that act on it live inside one container; that container controls access.
- **Hierarchy** — classes can be specialised into derived classes (covered in Lectures 16/17). For Lecture 14, the important bit is that the language even supports the idea.

14.2 Classes, Objects, Properties, Methods

- Class: template/blueprint integrating both code and data.
- Object: specific instance of a class — created with the `new` keyword.
- Property (data member): a data field, e.g., `array.Length`.
- Method: a function the object can perform, e.g., `account.Open()`.

14.2.1 “Class oriented programming”

The lecturer suggested that “object-oriented programming” is a slightly misleading name and that “**class**-oriented programming” would be more honest. The reasoning:

- When you sit down to design an OOP program, you spend almost all your time thinking about **classes** — their data, their methods, who can see what.
- Objects only appear at runtime, when a class is instantiated with `new`. “Until it is called and turned into an object, a class is just a placeholder.”
- Therefore the class is the template; the object is the consequence. The classes set up the framework; the objects come out of the classes.

14.2.2 What is a “module” here?

The slide deck uses the word *module* for a class. The lecturer flagged that this overloads the term:

- In structured programming, a “module” is a function — a chunk of code that does something.
- In OOP, a “module” is a class — which can do *something* (methods) but can also just *be* something (data).
- So the OOP module is more general: a function does work; a class bundles data with the work it can do.

14.3 Encapsulation and Access Control

- Encapsulation binds methods and properties together and controls external access.
- Modifiers: `private` (default) hides data; `public` exposes properties/methods.
- Get/Set methods provide controlled access — allowing validation, conversion, or other processing.

14.3.1 Two parts of encapsulation: grouping + gateway

The lecturer broke encapsulation into two ideas rather than one:

- **Grouping.** Everything related to one concept lives inside one container (the class). If the container is destroyed, the contents go with it.
- **Gateway.** You cannot wander into the container at will — you must use the published entrance (public method or property). The class controls what an outsider can read or write.

14.3.2 Why `public` is a global variable in disguise

- “Public is essentially a global variable.” Anything can read it and anything can write it.
- Global variables are dangerous in any paradigm — unrelated parts of the program can fight over the same value, causing race conditions and stale reads.
- Concrete example from class: a public `balance` field is “not so cool” — you might be happy for the bank to deposit, but you certainly do not want any caller to be able to withdraw or set the balance to whatever they like.
- Hence the default position: make fields `private` and expose a controlled gateway (a method or a property).

14.4 C# Memory and Initialization Mechanics

- Classes are reference types. Declaring a variable creates a reference on the stack; `new` allocates the object on the heap.
- Constructor: special method run on instantiation — same name as the class, no return type.
- Overloading: multiple methods/constructors with the same name and different parameter signatures.

14.5 Introduction to C#

- Modern, general-purpose, simple OOP language built by Microsoft (2001), evolved from FORTRAN/C/C++.
- Built on the .NET framework — originally Windows-only, increasingly Linux/Mac via cross-platform .NET.
- C# is compiled to an intermediate language and then to machine code at runtime.

14.6 Capabilities and Use Cases

- GUI development for Windows.

- Web/cloud (ASP.NET, Azure) and Silverlight programs.
- Office add-ins, libraries, and integration with C/C++/Java/FORTRAN/MATLAB.
- Case study: JSteam — integrates C++/C#/Excel with custom ribbons, functions, and graphical plant models (HP/MP/LP users).

14.7 Why Use C# (and Why Not)

14.7.1 Advantages

- CLI-based — fast app development with rich libraries.
- Simpler than C++ with great Office/Cloud integration.
- Managed code: automatic GC and platform independence.

14.7.2 Disadvantages

- Not ideal for high-speed numerical work, compression, or encoding (no direct compile to native code).
- Historically Windows-only; improved by open-source .NET (2014) and cross-platform Visual Studio (2015).

14.8 Use Cases & Applications

- Modelling real-world entities (Account, Sensor, Order) as classes keeps related data and behaviour together — easier to reason about than passing big structs through dozens of free functions.
- Reusability across products: a Sensor base class with TemperatureSensor, HumiditySensor children means the rest of the application code doesn't need to know which model it's reading from.
- C# has first-class GUI, networking, and database libraries, making it strong for desktop tools and the PC side of an embedded product.
- Cross-platform with modern .NET: the same C# can run on Windows, Linux, and macOS — useful for companion apps that ship to multiple OSs.

14.9 Worked Example: A Tiny Account Class

```
public class Account
{
    private decimal balance;
    private string owner;

    public Account(string owner, decimal opening)
    {
        this.owner = owner;
        this.balance = opening;
    }

    public void Deposit(decimal amount)
    {
```

```

        if (amount <= 0) throw new ArgumentException("positive only");
        balance += amount;
    }

    public bool Withdraw(decimal amount)
    {
        if (amount <= 0 || amount > balance) return false;
        balance -= amount;
        return true;
    }

    public decimal Balance => balance;
}

```

The fields are private (no caller can corrupt the balance), the constructor enforces a valid initial state, and Withdraw/Deposit each maintain the invariant that balance never goes negative.

14.10 C# vs C++ vs C — Quick Compare

Trait	C	C++	C#
Memory	Manual	Manual / RAII	Garbage-collected
Compiled to	Native	Native	CIL → JIT to native
GUI ease	Hard	Hard (Qt etc.)	Easy (WinForms, WPF, MAUI)
Performance	Highest	Very high	Slightly lower (managed runtime)
Best for	Embedded firmware	Performance-critical apps with OOP	Desktop apps, services, tooling

14.11 Intuition — Why OOP Feels Strange at First

Coming from structured/procedural programming, OOP often feels disordered — ‘a ball of clay thrown at the wall’ rather than the linear flow you are used to. The reason it works is that programs are organised around data lifetimes (stack vs heap) and encapsulation, not around the order of statements. Once you map every class decision back to ‘where does this live in memory and who is allowed to touch it?’, the apparent chaos becomes deliberate structure.

- Structured programming organises code by control flow.
- OOP organises code by data ownership and lifetime.
- Both compile down to the same machine instructions; the difference is at design time.

14.12 Value Types vs Reference Types — In Practice

Value-type assignment copies the bits. Reference-type assignment copies the pointer — both names now refer to the same heap object. Mutating one is visible to the other.

```

// Value type -- independent copies
int c = 100;
int d = c;           // d gets its own 100
d = 200;             // c is still 100

```

```
// Reference type -- shared object
int[] a = { 1, 2, 3 };
int[] e = a;           // e and a point at the SAME array
e[0] = 99;             // a[0] is now 99 too

// Decoupling a reference: make a real copy
int[] f = new int[a.Length];
Array.Copy(a, f, a.Length); // f is independent of a
```

- Strings, arrays, and class instances are reference types — assignment shares them.
- int, double, bool, char, struct, enum are value types — assignment copies them.
- Array.Copy or new T[] { ... } when you need a real copy.
- Array.Sort sorts in place — every reference observes the sorted result.

14.13 Mutable vs Immutable Types

- Mutable: contents can change after construction (List<T>, arrays, most classes).
- Immutable: contents are fixed once created (string, ValueTuple, record with init-only members).
- Mutable + shared reference is the classic source of 'spooky action at a distance' bugs.
- Immutable types are inherently thread-safe — no synchronisation needed.
- Default to immutable when the data is small and conceptually a value (point, money amount, configuration).

14.14 Picking a Type — var, decimal, double, int

- int: 32-bit whole numbers — the default for counters, indices, sizes.
- double: 64-bit floating-point — for measurements, scientific calculations, anywhere precision tolerates rounding error.
- decimal: 128-bit fixed-point — money and other values where rounding errors would be unacceptable.
- Don't allocate a double when you only need an int — you are using 8 bytes for what fits in 4.
- var asks the compiler to infer the type from the initialiser — useful for long generic types and anonymous objects, never for code reviewers' convenience.

```
var person = new { Name = "Bob", Age = 25 }; // anonymous type -- must use var
var list = new List<KeyValuePair<int, string>>(); // long type, var is cleaner
int count = 0; // simple -- be explicit
```

14.15 Methods vs Properties — Reading the Brackets

In Visual Studio's IntelliSense list, the trailing brackets tell you what something is. Brackets ⇒ method (an action). No brackets ⇒ property/field (a value).

Member	Brackets?	Meaning
<code>array.Length</code>	no	Property — read the current length
<code>array.Sort()</code>	yes — empty	Method — performs an action with no inputs
<code>account.Deposit(50)</code>	yes — with arg	Method — action that takes input
<code>account.Balance</code>	no	Property — exposes the balance

14.15.1 The in-class walkthrough — “brackets mean it does something”

The class worked through the brackets rule live, using a hypothetical **Account** object alongside the built-in **Array** type:

- `array.Length` and `account.Balance` have no brackets — they are **properties**. They expose a value. Reading them is like reading a labelled gauge.
- `array.Sort()` and `array.Copy()` *do* have brackets — they are **methods**. They perform an action. The brackets are how you “call” the action.
- `account.Open()` and `account.Deposit(50)` are methods too. Whatever you put between the brackets is the **argument** that the method works with.
- `array.Copy()` takes no arguments in this example; that does not change anything: it is still a method, the brackets just happen to be empty.
- Trying to pass arguments to a method that does not accept them (e.g. writing `array.Open(5)`) is a compile error — the method signature has to match.

14.15.2 Why IntelliSense is your first OOP debugger

- Type the variable name, then “.” — Visual Studio opens a dropdown of every public member.
- Methods are marked with a different icon than properties, and the bracket convention above lets you read intent at a glance.
- If a member does not appear, it does not exist on that type. The lecturer’s example: `array.Length` works because the **Array** class published a **Length** property; an arbitrary type without **Length** will simply not offer it.
- This is the fastest way to discover what a class can do without opening its source.

14.16 Abstraction — The House / Street / Neighbourhood Analogy

Abstraction means choosing the right level of detail for the question being asked.

- Standing in front of the door: the house number matters.
- Driving in the suburb: the street name matters; the house number is noise.
- Crossing the city: the neighbourhood matters; street and number are noise.
- .NET works the same way — your code talks about `Console.WriteLine`; the runtime worries about which registers, pages, and I/O ports it touches.
- Good APIs hide the level below; you only drop a level when the level you are at can no longer answer your question.

14.16.1 Why this matters for .NET

The lecturer used this analogy explicitly to explain what the .NET runtime is doing for you:

- Without .NET, you would have to talk to specific CPU registers, memory pages, and I/O ports yourself — that is the “house number” level.
- .NET “does all the heavy lifting underneath” so your code can stay at the “city” level: `Console.WriteLine`, `new FileStream`, `Array.Sort`, and so on.
- “As we are further removed (higher level), the less detail we need.” That is the entire reason high-level languages exist.
- Trade-off: the more layers of abstraction you sit on top of, the less direct control you have. For application code that is a feature; for hard real-time embedded firmware it is a problem, which is why C remains the default in that domain.

14.17 Namespaces — The Same-Realm Rule

- Everything inside the same namespace can refer to each other by simple name.
- Code in a different namespace must either fully qualify the name or use a `using` directive.
- Same role as `#include` for organising C, but stricter — the namespace is part of the type’s identity, not just a copy/paste of declarations.
- Multiple files can share one namespace (e.g., one `System.IO` across hundreds of source files).
- A namespace is not the same as an assembly: a NuGet package is one assembly that may expose several namespaces.

14.17.1 The header-file analogy from class

The lecturer compared namespaces to header files in structured C:

- In C, you `#include` a header to bring a different file’s declarations into your translation unit. The header is essentially a copy/paste mechanism.
- In C#, “anything that has the same namespace exists in the same realm.” Files in the same namespace see each other’s classes by simple name — no copy/paste required.
- Even if you do not write an explicit `namespace` block, every file is in *some* namespace (the global one).
- To make classes spread across multiple files cooperate, put them in the same namespace. To organise a large library, split it into several namespaces that callers can pull in with `using`.

14.18 Encapsulation — The Rainbow’s End Analogy

The lecturer used the Auckland theme park *Rainbow’s End* as a concrete picture of encapsulation. The park encloses the rides (grouping), and entry is only via the ticket office (controlled access).

- The park boundary = the class itself.
- The ticket office = a public method or property — the only sanctioned entry point.
- Climbing the fence = bypassing encapsulation (reflection, public fields) — possible but discouraged.

- Not every part is private: the parking lot is public — everyone can use it (analogous to truly public helper methods).
- If the park ceases to exist, so do its rides — when a class instance is collected, its private state goes with it.

14.18.1 Why this analogy is worth memorising

- **If Rainbow’s End were a public class**, it “would have no fence” — anyone could walk in and use anything without supervision. That is exactly what unrestricted **public** fields do to a class.
- Because Rainbow’s End is effectively **private**, only callers who came through the ticket office (the public API) get to use the rides. The class keeps full control over what they can do once they are in.
- The parking lot is a useful counter-example: not every member of a class has to be private. Some helpers *are* meant for the outside world (think `Math.PI` or a public utility method) — they are deliberately exposed.
- Two different classes can each have a private field called `x`, with completely different values and types. They live in their own park and never collide. That isolation is one of the single biggest reasons OOP scales to large programs.
- When a class instance is finally collected, “the park ceases to exist, and so do the rides.” The private data goes with the object — you do not have to remember to free it separately.

14.19 The Building-After-Hours Analogy

The lecturer offered a second, simpler picture in case the theme-park image does not land:

- During the day, the building is open: the foyer doors are unlocked, anyone can walk into any room. This is the “everything is **public**” state.
- After hours, the building shifts to private access: only people with the right key (the published API) can reach individual rooms.
- Designing a class is choosing which doors are unlocked during the day, and which require the key.

Key Definitions

OOP

Programming model organised around objects and data.

Object

Instance of a class created with the `new` keyword.

Class

Template/blueprint integrating code and data.

Encapsulation

Bundling methods/properties and controlling access — info hiding.

Modularity

Structuring code into organized, independent components.

Method

Function an object can perform.

Property

Data field/member that holds an object's data.

Constructor

Method that runs automatically when an object is created.

Managed Code

Code that runs under the CLR with automatic garbage collection.

CLI

Common Language Infrastructure — standardized infrastructure C# implements.

Garbage-Collected Language

Language whose runtime automatically frees memory that no live variable references.

Managed Code

Code that runs under a runtime (the CLR for C#) which provides services like type safety and GC.

Reference Type

Class instance whose variable holds an address to the actual data on the heap.

Constructor Chaining

Calling another constructor of the same or base class (`this(...)` / `base(...)`) to share initialisation logic.

Value type

Type whose values are stored directly (int, double, struct, enum). Assignment copies the value.

Reference type

Type whose values are accessed through a reference (class, array, string, delegate). Assignment copies the reference, not the data.

Mutable type

Type whose contents can be changed after construction.

Immutable type

Type whose contents are fixed at construction; any 'change' produces a new instance.

Abstraction

Hiding lower-level detail behind an interface that exposes only what the caller needs at this level.

Namespace

Logical grouping that scopes type names to avoid collisions; required to share types across files.

Sources

- Lecture 14.pdf
- In-class transcript: *Introduction to OOP* (Dr. Banner) — used for the “ball of clay” framing, the four-pillars plain-words summary, “class-oriented programming”, the public-as-global warning, the Rainbow's End and after-hours-building analogies, the brackets-mean-methods

walkthrough, the house/street/ neighbourhood explanation of abstraction, and the namespace = same-realm rule.

Chapter 15

OOP in C# — Class Mechanics & Encapsulation

ENEL712 Embedded Systems Design — Lecture 15

Summary

Deeper dive into C# OOP: class mechanics (reference vs value), access modifiers, constructors and overloading, properties via get/set, and the Circle class evolution as a case study.

Topics Covered

- Introduction to OOP
- Encapsulation in detail
- Class mechanics in C#
- Constructors and overloading
- Properties: get and set
- Code example analysis (Circle class)

15.1 Introduction to OOP

- Programming paradigm using objects (classes) as modular building blocks.
- Each object has its own properties (data) and methods (functions).
- Designed to make large programs manageable; modules can be programmed/debugged/verified independently.
- Example: a bank account object — properties like balance, overdraft; methods like open, close, suspend.

15.2 Encapsulation in Detail

- Combine methods and properties into a single unit and control accessibility.

- Users of the class don't care how it works — they just want it to work (e.g., open a file).
- Internal data stays private — improves reliability and simplifies management.

15.3 Class Mechanics in C#

- Defined with the `class` keyword inside a namespace.
- Classes are reference types (cf. value types like `int`).
- `Circle c;` only creates an empty reference on the stack; memory is allocated on the heap with `new`.

15.3.1 In-class walkthrough: declaring vs instantiating

The lecturer asked the class to walk through exactly what happens when you write each of these:

```
Circle c;           // empty reference on the stack, points at nothing
Circle c = new Circle(); // reference on the stack + object on the heap
double a = c.Area(); // call a method on the heap-allocated object
```

- `Circle c;` creates an *uninitialised reference* on the stack. You cannot do anything useful with it; calling a method on it would throw a `NullReferenceException`.
- `new Circle()` is the line that actually allocates memory on the heap, runs the constructor, and returns the reference that is then stored in `c`.
- Once `c` goes out of scope on the stack, no live reference points at the heap object — the garbage collector is free to reclaim it. “When the class ceases to exist, it automatically deletes.”

15.3.2 Naming convention: capital letters announce a class

- Class names use **PascalCase** (`Circle`, `BankAccount`). The capital at the start is the visual cue that you are looking at a type, not a variable.
- Local variables and parameters use **camelCase** (`radius`, `accountId`).
- Backing fields use **__camelCase** with a leading underscore (`_radius`). The underscore says “internal to the class, do not write to me from outside.”
- Curly braces are the delimiters for the class body. The opening brace says “everything inside here is part of this class”; the closing brace ends the scope. `C#` and `C++` share this convention; Python uses indentation instead.

15.3.3 Access modifiers

Modifier	Effect
private	Default; only the class itself can modify the data.
public	Accessible from outside the class.

15.4 Constructors and Overloading

- Constructor runs automatically on `new` — initialises properties.

- Same name as the class; no return type, not even void.
- Compiler generates a default constructor if you don't define one.
- Overloading: multiple methods/constructors with the same name but different parameter lists.

15.5 Properties: Get and Set

- Provide indirect, controlled access to private fields.
- set uses the implicit value parameter (e.g., `_radius = value;`).
- get returns the private field's value.
- Allow processing on read/write (e.g., unit conversion) and protect internal data.

15.6 Code Example: Circle Class Evolution

- Initial class: `public radius` — bad practice; lets external code change the field directly.
- Improved class: `private radius` enforces encapsulation.
- Final class: overloaded constructor (set radius on create) plus get/set properties — the recommended pattern.

15.6.1 The bank-balance question that motivates encapsulation

The lecturer used the same example twice — the `Circle.radius` field and a bank `balance` — to ask the class one question: *“do you want anyone to be able to write to this directly?”*

- For a `balance`: “I am happy for them to deposit, I am not happy if they withdraw.” Plain field access cannot tell the difference — it lets every caller write any value.
- For a `radius`: a negative radius is meaningless. Plain field access happily accepts `c.radius = -5;` and produces a nonsense area on the next line.
- Both problems have the same fix: **stop exposing the raw field**. Make it private and provide a controlled gateway (a method, a property with a setter, or a constructor argument) that can enforce the rules.

15.7 Use Cases & Applications

- Sensor models: a `Sensor` class with a private calibration array exposes a clean `Read()` method while hiding the calibration math.
- Validation in setters: a `Temperature.Celsius` set { } block rejects values below absolute zero so bad data never reaches downstream code.
- Multiple constructors: a `Vector2D(double x, double y)` plus `Vector2D(double angle)` (with overloading) lets callers create the same object from different inputs.

15.8 Worked Example: Circle Class Evolution

15.8.1 Bad — public field

```
public class Circle
{
    public double radius;    // anyone can set this to -1
}
```

15.8.2 Better — private field

```
public class Circle
{
    private double radius;
    public Circle(double r) { radius = r; }
    public double Area() => Math.PI * radius * radius;
}
```

15.8.3 Best — encapsulated with property and validation

```
public class Circle
{
    private double radius;

    public Circle(double r) { Radius = r; }

    public double Radius
    {
        get => radius;
        set
        {
            if (value < 0) throw new ArgumentOutOfRangeException();
            radius = value;
        }
    }

    public double Area => Math.PI * radius * radius;
}
```

15.9 Stack vs Heap for a Class Instance

Circle c = new Circle(2.5); allocates as follows:

- Reference c lives on the stack (4–8 bytes).
- The Circle object (radius field, type info) lives on the managed heap.
- When c goes out of scope and no other reference exists, the GC can reclaim that heap memory.

15.10 Naming Conventions — Read at a Glance

Element	Convention	Example
Class / Type / Method	PascalCase	Circle, BankAccount, Open()
Public property	PascalCase	Radius, Balance
Local variable / parameter	camelCase	radius, accountId
Private field	<code>_camelCase</code>	<code>_radius</code> , <code>_balance</code>
Constant	PascalCase or UPPER	Pi, MAX_SIZE
Namespace	PascalCase, dotted	MyCompany.Embedded.Drivers

- The leading underscore on a private field is a visual cue: 'this is internal, do not write to it from outside the class'.
- Using PascalCase for the public-facing Radius property and `_radius` for its backing field lets callers use Radius while the class controls `_radius`.
- Stick to one convention across the whole codebase — readers should be able to read intent from a name without checking the modifier.

15.11 Constructors — Default, Explicit, Overloaded

- If you do not write any constructor, the compiler generates a parameterless one that runs the default field initialisers (`int` $\rightarrow$ 0, `bool` $\rightarrow$ false, reference $\rightarrow$ null).
- Writing your own constructor replaces the implicit one — relying on default values is then your call to make explicit.
- Overloading lets you offer several construction shapes: parameterless, single-arg, copy constructor, etc.
- Once you write a constructor that takes parameters, the compiler-supplied no-arg constructor disappears unless you re-declare it.
- Constructor name = class name; no return type, not even void.

15.11.1 The in-class confusion: “what is the area going to be?”

The lecturer ran the class through this scenario step by step:

```
class Circle {  
    private int radius;  
    public double Area() => Math.PI * radius * radius;  
}  
  
Circle c = new Circle();  
double a = c.Area();    // what is `a`?
```

- In C#, an uninitialised `int` field defaults to 0. So `radius` is 0, and the call to `Area()` returns $\pi \cdot 0^2 = 0$.
- “Assuming it initialises as zero, then it will work. If it doesn’t initialise, then the whole thing is going to fail.”

- The bigger point: **do not rely on the default**. Write a constructor that puts the object into a known state before any caller can use it.
- This is exactly what the explicit constructor below is for — it removes the ambiguity by writing the initial value yourself.

15.11.2 “Constructor shares its name with the class”

- The constructor of `Circle` is called `Circle`. Same name as the class is the rule — that is how the compiler tells it is a constructor and not a normal method.
- No return type appears, not even `void`. If you write a return type, the compiler treats the method as a normal method with a coincidental name, and stops calling it on `new`.
- It can be `public`, `private`, or `internal`; most are public. A private constructor is the classic way to force callers to go through a factory method instead of `new`.

15.11.3 Overloading the constructor: what changes for the caller

- If you only write `Circle(int r)`, then `new Circle()` becomes a compile error — “the default constructor only gets created if we have not written one.”
- If you write both `Circle()` and `Circle(int r)`, the caller chooses by what they pass. `new Circle()` runs the no-arg version; `new Circle(4)` runs the `int` version.
- Pass a type that does not match any overload (e.g. a string), and the compiler picks neither and reports an error.
- This is the same overloading rule applied to constructors: same name, different parameter lists.

```
class Circle {
    private int _radius;

    // Explicit no-arg constructor -- initialises the field to a known value
    public Circle() { _radius = 0; }

    // Overload -- takes the radius
    public Circle(int r) { _radius = r; }

    public double Area() => Math.PI * _radius * _radius;
}

var a = new Circle(); // _radius = 0
var b = new Circle(4); // _radius = 4
```

15.12 The Circle Class — Three-Step Evolution

15.12.1 Step 1: public field — too open

```
class Circle {
    public int radius; // any caller can mutate
    public double Area() => Math.PI * radius * radius;
}

var c = new Circle();
c.radius = -5; // nothing stops nonsense input
```

```
Console.WriteLine(c.Area()); // garbage out
```

External code can write any value, including invalid ones. There is no place to add validation or unit conversion later.

15.12.2 Step 2: private field + constructor — too closed

```
class Circle {
    private int _radius;
    public Circle(int r) { _radius = r; }
    public double Area() => Math.PI * _radius * _radius;
}

var c = new Circle(4);
// c._radius = 5; // compile error -- cannot access private field
// no way to change radius without a new Circle(...)
```

Encapsulated, but rigid — there is no controlled way to change the radius after construction.

15.12.3 Step 3: property with get/set — the gateway

```
class Circle {
    private int _radius;

    public Circle(int r) { _radius = r; }

    public int Radius {
        get { return _radius; }
        set {
            if (value < 0)
                throw new ArgumentOutOfRangeException(nameof(value));
            _radius = value;
        }
    }

    public double Area() => Math.PI * _radius * _radius;
}

var c = new Circle(4);
c.Radius = 7; // goes through the set accessor -- validation runs
Console.WriteLine(c.Radius); // get accessor returns 7
```

- set runs whenever a caller writes to the property; the new value arrives in the implicit parameter named value.
- get runs whenever a caller reads the property.
- Either accessor can do work — validate input, lazy-load, raise an event, log, or simply return `_radius`.
- From the caller's perspective the property looks like a field, but the class keeps full control.

15.13 Properties as Gateways — Why Bother

For a single int the get/set wrapper looks like ceremony, but it pays off as the program grows.

- You can add validation in the setter without changing any caller.
- You can add unit conversion (set in metres, store in millimetres) transparently.
- You can refactor the backing storage (compute on the fly, lazy-load, cache) without breaking the public API.
- You can fire `INotifyPropertyChanged` or other events from the setter, enabling data binding.
- C# auto-properties (`public int Radius { get; set; }`) give you the gateway with zero ceremony — promote them to a full property only when behaviour is needed.

15.13.1 The honest in-class objection — “why bother, this is just an int?”

A student raised the very question this whole section answers: if all the property setter does is `store _radius = value;`, you have just “abstracted it up a level” for no benefit. The lecturer’s reply is worth repeating verbatim because it is the actual reason to use properties:

- “For a simple thing like this, yes — it just looks like extra ceremony.”
- “However, as your programs start getting more complicated, you will see why we want to do this and start limiting the behaviour, so that the values inside are only controlled by the external — on our terms.”
- In other words: **you write the gateway before you need it**, because the moment you do need validation, conversion, logging, events, lazy-loading, or a different backing field, the gateway is already in place and no caller has to change.
- If `Radius` were a public field, then on the day you decide to validate input you would have to change every single caller. If `Radius` is already a property, only the setter body changes.
- This is the most important habit to take away from Lecture 15: *default to a property, not a public field*.

15.13.2 The role of the implicit value keyword

- Inside a property’s `set` block, `value` is an implicit parameter that holds the value the caller is assigning.
- “So if I write `c.Radius = 7;`, `value` inside the setter is 7.”
- The setter then decides what to do with it: store it in the backing field, validate it first, convert units, log it, raise an event — the class is in charge.
- Forgetting to assign `value` to anything is a common bug (the setter silently throws the input away).

15.14 Method Overloading — Same Name, Different Signature

Overloading lets a single name describe a family of operations distinguished by their parameter list. The compiler picks the matching overload at the call site.

```
class Logger {
    public void Write(string message) { /* ... */ }
    public void Write(string message, int level) { /* ... */ }
    public void Write(Exception ex) { /* ... */ }
}

var log = new Logger();
```

```
log.Write("hello");           // Write(string)
log.Write("hello", 2);        // Write(string,int)
log.Write(new InvalidOperationException()); // Write(Exception)
```

- Overloads must differ in the number or type of parameters; differing only in return type is not enough.
- Constructors overload with the same rules.
- If two overloads are both applicable, the compiler picks the most specific match; ambiguities surface as compile errors.

Key Definitions

OOP

Paradigm structured around objects rather than functions and logic.

Class

Template/blueprint integrating methods and properties.

Object

Specific instance of a class created with new.

Encapsulation

Bundling methods and properties and restricting external access.

Property

Data field/member of an object.

Method

Function/action an object can perform.

Constructor

Automatically-invoked method that initialises a new object.

Overloading

Multiple methods/constructors with the same name but different parameter signatures.

Reference Type

Type whose variable holds a reference to data on the heap (e.g., class).

private

Access modifier limiting access to the defining class.

public

Access modifier allowing external access.

value (keyword)

Implicit parameter inside a property's set block.

Auto-Property

Shortcut C# syntax (public double Radius { get; set; }) where the compiler generates the backing field.

Expression-Bodied Member

Single-expression member written with => (e.g., public double Area => $\pi * r * r$;

Defensive Setter

set block that validates the incoming value before storing it, preserving the object's invariants.

Auto-property

Shorthand syntax `public T Name { get; set; }` that lets the compiler synthesise the backing field.

Backing field

Private field that stores a property's value; conventionally named with a leading underscore (`_radius`).

value (keyword)

Implicit parameter inside a property setter; holds the value the caller is assigning.

Overloading

Defining multiple methods or constructors with the same name but different parameter lists.

Default constructor

Compiler-generated parameterless constructor that runs default field initialisers; suppressed once you declare any constructor of your own.

Sources

- Lecture 15.pdf
- In-class transcript: *Introduction to OOP* (Dr. Banner) — used for the declare-vs-instantiate walkthrough, the PascalCase and underscore naming conventions, the bank-balance motivation for encapsulation, the “what is the area?” default-initialiser question, the constructor naming/return-type rules, and the honest “why bother with a property for an int?” exchange that motivates the gateway pattern.

Chapter 16

OOP Part B — Static, Inheritance, Polymorphism

ENEL712 Embedded Systems Design — Lecture 16

Summary

OOP continued: static members and constants, inheritance and the base keyword, polymorphism via overloading, enumerations, and a coaxial-cable modelling exercise.

Topics Covered

- C# naming conventions (PascalCase / camelCase / underscores)
- Static methods and data members
- Inheritance
- Polymorphism and overloading
- Enumerations
- Coaxial cable programming exercise

16.1 C# Naming Conventions

The lecture opened with the full Microsoft-recommended naming scheme. The reason it is in the OOP lecture and not in the C# syntax lecture is that almost every example in this chapter (and the next two) uses it, so getting the conventions straight up front makes the code easier to read.

16.1.1 PascalCase — types and public members

- Used for the names of `class`, `interface`, `struct`, `record`, `enum`, `event`, and `method`.
- Also used for public properties, public fields, init-only properties, and local functions.
- Interface names additionally start with a capital “I” (`IWorkerQueue`, `IDisposable`).

```
public class ExampleEvents
{
    public bool IsValid;           // public field
```

```

public IWorkerQueue WorkerQueue { get; init; }           // init-only property
public event Action EventProcessing;                     // event

public void StartEventProcessing()                       // method
{
    static int CountQueueItems() => WorkerQueue.Count;  // local function
}

public record PhysicalAddress(string Street, string City,
                              string StateOrProvince, string ZipCode);
public struct ValueCoordinate { }
public interface IWorkerQueue { }

```

16.1.2 camelCase — locals, parameters, private fields

- Local variables and method parameters use plain camelCase (`someNumber`, `isValid`).
- Private/internal instance fields use a leading underscore plus camelCase: `_workerQueue`.
- Private static fields use `s_` as a prefix: `s_workerQueue`.
- Thread-static private fields use `t_`: `[ThreadStatic] private static TimeSpan t_timeSpan;`

```

public class DataService
{
    private IWorkerQueue _workerQueue;           // instance field
    private static IWorkerQueue s_workerQueue;    // static field

    [ThreadStatic]
    private static TimeSpan t_timeSpan;          // thread-static field
}

public T SomeMethod<T>(int someNumber, bool isValid) { /* ... */ }

```

16.1.3 What *not* to use

- `snake_case` and `SCREAMING_SNAKE_CASE` are *legal* identifiers but *non-standard* in C#.
- Identifiers must start with a letter or underscore, can use Unicode letters/digits, and **cannot contain two consecutive underscores** — those names are reserved for compiler-generated symbols.
- The compiler will reject identifiers that violate those rules; the linter (style analyser) will flag identifiers that follow the rules but violate the conventions.

16.2 Static Methods and Data Members

- Static members belong to the class itself, not to instances.
- Static methods: utility functions called as `Class.Method()`; can only call other static members.
- Static data members: shared fields, e.g., `public static int numCircles = 0;`
- Constants: `const` creates a static, immutable field (e.g., `Math.PI`).
- `Math.Sqrt()` is a classic example — only depends on the argument.

16.2.1 The transcript framing — “called by the class, not the instance”

The lecturer’s plain-English summary is worth keeping next to the bullet list:

- Static methods “belong to the class and are called using the class name, not object instances” — you write `Math.Sqrt(2)`, not `new Math().Sqrt(2)`.
- Because a static method has no `this` reference, it “can only access other static members”. It cannot read or write any per-instance field.
- Static fields exist **once in memory** and are shared across every instance of the class. Changing one through one instance is visible through every other instance.
- Constants (`const`) are immutable, *implicitly* static fields. They are evaluated at compile time and baked into every caller; `readonly static` is the runtime equivalent when the value is not known until startup.

16.2.2 Two everyday static classes you have already used

- `Math` — a static class. You call `Math.Sqrt`, `Math.Pow`, `Math.PI` directly; there is no `new Math()`.
- `Console` — another static class. The whole of `Console.WriteLine`, `Console.ReadLine`, and `Console.ForegroundColor` sits on the type itself.
- Building your own static helper class is appropriate any time the function depends only on its inputs (a stateless calculator, a formatter, a converter).

16.3 Inheritance

- Promotes code reuse — derived class inherits from a base class.
- Syntax: `class DerivedClass : BaseClass`.
- A class can derive from only one base class.
- Good practice: derived constructor explicitly calls the base constructor with `base`.
- Example: `Employee` base class with pay rate; `Manager` and `Cleaner` derived classes.

16.3.1 Why inheritance exists: avoiding duplication

The lecturer’s framing was concrete: imagine writing the code that represents every person in an office.

- Every employee has an IRD number and a pay rate. Repeating those fields in `Manager`, `Cleaner`, `Apprentice`, `Engineer`... is duplication.
- Lift the common fields into a single `Employee` base class. Each specialised type inherits them automatically.
- Each derived type can then add its *own* properties and methods on top — `Manager` adds a bonus, `Cleaner` adds a shift roster, `Apprentice` adds a study leave entitlement.
- The point is not just less typing: a payroll routine that works on `Employee` works on *every* subclass automatically.

16.3.2 “You cannot derive from two base classes”

- C# enforces **single inheritance**: each class has at most one direct base class. The compiler will reject `class X : A, B`.
- This is a deliberate choice; multiple inheritance complicates what happens when two base classes have a method with the same name (the “diamond problem”).
- If you want to mix capabilities from several places, use **interfaces** — a class can implement any number of them. Interfaces are covered in Lecture 17.

16.3.3 Windows Forms — inheritance you have already seen

Every WinForms form in the labs is an inheritance example:

```
public partial class Form1 : Form
{
    public Form1()
    {
        InitializeComponent();    // constructor code generated by designer
    }

    // your event handlers and methods
}
```

- `Form1` inherits from `System.Windows.Forms.Form`. Without writing a single line, it already has dozens of properties (`Text`, `Size`, `Location`) and dozens of events (`Click`, `Resize`, `Load`).
- The `partial` keyword has nothing to do with inheritance — it just means the class definition is split across multiple files (one you edit, one the designer regenerates). The compiler stitches them back together.

16.3.4 Calling base-class constructors

- The derived class inherits the base class’s properties, so the base class typically needs to initialise them before the derived constructor does its own work.
- Use the `base(...)` keyword on the derived constructor signature: `public Manager(string n, decimal r, decimal b) : base(n, r)`.
- This is good practice even when the base class has a no-arg constructor — writing `: base()` explicitly documents the intent.

16.4 Polymorphism and Overloading

- Polymorphism = many forms; here implemented via overloading.
- Function overloading: multiple methods with the same name but different parameter types/counts.
- Operator overloading: redefine `+` or `-` for custom classes; methods must be public static.
- Constraints: cannot change precedence, invent new operators, or change behaviour for built-in types.

16.5 Enumerations (enum)

- Assign meaningful names to fixed numeric values for readability — common in switch-case.
- Example: `enum Season {Spring, Summer, Autumn, Winter};` — Autumn corresponds to 2.

16.5.1 “Not actually an OOP concept”

- The lecturer pointed out that `enum` is not an OOP feature — it is a C-style fixed integer set with named values. It just happens to be most useful inside the `switch` blocks that classes use to dispatch on state.
- To read an enum value, assign it to a variable of the enum type: `Season s = Season.Autumn;`
- Because each value is implicitly an integer (`Spring=0`, `Summer=1...`), casting between the enum and an `int` is allowed but should be done deliberately — not by accident.
- Compare with “magic numbers”: a parameter of type `int mode = 2` tells the reader nothing; a parameter of type `Season s = Season.Autumn` is self-documenting.

16.6 Coaxial Cable Class Exercise

Model a coaxial cable with two private radius members (inner a, outer b) and public methods to compute physical properties.

Parameter	Formula	Physical meaning
Series Inductance L	$L = (\mu_0 \mu_r \ell / 2\pi) \cdot \ln(b/a)$	Opposition to changes in current; depends on geometry and magnetic properties.
Shunt Capacitance C	$C = (2\pi \varepsilon_0 \varepsilon_r \ell) / \ln(b/a)$	Ability to store charge; depends on geometry and dielectric.

- μ_0 : permeability of free space.
- μ_r : relative permeability (dimensionless, vs. μ_0).
- ε_0 : permittivity of free space.
- ε_r : relative permittivity (dielectric constant).

16.7 Use Cases & Applications

- static class for utilities — Math, File, Console, and your own Helpers — when there’s no per-instance state.
- Inheritance for shared behaviour: an Employee base with `PayEmployee()` and Manager / Cleaner specialisations.
- `const` for compile-time constants like `Math.PI`, conversion factors, or sensor part numbers.
- Operator overloading for math types (`Vector2D`, `Complex`) so client code stays readable: `a + b` instead of `Add(a, b)`.
- `enum` for finite, named state sets — much safer than ‘magic numbers’ inside switch statements.

16.8 Worked Example: Employee Hierarchy

```
public class Employee
{
    public string Name { get; }
    public decimal HourlyRate { get; }

    public Employee(string name, decimal rate)
    { Name = name; HourlyRate = rate; }

    public virtual decimal WeeklyPay(int hours) => HourlyRate * hours;
}

public class Manager : Employee
{
    public decimal Bonus { get; }
    public Manager(string n, decimal r, decimal b) : base(n, r) { Bonus = b; }
    public override decimal WeeklyPay(int h) => base.WeeklyPay(h) + Bonus;
}
```

16.9 Worked Example: Coaxial Cable Model

Translate the coaxial cable formulas into a class with private radii and public computed properties.

```
public class CoaxCable
{
    private readonly double a, b, length;
    private const double mu0 = 4e-7 * Math.PI;
    private const double eps0 = 8.854e-12;
    public double MuR { get; set; } = 1.0;
    public double EpsR { get; set; } = 2.25; // typical PE dielectric

    public CoaxCable(double inner, double outer, double l)
    { a = inner; b = outer; length = l; }

    public double SeriesInductance =>
        (mu0 * MuR * length / (2 * Math.PI)) * Math.Log(b / a);

    public double ShuntCapacitance =>
        (2 * Math.PI * eps0 * EpsR * length) / Math.Log(b / a);
}
```

16.10 Operator Overloading Sketch

```
public struct Vector2D
{
    public double X, Y;
    public Vector2D(double x, double y) { X = x; Y = y; }

    public static Vector2D operator +(Vector2D a, Vector2D b) =>
        new Vector2D(a.X + b.X, a.Y + b.Y);

    public static Vector2D operator *(Vector2D a, double k) =>
        new Vector2D(a.X * k, a.Y * k);
}
```

Key Definitions

Static Member

Member belonging to the class itself, not instances.

Static Method

Utility function that uses only static members.

Static Data Member

Field shared by all instances of a class.

Inheritance

Mechanism letting a derived class acquire base-class members.

Base Class

Class providing common features to derived classes.

Derived Class

Class that inherits from a base class and may extend it.

Function Overloading

Same name, different parameter list.

Operator Overloading

Redefining an operator for custom classes (public static methods).

Enum

Set of named integer constants improving readability.

Single Inheritance

C#'s rule that a class can have at most one direct base class (interfaces are the way to mix multiple contracts).

base Keyword

Used in a derived class to call the base constructor or invoke the base implementation of an overridden method.

readonly

Field modifier making the value settable only in declaration or constructor — a runtime equivalent of const.

Magic Number

Unnamed numeric literal embedded in code; usually replaced with a const or enum for clarity.

Sources

- Lecture 16.pdf
- In-class transcript: *Exploring complex principles and techniques in OOP* — used for the full PascalCase / camelCase / s_ / t_ naming convention table, the “static is called on the class, not the instance” framing, the Office employees example, the WinForms `Form1` : `Form` inheritance example, the single-inheritance rule, and the “enum is not actually an OOP concept” clarification.

Chapter 17

OOP Part C — Virtual, Override, Abstract, Sealed

ENEL712 Embedded Systems Design — Lecture 17

Summary

Polymorphism mechanics in C#: virtual methods and override, the protected modifier, abstract methods/classes, and sealed restrictions.

Topics Covered

- Virtual methods and the override keyword
- Polymorphism (many forms)
- Protected access modifier
- Abstract methods and classes
- Sealed classes and methods

17.1 Virtual Methods and the override Keyword

- Virtual methods let a base class supply a default implementation that derived classes may replace.
- Classic example: `ToString()` inherited from `System.Object` — base returns the class name; derived classes override to return state.
- Syntax: base method uses `virtual`; derived method uses `override`; signatures must match exactly.
- Virtual methods cannot be private. Redefining a virtual method without `override` hides it (compiler warning).

17.1.1 The lecturer's framing: “a placeholder for derived classes”

- “When designing an OO solution, sometimes methods act as placeholders for derived classes to provide their own implementation.”

- If the derived class chooses *not* to override, the placeholder is what runs — the base method provides a usable default.
- This is the defining difference between **virtual** and **abstract**: **virtual** = optional override, **abstract** = mandatory override.

17.1.2 ToString() — the virtual method you inherit for free

Every class in C# derives (eventually) from `System.Object`, and `System.Object` declares `ToString` as virtual:

```
namespace System
{
    public class Object
    {
        public virtual string ToString() { /* returns the type name */ }
    }
}
```

- Because the base method is virtual, any class is free to override it. Because it is virtual (not abstract), no class is *required* to.
- Override it when your class can produce something more useful than the type name — “Circle: r=2.5” is more useful than “Circle”.
- The debugger, `Console.WriteLine`, and `string.Format` all call `ToString()`, so overriding it pays off everywhere automatically.

```
public class Circle
{
    private double r;
    public Circle(double radius) { r = radius; }

    public override string ToString() => $"Circle: r={r}";
}
```

17.1.3 Rules for using virtual methods (don’t get any of these wrong)

- A virtual method **cannot** be **private**. There would be no derived class allowed to see it, so there is nothing to override.
- The override signature must match the base signature *exactly* — same return type, same parameter types in the same order. A mismatch means you have written a completely different method that happens to share a name.
- You can only override methods that the base class marked **virtual** (or **abstract** — see below).
- If you redeclare a method with the same name in a derived class **without** the **override** keyword, you *hide* the base method instead of overriding it. The compiler will emit a warning. **Do not do this** — callers using a base reference will silently get the base implementation, which is almost never what you want.

17.2 Polymorphism (Many Forms)

- Single method call can execute different logic depending on the actual runtime type.

- Calls via a base reference dispatch to the most-derived override at runtime.
- Distinct from overloading, which is compile-time signature matching.

17.3 Protected Access Modifier

- Middle ground between public (everyone) and private (same class only).
- Accessible to the defining class and any derived classes.
- Useful for sharing state with derived classes while staying encapsulated from external code.
- Example: Circle has protected double radius; — BottleCap can use radius for area without exposing it publicly.

17.4 Abstract Methods and Classes

- Abstract methods have no implementation in the base class — derived classes must override.
- Mandatory: not optional like virtual methods.
- Only allowed inside abstract classes — abstract classes cannot be directly instantiated.
- Example: Shape declares abstract void area(); — Circle, Square, Triangle each provide their own.

17.4.1 Virtual vs abstract — one-line difference

Modifier	Base provides an implementation?	Derived class must override?
virtual	Yes, a default	No — override is optional
abstract	No, the body is missing	Yes — otherwise the derived class is itself still abstract

- If Shape declares `abstract double Area();`, `new Shape()` fails to compile — the type is incomplete on purpose.
- Every concrete subclass (Circle, Square, Triangle) is forced to supply an `Area()`, which is the design goal: “every shape has an area, but the base class has no opinion about how to compute it.”

17.5 Sealed Classes and Methods

- Sealed class: no other class can inherit from it — used for security/stability/finalisation.
- Sealed method on an override: no further derived class can override that specific method.
- Useful when a design must remain constant or stable.

17.5.1 In the lecture: “not often used — FYI only”

The lecturer was explicit that `sealed` is included in the course for completeness, not because students are expected to reach for it in everyday work:

- Sealing is a deliberate design choice, not a default. Marking everything sealed by default would make perfectly reasonable subclassing impossible.
- Typical real-world use: a library author marks a class **sealed** because subclassing it would break invariants the author cares about (for example, value-like types such as **string** are sealed for exactly this reason).
- Inside a class hierarchy, marking an individual **override sealed** stops the chain at that level — the derived class still overrides the base method, but no further derived class can override it again.
- For most application code in this course, you will never need to type **sealed**. Recognise it when you read someone else's code; that is enough.

```
public sealed class Square : Shape    // no class can inherit from Square
{
    public override double Area() => Side * Side;
}

public class Animal
{
    public virtual void Speak() { }
}

public class Dog : Animal
{
    public sealed override void Speak() { }    // no further class can override Speak
}
```

17.6 Use Cases & Applications

- Logging: override `ToString()` on every domain class so log messages and the debugger show meaningful state automatically.
- Plug-in shapes/strategies: an abstract `Shape` with abstract `Area()` lets you add new shapes without changing the renderer.
- Library APIs: marking a class sealed prevents users from subclassing in ways the author hasn't anticipated; useful for value-like types.
- Protected helpers: a base `ConfigStore` can expose a protected `ReadRaw()` to subclasses so each format (JSON, INI, ...) shares the file-handling code without exposing it publicly.

17.7 Worked Example: Shape Hierarchy with Polymorphism

```
public abstract class Shape
{
    public abstract double Area();
    public override string ToString() =>
        $"{GetType().Name}: area = {Area():F2}";
}

public class Circle : Shape
{
    public double R { get; }
    public Circle(double r) { R = r; }
```

```

    public override double Area() => Math.PI * R * R;
}

public sealed class Square : Shape
{
    public double Side { get; }
    public Square(double s) { Side = s; }
    public override double Area() => Side * Side;
}

Shape[] shapes = { new Circle(2), new Square(3) };
foreach (var s in shapes) Console.WriteLine(s);    // dynamic dispatch via Area()

```

17.8 Common Pitfalls

- Hiding instead of overriding: if you redeclare a base method without override, the base version still runs through a base reference — the compiler warns about this.
- Calling virtual methods from a base constructor runs the derived override before the derived class is fully initialised — surprising behaviour.
- Sealing a class everyone needs to extend; sealing is a design statement, not a default.
- abstract classes cannot be instantiated directly: `new Shape()` doesn't compile.

Key Definitions

Polymorphism

Single method name dispatching to different logic at runtime depending on object type.

virtual

Modifier marking a base method as overridable by derived classes.

override

Modifier replacing a virtual/abstract method in a derived class.

protected

Access modifier limiting members to the class and its derived classes.

abstract

Modifier marking an incomplete method/class; abstract methods must be overridden.

sealed

Modifier preventing inheritance (on a class) or further override (on a method).

Static (Compile-time) Polymorphism

Overload resolution decided at compile time based on argument types.

Dynamic (Runtime) Polymorphism

Method dispatch decided at runtime using the actual object's type, enabled by virtual/override.

Liskov Substitution Principle

Derived objects should be usable wherever a base reference is expected, without surprising the caller.

Interface

C# construct similar to a fully abstract class; supports multiple inheritance of contracts.

Sources

- Lecture 17.pdf
- In-class transcript: *Exploring complex principles and techniques in OOP* — used for the “placeholder for derived classes” framing of virtual methods, the `System.Object.ToString` inheritance walkthrough, the rules-for-virtual-methods checklist (including the explicit “do not hide instead of override” warning), the virtual-vs- abstract one-line comparison, and the “sealed is FYI only” positioning.

Chapter 18

.NET Framework & Automatic Memory Management

ENEL712 Embedded Systems Design — Lecture 18

Summary

Introduction to .NET, the Common Language Runtime, JIT compilation, namespaces, and how the Garbage Collector manages the managed heap (with optimisations and unsafe code).

Topics Covered

- Introduction to .NET
- Core .NET components
- Common Language Runtime (CLR)
- .NET Class Library and data types
- Namespaces
- Automatic memory management (GC)

18.1 Introduction to .NET

- Microsoft framework providing a consistent OOP environment.
- Goals: minimise deployment issues; consistent dev experience across Windows and Web; high performance, security, and integration on industry standards.
- Recent evolution: open-source and cross-platform — ported to Linux/Mac; server-side .NET core open-sourced (e.g., Roslyn).

18.1.1 .NET Framework vs .NET — the distinction that matters in this course

The lecturer opened by emphasising that “everything we’re doing in this course is .NET Framework, not .NET”. They are related, but they are not the same thing, and choosing between them affects which projects you can actually build.

Aspect	.NET Framework (4.8 / 4.8.1)	.NET (5, 6, 7, 8...)
Platforms	Windows only	Cross-platform (Windows / Linux / macOS)
Status	Maintenance only — no new features	Actively developed; one major version per year
WinForms	Supported (full designer integration)	Supported but not the focus
ASP / Silverlight / ActiveX	Supported (legacy)	Not supported
Recommended version	4.8 (4.8.1 has minor instabilities)	Latest LTS

- **Why this course picks .NET Framework:** the labs use Windows Forms. WinForms is fully supported by the .NET Framework designer and is the fastest way to put a clickable UI in front of an embedded device, which is the lab goal.
- Modern *.NET* also runs WinForms (since .NET Core 3), but the tooling story is different and many of the classes/types in older WinForms examples are .NET-Framework-flavoured.
- Legacy technologies like **ASP (classic)**, **ActiveX**, and **Silverlight** were all built on .NET Framework. They are deprecated; if you encounter them in real codebases, the underlying runtime is .NET Framework.
- “Future development: no new features, not recommended for new projects” is what Microsoft now writes on the .NET Framework download page. For this course, that warning is acceptable; for new commercial work, prefer modern .NET.

18.1.2 NuGet — “it is like pip”

- NuGet is .NET’s package manager. The lecturer’s analogy: “if you’ve used Python’s pip, NuGet is the same idea for C#.”
- Two ways to use it: a **console** (**Install-Package ...**) similar to pip on the command line, and a **desktop GUI** inside Visual Studio (right-click the project, *Manage NuGet Packages*).
- Packages add classes, types, and sometimes designer support to your project. Some of the lab dependencies (custom controls, helper libraries) come through NuGet.

18.2 Core .NET Components

- Common Language Runtime (CLR): execution engine, runs code, provides services.
- Common Type System (CTS): shared data types across .NET languages.
- .NET Class Library: comprehensive collection of OO types for app development.

18.3 Common Language Runtime (CLR)

CLR handles memory management, thread execution, code-safety verification, compilation, and security.

18.3.1 How the CLR works

- C#/VB.NET → compiled to Common Intermediate Language (CIL/MSIL).
- At runtime, CLR JIT-compiles CIL to native machine code for the host architecture.

- CLR is Microsoft’s implementation of the Common Language Infrastructure (CLI) — also implemented by Mono on Linux.

18.3.2 Native vs JIT vs interpreted — where C# sits

The lecturer used three reference points to put C#’s execution model in context:

- **Native C** — the source is compiled all the way to machine code before the program runs. Fastest at runtime; you pay the compile cost up front.
- **Interpreted languages (classic BASIC)** — no compile step; every statement is translated to machine code *at runtime*, every time it runs. Easiest to write, slowest to execute.
- **Just-In-Time (C#, Java)** — the source compiles to a portable intermediate language (CIL), which is translated to machine code *the first time each method runs*. Subsequent calls reuse the cached native code. Slower than native, much faster than interpreted, and benefits from platform-specific optimisations performed at install time.

The trade-off is deliberate: by deferring the final compile, the CLR can produce machine code optimised for the actual CPU the user has, and the same intermediate code runs on any machine that has a CLR.

18.3.3 Common Type System — “write once in any .NET language”

- The CTS is the rule that says “an `int` in C# and an `Integer` in VB.NET are the *same type* as far as the runtime is concerned.”
- This is what lets you call a VB.NET DLL from C# (or vice versa) with no marshalling layer in between.
- Practical consequence: the lecturer noted that even though C-style `uint` maps to a perfectly valid CTS type (`UInt32`), Microsoft’s coding guidelines recommend sticking with `int` **even when your values are always non-negative**, purely for CTS-style normalisation across languages. Use `uint` only if you genuinely need the full 32-bit unsigned range.

18.3.4 Pros & cons

Feature	Description
Productivity	Devs focus on logic, not low-level memory.
Language interop	Multiple languages compile to one CIL description.
Memory safety	Virtually eliminates leaks and invalid references.
Performance hit	Managed code generally slower than native.
Complexity	Interfacing with legacy native code requires marshalling.

18.4 .NET Class Library and Data Types

- CLS-compliant classes/interfaces for File I/O, strings, collections, DBs, GUIs.
- Integers: `Byte` (8), `Int16` (short), `Int32` (int), `Int64` (long).
- Floats: `Single` (float), `Double`.
- Other: `Bool`, `Char` (16-bit Unicode), `String`, `Object` (root).

18.5 Namespaces

- Containers grouping related classes — like folders for thousands of classes.
- Hierarchy with dots (e.g., `System.IO.Stream`).
- using directive imports a namespace at the top of a file.

18.6 Automatic Memory Management

18.6.1 Allocation on the managed heap

- On process start, the runtime reserves a contiguous managed heap.
- `NextObjPtr` tracks the next free address — allocation is fast (consecutive layout).
- Faster than C-runtime heap, which searches a linked list for a fitting block.

18.6.2 In-class framing: “the heap has a shape”

The lecturer drew a deliberate contrast between the C-runtime heap and the .NET managed heap:

- C / C++ use a **linked-list heap**. When you call `malloc`, the allocator walks a linked list of free blocks looking for one big enough. The free list fragments over time as objects are allocated and freed in a random order.
- .NET uses a **contiguous managed heap**. The runtime keeps a single pointer (`NextObjPtr`) to the next free byte. Each `new` just bumps the pointer forward — no list traversal, no fitting search.
- Consequence: “the operation of putting data onto and out of the managed heap is approaching that of the stack.” Allocation in C# is roughly as cheap as pushing a stack frame.
- The price is the garbage collector — the contiguous layout only works if the runtime can periodically compact the heap. That is what the GC is doing in the background.
- Net effect: “swings and roundabouts. Data in and out is quicker because of the reference point, but because of the garbage collection, it’s slower overall” compared to a well-tuned native allocator. For application code this is invisible; for hard real-time code it matters.

18.6.3 Garbage Collection (GC) algorithm

- Build object list: assume all objects are garbage; trace from Application Roots (globals/statics, locals on stack, CPU registers) to find reachable objects.
- Free and compact: shift non-garbage objects down to overwrite freed space.
- Update pointers: roots and internal references are rewritten to new locations.

18.6.4 In-class walkthrough: “assume everything is garbage”

The lecturer described the algorithm in two lines:

“The algorithm starts off by assuming that *everything* is garbage. It only doesn’t get rid of an object when it finds a root. If a root is found, it’s marked as not garbage. If no roots can be found and there are no references to it, it gets left, marked as garbage, and deleted.”

- Roots = anything the program can reach without help: globals, static fields, locals currently on the stack, parameters in CPU registers.
- Reachability is **transitive**: if object D is reachable from a root and D holds a reference to object H, then H is also reachable, even if nothing outside D refers to H.
- Anything not transitively reachable from any root is garbage. Objects P, A, G, I, J in the slide diagram are reclaimed because no live reference can reach them; H survives because D keeps it alive.
- After collection, surviving objects are slid down to close the gaps (*compaction*). Every reference to a moved object is rewritten so the program never notices.

18.6.5 Pointers and **unsafe** — and why you almost never need them

- C# does support raw C-style pointers, but only inside a block marked **unsafe**, and the entire project must be compiled with the `/unsafe` option.
- “Once a pointer is created, that memory is used. If that memory is not completely isolated, there’s a chance another thread can come along and stomp your memory location” — the lecturer’s plain-English version of “raw pointers are not thread safe.”
- Worse, the GC may move the object the pointer used to point at, leaving the pointer dangling. The fix is `GCHandle.Alloc(..., GCHandleType.Pinned)` which tells the GC not to move that specific object.
- Realistic use cases: interop with legacy unmanaged libraries, or extreme performance-critical inner loops where you have measured a problem. “It is extremely unlikely that there will be any situations where you’ll need to do this” inside the scope of this course.

18.6.6 Threading inside the CLR (and the timer-vs-comm bug)

- The CLR provides built-in support for threads, which the lecturer described as “allowing unutilised clock cycles to be filled up” — seemingly parallel, actually interleaved on a single core.
- Concrete embedded scenario from class: a WinForms app has a timer that polls a serial port. If the serial read has not finished and the timer fires again, the second thread can “smash what you’re currently doing”, causing the program to hang or crash.
- Two defences: (1) make sure the timer handler returns quickly so the next tick has clean state to work with, or (2) protect the shared resource with a lock or by setting a busy flag.
- This is why the lecturer’s threading rule of thumb is “threads share the heap; locals are safe; anything shared needs a story.”

18.6.7 Optimisations and **unsafe** code

- Generational GC: heap split by object age; collections focus on young (Gen 0).
- Pointers in **unsafe** blocks risk dangling references when GC moves objects.
- `GCHandle` pins memory so the GC won’t move specific objects (used with native interop).

18.7 Use Cases & Applications

- Most desktop applications, web services, and tooling on Windows run on the CLR — knowing how it works helps you debug startup time, deployment size, and memory usage.

- Modern .NET also targets Linux/macOS — useful for Docker images and cloud-native services.
- GCHandle / Marshal lets you safely pass managed memory to native libraries (e.g., USB drivers, vendor SDKs).
- Generational GC means small, short-lived allocations are essentially free; large or long-lived ones (caches, big buffers) deserve more thought.

18.8 Worked Example: Tracing a GC Pass

Imagine a method allocates 1,000 small temporary strings for parsing. After the method returns, all 1,000 are unreachable.

- All 1,000 are placed in Generation 0.
- When Gen 0 fills, GC traces from application roots — the parsing method has returned, so none of the 1,000 are reachable.
- GC frees their memory and compacts Gen 0; survivors (almost none here) get promoted to Gen 1.
- Net effect: cleanup is microseconds and developers never write a `free()` call.

18.9 Code Snippet: Pinning Memory for Native Interop

```
byte[] buf = new byte[1024];
GCHandle h = GCHandle.Alloc(buf, GCHandleType.Pinned);
try
{
    IntPtr ptr = h.AddrOfPinnedObject();
    NativeApi.WriteBytes(ptr, buf.Length); // safe -- GC won't move buf
}
finally
{
    h.Free();
}
```

18.10 Practical Notes

- Don't worry about object allocation cost in normal logic — premature 'optimisation' against the GC usually hurts more than it helps.
- Do worry about retention: long-lived caches that grow without bound, or event handlers you never unsubscribe, become Gen 2 garbage that's expensive to collect.
- Use the `dotnet-counters` / `PerfView` tools to inspect Gen 0/1/2 allocation rates in real apps.

Key Definitions

Application Roots

Storage locations (globals/statics, locals, CPU regs) the GC traces from.

CIL

Common Intermediate Language — platform-neutral bytecode that all .NET languages emit.

CLI

Common Language Infrastructure — open spec defining .NET’s runtime and code format.

GCHandle

Managed type that pins an object so the GC cannot move it.

Generational GC

Optimisation splitting the heap by object age for faster collections.

JIT Compilation

Runtime translation of CIL to native machine code.

Managed Heap

Contiguous memory region allocated by the runtime for reference types.

Marshalling

Layer/process for interop between managed and native code.

Application Domain

(Legacy .NET Framework) Logical container for code/assemblies; modern .NET uses `AssemblyLoadContext` instead.

IDisposable

.NET interface for releasing unmanaged resources (files, sockets) deterministically via using or `Dispose()`.

Finalizer

Method run by GC before reclaiming an object; rarely needed when `IDisposable` is used correctly.

Roslyn

Microsoft’s open-source C# / VB.NET compiler platform that exposes parsing and analysis APIs.

Sources

- [Lecture 18.pdf](#)
- In-class transcript: *.NET and C# concepts* — used for the .NET Framework vs .NET (cross-platform) distinction, the WinForms / ASP / Silverlight history, the NuGet = pip analogy, the contiguous-managed-heap vs linked-list comparison, the native-vs-JIT-vs-interpreted positioning, the CTS “prefer `int` over `uint`” guideline, the “assume everything is garbage” GC explanation, and the timer-vs-communication threading example.

Chapter 19

C# Fundamentals — Syntax, Types, Methods, Arrays

ENEL712 Embedded Systems Design — Lecture 19

Summary

Core C# language fundamentals: syntax rules, data types and conversion, methods and parameter passing (ref/out/params), operators, arrays (with stack/heap semantics), and exception handling.

Topics Covered

- C# fundamentals and syntax
- Variable naming
- Data types and conversion
- Methods and parameters
- Operators and mathematical functions
- Arrays
- Exception handling

19.1 C# Fundamentals and Syntax

- Statements end with `;`. Whitespace doesn't terminate; statements can span multiple lines.
- Identifiers are case-sensitive and must start with a letter or underscore.
- Comments: `//` single-line; `/* ... */` multi-line.
- Reserved keywords: `abstract`, `class`, `if`, `namespace`, `public`, `static`, `void`, etc.

19.1.1 The keyword that should usually scare you: `goto`

The lecturer highlighted `goto` in red on the reserved-keyword slide and asked the class *why*.

- “`goto` is a real relic. I've been programming for many, many years and other than back in the day with BASIC, I've never needed to use `goto` ever.”

- Assembly equivalent: `JMP` (“jump”). At the machine level, every branch is a `goto`; high-level languages exist precisely so you do not have to write them by hand.
- Modern C# does include `goto` — a handful of *very* narrow uses inside `switch` statements (`goto case`, `goto default`) are sometimes acceptable. Anything that looks like classic unstructured branching is almost certainly a code smell.

19.2 Variable Naming

- Start with a lowercase letter; avoid leading underscores.
- Avoid names differing only in case (`var` vs `Var`).
- Use camelCase for multi-word variables (`myVar`, `fileID`).

19.3 Data Types and Conversion

Type	Kind	Size (bits)	Range / precision
<code>byte</code>	Whole numbers	8	0 to 255
<code>int</code>	Whole numbers	32	-2^{31} to $2^{31}-1$
<code>long</code>	Whole numbers	64	-2^{63} to $2^{63}-1$
<code>float</code>	Floating point	32	—
<code>double</code>	Floating point	64	—
<code>decimal</code>	Monetary	128	28 significant figures
<code>char</code>	Single character	16	0 to $2^{16}-1$
<code>string</code>	Sequence of chars	16/char	—
<code>bool</code>	Boolean	8	true / false

- Unlike C++, using an unassigned local variable is a compile-time error.
- Implicit cast when no information is lost (`float` $\rightarrow$ `double`); explicit cast otherwise.
- `Convert.ToInt32()` (returns 0 for nulls) vs `Int32.Parse()` (throws on null).
- Numbers to strings: `.ToString()`.

19.3.1 Float to double vs double to float — “you’re losing fidelity”

The lecturer’s framing of why one direction needs an explicit cast and the other does not:

- `float` $\rightarrow$ `double`: **implicit** cast. “Not an issue because we are not losing fidelity” — every `float` value is representable as a `double`.
- `double` $\rightarrow$ `float`: **explicit** cast required. “We are losing information, and the explicit cast is the language saying *I know I’m losing information and I’m willing to accept it.*”
- Without the explicit cast you would get a warning or a compile error — the language refuses to silently narrow.

19.3.2 `Convert.ToInt32` vs `int.Parse` — the exception-vs-zero choice

- Both convert a `string` into an `int`; the difference is what they do when the input is not a number.
- `Convert.ToInt32("hello")` — “just does it, zero.” Returns 0 silently. Easy to misuse.

- `int.Parse("hello")` — “throws an exception.” Loud failure you must handle (or wrap in `try/catch`).
- `int.TryParse(..., out int n)` is the modern compromise: no exception, no silent zero — a `bool` success flag and an `out` parameter that receives the value on success.
- All numeric types have a corresponding `ToString()` method (a *method*, hence the parentheses). It accepts an optional format specifier: `x.ToString("F2")` returns the number formatted to two decimal places.

19.4 Methods and Parameters

- C# methods pass by value by default — receiver gets a copy.
- `ref`: pass by reference; variable must be initialised before passing.
- `out`: pass by reference but allows unassigned variables; must be initialised inside the function.
- `params`: variable number of arguments received as a single array (distinct from overloading).

19.4.1 Why `ref` exists — “behaves like a pointer”

The lecturer walked through a deliberately odd example:

```
void NotUseful(int x) { x = x + 1; }    // adds 1 to a local copy, throws it away
```

- The method receives a *copy* of `x`. Adding 1 to the copy and returning `void` means the call site sees no change. “Literally no point in doing this.”
- Add the `ref` keyword and the method now “works directly on the memory allocation for that parameter” — “it’s essentially behaving like a pointer.”

```
void Increment(ref int x) { x = x + 1; }    // mutates the caller's variable

int y = 10;
Increment(ref y);
// y is now 11
```

- “Do we have pointers as well when about the `ref` keyword? Generally not.” `ref` is the safe, GC-aware substitute that lets a method update a value-type variable in place without exposing a raw memory address.

19.4.2 `out` — the deliberate “not initialised yet” contract

- The key difference: a `ref` parameter must be initialised *before* the call; an `out` parameter *may* be uninitialised before the call but **must be** assigned inside the method before it returns.
- Practical use case: the *TryParse* pattern. `int.TryParse(s, out int n)` can return without ever reading the previous value of `n`; it must, however, write a value to `n` on every code path before returning.
- Otherwise `out` behaves like `ref`: the method works on the caller’s variable directly.

19.5 Operators and Mathematical Functions

19.5.1 Operator precedence (high → low)

- Primary: `x.y`, `f(x)`, `a[x]`, `x++`, `new`
- Unary: `+`, `-`, `!`, `~`, `++x`, `--x`, `(T)x`
- Multiplicative: `*`, `/`, `%`
- Additive: `+`, `-`
- Relational: `<`, `>`, `<=`, `>=`, `is`, `as`
- Logical: `&`, `^`, `|`
- Conditional: `&&`, `||`, `?:`
- Assignment: `=`, `*=`, `/=`, `%=`, `+=`, `-=`, ...

19.5.2 Math, randoms, dates

- `System.Math`: `Abs`, `Sin`, `Sqrt`, `Pow`...
- Random class: pseudo-random ints/doubles.
- `DateTime`: dates/times/leap years (e.g., `DateTime.Now`, `AddDays`).

In-class question — why seed `Random` with the current time?

The lecturer asked the class why their lab code seeded the random number generator with the current system time. A student gave the correct answer immediately:

- `Random` is a **pseudo-random** generator — the sequence it produces is completely determined by its initial seed.
- Construct `new Random(42)` twice in two different runs and you will get the *same sequence of numbers* twice. Useful for reproducible tests, awful for anything that needs variety.
- Seeding with `DateTime.Now.Ticks` (or, in modern code, the default constructor `new Random()`, which does this internally) ensures that every run starts from a different point in the pseudo-random stream.
- For *cryptographic* randomness, do not use `Random` — use `System.Security.Cryptography.RandomNumberGenerator`.

19.6 Arrays

- Ordered sequence of same-type elements stored contiguously.
- Declare with brackets after the type (`int[] pins;`); allocate with `new`.
- Reference lives on the stack; element memory lives on the heap. Numeric elements default to 0.
- Iterate with `for + .Length` or with `foreach`.
- Assignment copies the reference; use `pins.CopyTo(copy, 0)` to duplicate contents.
- Multidimensional rectangular arrays (`int[,]`) and jagged arrays (arrays of arrays).

19.6.1 Declare vs initialise — two separate steps

```
int[] pens;                // (1) declared but EMPTY -- no heap memory yet
pens = new int[4];         // (2) now allocated -- 4 ints on the heap, all 0
int[] morePens = { 9, 13, 7, 2 }; // declaration + initialiser in one step
```

- Line (1) creates only a reference on the stack. There is no array on the heap yet; reading or writing to `pens[0]` would throw `NullReferenceException`.
- Line (2) uses `new int[4]` to allocate four ints on the heap. The number “4” is the *size* of the array, not its content; every element defaults to 0.
- The shorthand `{ ... }` initialiser declares, allocates, and assigns in one step. The size is inferred from the number of items in the braces.

19.6.2 for with .Length vs foreach

```
// Classic for-loop --- explicit index
for (int i = 0; i < pens.Length; i++)
    Console.WriteLine(pens[i]);

// foreach --- one variable, no index
foreach (int pen in pens)
    Console.WriteLine(pen);
```

- Both loops produce identical output for a simple iteration.
- `.Length` is a **property** on every array. “We don’t need to memorise what the length of an array is” — read it from the array itself.
- `foreach` is much more compact and the lecturer preferred it. The trade-off: you do not get the index, and you cannot *mutate* the array element through the loop variable.
- Convention: the plural variable name is the array (**pens**); the singular is the loop variable (**pen**). This is purely a readability convention, not a language rule.

19.6.3 Multidimensional and jagged arrays

```
int[,] matrix = new int[4, 6]; // rectangular: 4 rows of 6
int[][] jagged = new int[3][]; // 3 rows, each of independent length
jagged[0] = new int[] { 1, 2 };
jagged[1] = new int[] { 1, 2, 3, 4 };
jagged[2] = new int[] { 1 };
```

- Rectangular arrays use commas inside the brackets: `int[,]` for 2D, `int[,,]` for 3D. Iterating them needs one nested `for` per dimension — a 4 by 6 matrix needs two nested loops.
- **Jagged arrays** (`int[][]`) are arrays of arrays. Each inner array can have its own size. The lecturer’s two-word verdict: “don’t use them.” The memory layout is awkward (each inner array is a separate heap object), and rectangular arrays cover almost every real use case.

19.6.4 params — not the same as overloading

```
int Sum(params int[] numbers)
{
    int total = 0;
    foreach (int n in numbers) total += n;
}
```

```

    return total;
}

Sum();           // numbers is an empty array
Sum(1);          // numbers is { 1 }
Sum(1, 2, 3, 4); // numbers is { 1, 2, 3, 4 }

```

- `params` lets the same method accept any number of arguments — the compiler packs them into an array for you.
- This is **different from overloading**. With overloading you have several separate methods that each accept a fixed but different parameter list (potentially of *different types*). With `params` you have a single method whose parameters are all the same type.
- Familiar example: `Console.WriteLine(format, params object[] args)` — that is why `Console.WriteLine("{0} + {1} = {2}", a, b, a+b)` works.

19.7 Exception Handling

- `try` block contains code that may throw.
- `catch` block handles specific exceptions (e.g., `FormatException`) or all via `catch(Exception ex)`.
- `ex.Message` provides error details for logging or user feedback.

19.7.1 Multiple catch blocks — “you can overload your catches”

```

try
{
    int n = int.Parse(textBox.Text);
    int avg = total / n;
    File.WriteAllText("result.txt", avg.ToString());
}
catch (FormatException fx)
{
    MessageBox.Show("Please enter a number: " + fx.Message);
}
catch (DivideByZeroException dx)
{
    MessageBox.Show("Cannot divide by zero: " + dx.Message);
}
catch (IOException ix)
{
    MessageBox.Show("Could not write result file: " + ix.Message);
}
catch (Exception ex)           // generic fallback --- last
{
    MessageBox.Show("Unexpected error: " + ex.Message);
}

```

- A single `try` can be followed by any number of `catch` blocks. The runtime picks the first one whose exception type matches.
- The lecturer’s strong recommendation: “catching specific exceptions is recommended; doing a generic catch is actually not recommended.” A bare `catch (Exception ex)` can hide bugs (null reference, stack overflow) that you really want to know about.

- Order the catches **specific to general**. If you put `catch (Exception)` first, it will swallow every exception and the more specific catches below it will never run.
- Variable names: `e`, `ex`, or a per-exception name (`fx`, `dx`, `ix`) all work. `ex.Message` carries the human-readable text; `ex.StackTrace` and `ex.InnerException` carry the rest of the story.

19.8 Use Cases & Applications

- ref params: useful for in-place updates of a struct that would otherwise be copied (e.g., a large `GameState`).
- out params: classic Try-pattern, e.g., `int.TryParse(s, out int n)` — succeeds without throwing.
- params arrays: variadic methods like `Console.WriteLine(format, params object[] args)`.
- Type conversions: `Int32.Parse` vs `Convert.ToInt32` — `Parse` is strict, `Convert` is more lenient and may swallow nulls.
- Jagged arrays: triangular tables (e.g., Pascal's triangle, lower-triangular matrices) where each row has a different length.

19.9 Worked Example: TryParse Pattern

```
if (int.TryParse(textBox.Text, out int hours))
    label.Text = $"You entered {hours} hours.";
else
    label.Text = "Please enter an integer.";
```

`TryParse` is preferred over `Parse` + `try/catch` in hot paths — exceptions are expensive in .NET (kilobytes of stack walk per throw).

19.10 Worked Example: Copy-by-Reference Pitfall

```
int[] a = { 1, 2, 3 };
int[] b = a;           // copies the reference, not the data
b[0] = 99;
Console.WriteLine(a[0]); // prints 99

int[] c = (int[])a.Clone(); // or new int[] { ... }
c[0] = -1;
Console.WriteLine(a[0]); // prints 99 (independent now)
```

19.11 Operator Precedence Cheat Sheet

Group (high → low)	Operators
Primary	<code>x.y</code> , <code>f(x)</code> , <code>a[x]</code> , <code>x++</code> , <code>new</code>
Unary	<code>+</code> , <code>-</code> , <code>!</code> , <code>~</code> , <code>++x</code> , <code>-x</code> , <code>(T)x</code>
Multiplicative	<code>*</code> , <code>/</code> , <code>%</code>
Additive	<code>+</code> , <code>-</code>
Relational	<code><</code> , <code>></code> , <code><=</code> , <code>>=</code> , <code>is</code> , <code>as</code>
Logical	<code>&</code> , <code>^</code> , <code> </code>
Conditional	<code>&&</code> , <code> </code> , <code>?:</code>
Assignment	<code>=</code> , <code>*=</code> , <code>/=</code> , <code>%=</code> , <code>+=</code> , <code>-=</code> , ...

Key Definitions

ref Method parameter modifier passing by reference; arg must be initialised.

out

Method parameter modifier passing by reference; arg may be unassigned but must be set inside.

params

Modifier letting a method accept a variable number of args as a single array.

Stack

LIFO memory region for locals and reference variables.

Heap

Memory region for dynamic allocation (e.g., array contents and class instances).

Dynamic Memory Allocation

Allocating memory at runtime, typically via `new`.

Jagged Array

Array of arrays where inner arrays may have different lengths.

Operator Precedence

Rules for the order in which operators evaluate.

System.Math

Static class with mathematical constants and functions.

Exception Handling

try/catch mechanism for handling runtime errors gracefully.

Boxing

Wrapping a value-type instance into a heap-allocated object; happens when you assign an `int` to an object variable.

Unboxing

Extracting the value from a boxed object back into its value type; requires a runtime cast.

params Modifier

Lets a method accept a variable number of arguments collected into an array.

TryParse Pattern

Static method returning `bool` that signals success and writes the parsed value to an `out` parameter.

Sources

- Lecture 19.pdf
- In-class transcript: *.NET and C# concepts* — used for the `goto = JMP` aside, the float-to-double “losing fidelity” explanation, the `Convert.ToInt32` vs `int.Parse` side-by-side, the “`ref` behaves like a pointer” framing of method parameters, the `out` vs `ref` contract, the “why seed Random with the current time” exchange, the declare-vs-initialise array steps, the `foreach`-preferred-over-for-loop framing, the “don’t use jagged arrays” verdict, the `params` clarification, and the “overload your catches” multiple- catch recommendation.

Chapter 20

C# Strings, Structures & Collections (with Embedded Review)

ENEL712 Embedded Systems Design — Lecture 20

Summary

C# strings, structures, and collection classes, paired with a recap of foundational embedded concepts (MCU architecture, memory, comms, and software design).

Topics Covered

- C# strings
- C# structures (struct)
- Collection classes
- Embedded review: MCU architecture and operation
- Embedded review: MCU memory types
- Embedded review: digital communications
- Embedded review: software design and estimation

20.1 C# Strings

- String = specialised array of 16-bit UTF-8 characters (backward-compatible with ASCII).
- Concatenate with +.
- Substring(startIndex, count) extracts.
- IndexOf(substring) is case-sensitive; returns -1 if not found.
- Trim/TrimStart/TrimEnd manage whitespace.
- Replace and Split modify or partition strings.

20.1.1 “Strings are completely immutable”

The lecturer was emphatic about this and it answers a lot of confusing behaviour:

```
string s1 = "hello ";
string s2 = "world";
s1 = s1 + s2;           // looks like we mutated s1
```

- “If I write `s1 = s1 + s2`, what happens to `s1`? The original `s1` is overwritten with a *new* `s1`, and the original is given up for garbage collection.”
- Strings *look* mutable because the variable can point at a new string, but the original sequence of characters is never modified in place.
- Consequence: building a long string with `+=` inside a loop allocates a new string on *every* iteration. For anything bigger than a handful of joins, use `StringBuilder`.
- The plus operator is being **overloaded** for strings — it does not arithmetic-add anything, it concatenates. This is one of the few cases of operator overloading you use every day without thinking about it.

20.1.2 Three kinds of string literal

C# supports three syntaxes for writing a string literal in source code. The right choice depends on what is *inside* the string.

1. Regular string — with escape sequences

```
string a = "Hello\nWorld"; // \n becomes a real newline
```

The compiler interprets backslash escape sequences (`\n`, `\t`, `\"`, etc.). Use this for “normal” strings.

2. Verbatim string — with the `@` prefix

```
string path  = @"C:\Users\aaaron\Documents\file.txt";
string lines = @"line one
line two
line three";
```

- Prefix with `@`. Backslashes are taken literally — no escape sequences. Newlines in the source are preserved.
- The lecturer’s main use cases: Windows file paths and multi-line text where you do not want to litter the literal with `\n`.

3. Raw string literal — triple quotes (C# 11+)

```
string json = """
    {
        "Name": "Alice",
        "Age": 25
    }
    """;
```

- Open and close with at least three double-quotes. Everything between them is taken *exactly* as written, including any internal double-quotes.

- Multi-line raw strings have rules: the opening `"""` should sit on its own line, the closing `"""` should sit on its own line, and the indentation of the closing delimiter is stripped from every content line. Violating either rule is a compile error.
- Use case: pasting JSON, XML, regular expressions, or any multi-line text that already contains quotes and backslashes. Verbatim strings still need `"""` to embed a quote; raw strings just let you type the quote directly.

20.1.3 String interpolation — the `$` prefix

```
int x = 2;
string a = "x = " + x.ToString();    // old style: concatenate + ToString
string b = $"x = {x}";               // new style: interpolation
```

- Prefix the literal with `$` and embed any expression in braces. The compiler builds the `ToString` + concatenation for you.
- Format specifiers work inside the braces: `($"{x:F2}")` formats `x` with two decimal places.
- Interpolation can be combined with verbatim (`$@"..."`) or raw (`$"""..."""`) literals. Each gives the same string back, just with different readability.

20.1.4 Quick reference for the main string methods

Method	Behaviour
<code>s.Substring(start, count)</code>	Returns the slice starting at <code>start</code> for <code>count</code> characters.
<code>s.IndexOf("xy")</code>	Returns the index of the first match, or <code>-1</code> if not found. Case-sensitive.
<code>s.Trim()</code> / <code>TrimStart()</code> / <code>TrimEnd()</code>	Removes whitespace from both ends / start / end.
<code>s.Replace(old, new)</code>	Returns a new string with every occurrence of <code>old</code> replaced.
<code>s.Split(' ')</code>	Returns a <code>string[]</code> cut at every space (delimiter can be anything).

Every one of these returns a *new* string — again, because strings are immutable. The original `s` is unchanged.

20.2 C# Structures (struct)

- Allow grouping fields of multiple data types (unlike arrays).
- Stored on the stack — efficient for small data.
- If a constructor is used, all fields must be initialised; otherwise default constructor zeros the fields.
- Value type — copies are real copies, but only when initialised.

20.2.1 Struct vs array — “different data types in one place”

- An array is a fixed-size collection of items *of the same type*. `int[]` cannot hold a string in slot 2.
- A struct lets you group a fixed set of fields, each of *whatever type makes sense*: an `hours`, `minutes`, `seconds` time value, or an `X`, `Y`, `Z` 3D point.

- Struct = value type. The variable holds the actual data, not a reference to it. Assignment, parameter passing, and return all copy the bytes.

20.2.2 The all-or-nothing constructor rule

```
public struct Time
{
    public int Hours, Minutes, Seconds;

    public Time(int h, int m, int s)
    {
        Hours    = h;
        Minutes   = m;
        Seconds   = s;    // ALL fields must be assigned before the constructor returns
    }
}
```

- In a class constructor you can leave fields unassigned — the runtime defaults them to 0, false, or null.
- In a struct constructor, the compiler refuses to let you return without assigning *every* field. “It’s all or nothing.”
- The reason: the default `new Time()` (with no constructor arguments) already produces a valid all-zeros struct. If your custom constructor could leave fields unassigned, two different `new Time(...)` calls could produce different “defaults” — confusing and bug-prone.

20.3 Collection Classes

Collection	Use
ArrayList	Dynamic-size lists; insertion/removal/shuffling.
Dictionary	Key-value pairs; very fast lookup via hash table.
Queue	First-In-First-Out (FIFO).
Stack	Last-In-First-Out (LIFO).
Hashtable / SortedList	Hashtable allows any data type as keys; SortedList is a sorted Hashtable.

20.3.1 ArrayList — “a more flexible array”

- Behaves like an array, but the size is not fixed at construction. You can **Add**, **Insert**, and **Remove** items at runtime.
- Built-in helpers like **Sort** are available, which plain `int[]` can also do via the static `Array.Sort` method.
- Supports **foreach** just like an array.
- Caveat: **ArrayList** (the non-generic version) stores everything as **object**, which means every value-type insertion is boxed onto the heap. In new code, prefer the generic `List<T>` — same behaviour, no boxing, no casting on read.

20.3.2 Dictionary — “it’s like a bunch of tuples”

- A `Dictionary<TKey, TValue>` is a hash-table mapping unique keys to values. The lecturer’s plain-English description: “basically it’s like a bunch of tuples — key and value.”

- **Keys must be unique.** Adding a key that already exists throws (with `Add`) or overwrites silently (with the indexer).
- Values *can* be duplicated. “The value may be duplicated, and the reason it can be is because it’s separated out by the key” — the dictionary cares about the keys, not the values.
- Lookup is amortised $O(1)$, far faster than walking an `ArrayList` or array.

```
var ages = new Dictionary<string, int>
{
    { "Alice", 30 },
    { "Bob", 25 },
    { "Cara", 30 } // same value as Alice -- fine
};

int alice = ages["Alice"]; // 30
ages["Alice"] = 31; // updates Alice's value
if (ages.ContainsKey("Dan")) { /* ... */ }
```

20.4 Embedded Review: MCU Architecture & Operation

- CPU = fetch–decode–execute cycle.
- Program Counter holds the address of the next instruction.
- ALU performs arithmetic and logic.
- Pipelining overlaps stages but is disrupted by branches.
- Von Neumann shares one bus; Harvard separates program/data buses for concurrent access.

20.5 Embedded Review: MCU Memory Types

- Flash: non-volatile firmware storage; finite write/erase endurance.
- SRAM: volatile run-time memory for variables, buffers, and stack; embedded C usually avoids the heap.
- EEPROM: non-volatile, byte-addressable for settings and calibration.

20.6 Embedded Review: Digital Communications

Protocol	Type	Key characteristics
UART / RS-232	Asynchronous serial	No shared clock; baud rate, start/stop framing; effective rate < baud.
SPI	Synchronous, full-duplex	SCK/MOSI/MISO/CS; master controls clock; high-speed continuous streaming.
I ² C	Two-wire synchronous bus	SCL/SDA; multi-device via addressing; pull-ups for open-drain.
CAN	Robust network	Multi-master; message-based arbitration; built-in error handling.

Differential signalling (RS-422/485) wins over single-ended for noise rejection and distance.

20.7 Embedded Review: Software Design and Estimation

- Architectures: simple polling loop vs event-driven (interrupts) vs RTOS (deterministic).
- Battery life: $I_{avg} = D \cdot I_{active} + (1 - D) \cdot I_{sleep}$ where D is the duty cycle.
- I_{avg} combined with battery capacity (mAh) gives expected lifetime.

20.8 Use Cases & Applications

Need	Pick	Reason
Ordered, indexable list with fast append	List<T>	Generic, dynamic; O(1) end-append, O(1) random access
Lookup by key	Dictionary<TKey,TValue>	Hash-table; O(1) average lookup
Stable, value-like grouping (point, color)	struct	Stack-allocated, no GC pressure for small data
Read-only sequence with reference semantics	IReadOnlyList<T>	Hides mutation, exposes only iteration/length
FIFO of work items	Queue<T>	Producer/consumer patterns
LIFO of state	Stack<T>	Undo stacks, parser scratch space

20.9 Worked Example: Word Counter

```
string text = File.ReadAllText("speech.txt");
var counts = new Dictionary<string, int>(StringComparer.OrdinalIgnoreCase);
foreach (var w in text.Split(new[] { ' ', '\n', '.', ',' },
    StringSplitOptions.RemoveEmptyEntries))
{
    counts[w] = counts.GetValueOrDefault(w) + 1;
}
foreach (var kv in counts.OrderByDescending(kv => kv.Value).Take(10))
    Console.WriteLine($"{kv.Key}: {kv.Value}");
```

20.10 Struct vs Class

Trait	struct	class
Type	Value	Reference
Default location	Stack (or inline)	Heap
Assignment / param	Copies the value	Copies the reference
Inheritance	No (only interfaces)	Yes
Default size guidance	$\leq \sim 16$ bytes	Larger / mutable / inheritance

20.11 Embedded Cross-Reference

If you only remember a few embedded facts to pair with this lecture's C# content:

- Pipelining and Harvard architecture (Lecture 1–2) explain why MCUs hit single-cycle execution.

- Memory hierarchy (Lecture 3, 13) is what your battery budget is competing for.
- $I_{avg} = D \cdot I_{active} + (1 - D) \cdot I_{sleep}$ is the formula every battery-powered design lives or dies by.
- I²C is open-drain with pull-ups; SPI is push-pull; UART has no shared clock — pick by pin count and rate.

Key Definitions

String (C#)

Specialised array of 16-bit UTF-8 characters.

Structure (struct)

Value type grouping multiple fields, stored on the stack.

Dictionary

Key-value collection backed by a hash table.

Embedded System

Hardware/software combo built for a specific function under constraints.

Microcontroller (MCU)

Single IC integrating CPU, memory, and peripherals.

Program Counter (PC)

Register holding the address of the next instruction.

ALU

Arithmetic Logic Unit; performs arithmetic/logical operations.

Pipelining

Overlapping CPU stages so multiple instructions are in flight.

Harvard Architecture

Program and data on separate buses for concurrent access.

SRAM

Volatile data memory for variables/buffers/stack.

EEPROM

Byte-addressable non-volatile memory for settings/calibration.

UART

Asynchronous serial protocol with baud-rate framing, no shared clock.

I²C

Two-wire synchronous bus with pull-ups; multi-device by address.

Differential Signalling

Two-wire complementary signalling rejecting common-mode noise.

Linear Regulator

Simple low-noise regulator that wastes power as heat.

Generic Collection

Strongly-typed collection (List<T>, Dictionary<K,V>) avoiding boxing/unboxing of value types.

IEnumerable<T>

Interface for read-only forward iteration; the basis for foreach and LINQ.

LINQ

Language-Integrated Query; chainable operators (Where, Select, OrderBy) on collections and other sources.

StringComparer

Helper that controls how strings are compared (case sensitivity, culture).

Sources

- Lecture 20.pdf
- In-class transcript: *.NET and C# concepts* — used for the “strings are completely immutable” framing (including the `s1 = s1 + s2` walk-through), the regular / verbatim / raw literal three-way comparison, the `$` string-interpolation introduction, the struct “different data types in one place” framing, the all-or-nothing struct-constructor rule, the `ArrayList` “more flexible array” description, and the `Dictionary` “bunch of tuples” analogy.

Chapter 21

GUI Design & Event-Driven Programming (Windows Forms)

ENEL712 Embedded Systems Design — Lecture 21

Summary

Windows Forms GUIs in C#: the toolbox of common controls, containers, event-driven programming with handlers, error handling, dialogs, and multi-form encapsulation.

Topics Covered

- Introduction to the GUI
- Visual Studio toolbox & common controls
- Containers and non-visual controls
- Data handling and conversion
- Event-driven programming
- Error handling and dialogs
- Advanced topics (multiple forms, encapsulation)

21.1 Introduction to the GUI

- GUI is the primary user interaction surface.
- Visual Studio provides drag/drop/program controls.
- Custom controls (LEDs, 7-segment displays) can be downloaded or created.
- Example: a weather app combining labels, charts, calendars, text boxes, buttons, combo boxes, and picture boxes.

21.2 Visual Studio Toolbox & Common Controls

Control	Purpose
Label	Display read-only text; main property is <code>Text</code> .
Button	Triggers a Click event; key props: <code>Name</code> , <code>Text</code> , <code>Enabled</code> , <code>Visible</code> , <code>Location</code> , <code>Size</code> .
TextBox	User input or program output; events include <code>TextChanged</code> , <code>Enter</code> , <code>Leave</code> ; supports multi-line.
ComboBox	Drop-down list; items via <code>Collection</code> editor or at run time.
NumericUpDown	Pick integer values within <code>Minimum/Maximum/Value</code> range.
CheckBox & RadioButton	Multi-select vs single-select among related options.

21.3 Containers and Non-Visual Controls

- Containers (`GroupBox`, `Panel`, `TabControl`) group related controls visually and manage shared properties (e.g., visibility).
- Non-visual controls (`Timers`, `MenuStrips`, `FileDialogs`) live in the designer tray below the form.

21.3.1 Common container controls — when to reach for each

- **GroupBox** — gives related controls a visible border and an optional heading. Use for “these controls belong together” (a radio-button cluster, an address block).
- **Panel** — like a `GroupBox` but without the visible border or heading. Useful for layout grouping and for scrolling regions.
- **TabControl** — stacks several pages onto the same area of the form, with selectable tab headers at the top. The lecturer’s tip: programmatically advancing the active tab can model a step-through wizard (fill page 1 → on submit, jump to page 2).

21.3.2 Non-visual controls — “things that happen in the background”

- Drag a `Timer`, `OpenFileDialog`, `SaveFileDialog`, or `SerialPort` onto a form and they end up in the designer tray below the form, not on the form itself. The user never sees them.
- `Timer` is the most common: set `Interval` in milliseconds, handle the `Tick` event, set `Enabled = true` to start.
- The lecturer’s warning: a `Timer`’s `Tick` fires on the UI thread on a fixed schedule. If the previous `Tick` has not finished executing when the next one is due, the next one is delayed but *queued*, and you can easily end up with overlapping work — hence the timer-vs-comm crash discussed in Lecture 18.

21.4 Data Handling and Conversion

- `TextBoxes` always handle text as strings.
- `String` → `number`: `Int32.Parse(string)`, `double.Parse(string)`, or `Convert.ToInt32(string)`.
- `Number` → `string`: `.ToString()`.

21.5 Event-Driven Programming

- Windows is event-driven — the program responds to clicks, key presses, etc.
- Event handlers are functions that respond to messages in the queue (auto-created by double-clicking a control).
- Critical: rename controls (e.g., `textBox1` → `openTBox`) before generating handlers; renaming after breaks the link.

21.5.1 “Windows doesn’t know when things are going to happen”

- In a classic procedural program you write the sequence of steps and the program runs them top to bottom. In a GUI program you do not control *when* the user clicks, types, or moves the mouse — the operating system delivers an event whenever it occurs.
- Your job is to write **handler methods** for the events you care about; the OS message loop calls them. “Mouse clicks, button presses, data being received — it just happens when it happens.”
- Multiple events can race. A timer-driven background task and a user click can fire seconds apart or microseconds apart; you cannot make assumptions about ordering.

21.5.2 Name the controls *before* you generate handlers

The single most painful mistake the lecturer warned about:

- Drag a button onto the form. *Before* doing anything else, rename it from `button1` to something meaningful (`plusBtn`, `openBtn`, `exitBtn`).
- *Then* double-click it to generate the handler. Visual Studio names the handler after the control: `plusBtn_Click`.
- If you forget to rename first and rename later, the auto-generated handler keeps the old name and stops being wired up. “It’s a pain. It actually is a pain. You basically need to hack the system... it is actually better to delete it and start again.”
- Same rule for TextBoxes (`aTBox` not `textBox1`), ComboBoxes, etc. “Otherwise you’ve got `TextBox1`, `TextBox2` — which one’s your name, which one’s your phone number? It gets very confusing very quickly.”

21.6 Error Handling and Dialogs

- Wrap risky operations (parsing) in try/catch to avoid crashes.
- Calculator example: parse with try and handle failure with `catch(Exception ex)`.
- `MessageBox.Show(text, title, buttons)` shows information or errors — itself an OOP class.

21.6.1 Calculator evolution — the lecture’s three-step refactor

The lecturer worked the calculator example up step by step. The progression is worth keeping because it illustrates how an event handler turns into clean, reusable code.

Step 1 — naïve parse, easy to crash

```
private void plusBtn_Click(object sender, EventArgs e)
{
    double a = double.Parse(aTBox.Text);
    double b = double.Parse(bTBox.Text);
    resultTBox.Text = (a + b).ToString();
}
```

Type “hello” into either box and `double.Parse` throws. The whole form crashes.

Step 2 — wrap in try / catch so the form survives

```
private void plusBtn_Click(object sender, EventArgs e)
{
    try
    {
        double a = double.Parse(aTBox.Text);
        double b = double.Parse(bTBox.Text);
        resultTBox.Text = (a + b).ToString();
    }
    catch (Exception ex)
    {
        MessageBox.Show("Could not parse input: " + ex.Message,
                        "Parse Error",
                        MessageBoxButtons.OK,
                        MessageBoxIcon.Error);
    }
}
```

Better, but the lecturer pointed out the obvious next problem: “we could copy and paste that into each function, but that’s not good coding.”

Step 3 — extract a helper method; Click handlers become one-liners

```
private string Calculate(string aText, string bText, string op)
{
    try
    {
        double a = double.Parse(aText);
        double b = double.Parse(bText);
        return op switch
        {
            "+" => (a + b).ToString(),
            "-" => (a - b).ToString(),
            "*" => (a * b).ToString(),
            "/" => (a / b).ToString(),
            _ => "Unknown operator"
        };
    }
    catch (Exception ex)
    {
        return "Error: " + ex.Message;
    }
}

private void plusBtn_Click (object s, EventArgs e) =>
```

```

        resultTBox.Text = Calculate(aTBox.Text, bTBox.Text, "+");

private void minusBtn_Click(object s, EventArgs e) =>
    resultTBox.Text = Calculate(aTBox.Text, bTBox.Text, "-");

private void mulBtn_Click (object s, EventArgs e) =>
    resultTBox.Text = Calculate(aTBox.Text, bTBox.Text, "*");

private void divBtn_Click (object s, EventArgs e) =>
    resultTBox.Text = Calculate(aTBox.Text, bTBox.Text, "/");

```

- All the parsing, error handling, and arithmetic live in one place. Each `Click` handler is a single line.
- Adding a fifth operator (“^” for power, say) means one new `switch` arm and one new button. The handlers do not change shape.
- This is the everyday application of the OOP pillars: pull repeated logic into a named function, keep handlers thin, and centralise error handling.

21.7 Advanced: Multiple Forms & Encapsulation

- Apps can hold multiple forms — created via Solution Explorer or programmatically (manual control layout required).
- Form controls are typically declared private; expose access through public methods (e.g., `write_aTBox(string newText)`) to keep encapsulation.

21.8 Use Cases & Applications

- Companion PC apps for embedded products: configuration tools, firmware updaters, live telemetry viewers.
- Lab utilities: a small calculator, sweep generator, or scope-style logger built from a Form, a Chart, and a `SerialPort`.
- Quick prototypes: WinForms is still the fastest way to get a clickable interface in front of a customer.
- MAUI / WPF / Avalonia replace WinForms for new cross-platform GUIs but the event-driven mindset transfers directly.

21.9 Worked Example: Two-Number Calculator

```

private void plusB_Click(object sender, EventArgs e)
{
    try
    {
        double a = double.Parse(aTBox.Text);
        double b = double.Parse(bTBox.Text);
        resultLbl.Text = (a + b).ToString("F3");
    }
    catch (Exception ex)
    {
        MessageBox.Show("Error parsing input: " + ex.Message,

```

```

        "Parsing Error",
        MessageBoxButtons.OK,
        MessageBoxIcon.Error);
    }
}

```

Notice the names aTBox/bTBox/resultLbl — set those before generating the Click handler. Renaming controls afterwards leaves the handler signature wired to the old name and Visual Studio cannot match them up.

21.10 Encapsulating Cross-Form Access

```

// In MainForm -- keep the control private and expose a deliberate API.
private System.Windows.Forms.TextBox statusTBox;

public void WriteStatus(string text)
{
    if (statusTBox.InvokeRequired)
        statusTBox.Invoke((Action)(() => statusTBox.Text = text));
    else
        statusTBox.Text = text;
}

```

Wrapping with `InvokeRequired` marshals back to the UI thread when calling from a worker thread (e.g., a `SerialPort.DataReceived` handler) — the most common WinForms threading bug.

21.11 Common Pitfalls

- Renaming a control after generating its handler — Visual Studio loses the link and the event silently stops firing.
- Calling `MessageBox` / UI updates from a non-UI thread — throws or hangs the form.
- Mutating the same `TextBox.Text` repeatedly during a tight loop redraws on every change; batch updates with `SuspendLayout/ResumeLayout` if perf matters.
- Storing all logic in event handlers — extract real work into separate methods or classes so it's testable.

Key Definitions

GUI

Graphical User Interface — visual layer for interacting with a program.

Event-Driven Programming

Computer responds to user/system events such as clicks and key presses.

Event Handler

Function executed in response to a specific event (e.g., button Click).

Encapsulation

Hiding form internals (private) and exposing controlled access through public methods.

Control Naming

Naming controls before creating handlers to keep code/handler links stable.

Try/Catch

Error-handling structure wrapping risky code to prevent crashes.

Container

Control grouping related GUI elements (GroupBox, TabControl).

Dialog

Window class such as MessageBox used to display info or prompt the user.

Event-Driven Programming

Programs whose flow is driven by external events (user input, messages) routed to handler functions.

UI Thread

Single thread that owns the message loop; almost all WinForms control accesses must happen on this thread.

Message Loop

Loop that pulls Windows messages from the queue and dispatches them to control handlers.

Modal Dialog

Dialog that blocks input to the rest of the application until it is dismissed (e.g., MessageBox.Show).

Sources

- Lecture 21.pdf
- In-class transcript: *.NET and C# concepts* — used for the container-controls (GroupBox / Panel / TabControl) descriptions, the non-visual-controls / designer-tray framing, the “Windows doesn’t know when things are going to happen” event-driven explanation, the rename-controls-before-generating-handlers warning, and the three-step calculator refactor (naïve → try/catch → helper method with `switch`).

Chapter 22

File Handling & Serial Ports

ENEL712 Embedded Systems Design — Lecture 22

Summary

C# file handling via System.IO streams (text and binary), Windows file dialogs, directory/path operations, exception handling, and SerialPort communication including endianness considerations.

Topics Covered

- File types and streams
- C# classes for file handling
- Windows dialog boxes
- Directory and path operations
- Exception handling in file operations
- Serial port communication
- Endianness in multi-byte data
- Image controls and basic 2D graphics
- Debugging in Visual Studio (breakpoints, locals, call stack)

22.1 File Types and Streams

- C# treats files as sequential streams of bytes with a current position, length, and access mode.
- Text files: ASCII or Unicode (Notepad-style readable).
- Binary files: 0s and 1s, e.g., .bmp, .wav, executables — interpretable only with the data format.
- Files end with EOF or after a specified byte count.

22.1.1 “You can’t just create a file — you need to create a stream first”

The transcript’s recurring theme:

- In C#, every file operation goes through a **Stream**. “You write to the file *through* the stream.”
- Text reading and writing: **StreamReader** / **StreamWriter**. They assume ASCII or another text encoding and handle the byte-to-character translation for you.
- Binary reading and writing: **FileStream** + **BinaryReader** / **BinaryWriter**. They make no assumptions about the bytes and require you to know the format you are reading or writing.
- The **Stream** abstraction is also what **using** cleans up at scope exit, so files do not stay locked when an exception is thrown.

22.1.2 Text: Write vs WriteLine

The lecturer asked the class to predict the output of this code:

```
using StreamWriter w = new StreamWriter("test_out.txt");
w.Write("Hello ");
w.WriteLine("World");
w.Write("Next");
```

The file ends up containing:

```
Hello World
Next
```

- **Write** writes the string as-is — no terminator. Two **Write** calls run together on the same line.
- **WriteLine** writes the string **and** appends the platform end-of-line character (`\r\n` on Windows, `\n` on Linux/macOS).
- Same rule for **Console.Write** vs **Console.WriteLine**. The naming convention runs through the whole standard library.

22.1.3 Reading binary — and why audio is sampled at 44.1 kHz

The lecturer used CD-quality audio as a concrete example of why binary file formats specify their internal structure.

- The human ear hears roughly 20 Hz to 20 kHz.
- Nyquist’s sampling theorem: to faithfully reconstruct a waveform, sample at *at least twice* the highest frequency — so ≥ 40 kHz.
- Add a little overhead for the anti-alias filter roll-off and you arrive at the famous 44.1 kHz CD-quality sample rate.
- “Binary contents are meaningful to programs designed for that specific data format.” Without knowing that a WAV file is 16-bit stereo PCM at 44.1 kHz, the bytes are noise.
- The same logic applies to your lab’s serial frame: you have a documented byte layout, so both the PC and the microcontroller can decode it the same way.

22.2 C# Classes for File Handling

Class	Purpose	Key details
StreamReader	Read general text	Character input in specific encodings (e.g., ASCII).
StreamWriter	Write general text	Auto-handles encoding; not for binary data.
FileStream	Byte-level read/write	Requires a FileMode (Append, Create, Open) to open or create.
BinaryReader	Read binary data	Reads 8–64 bit primitives; needs a known data format.
BinaryWriter	Write binary data	Used with a Stream for structured formats (JPEG, WAV).

22.3 Windows Dialog Boxes

- OpenFileDialog: lets users select a file; non-visual element placed in the designer tray.
- SaveFileDialog: provides a save path and warns about existing files; doesn't create the file itself.
- Filter property restricts selectable types (e.g., 'MP3 Files | *.mp3').
- Best practice: check ShowDialog() == DialogResult.OK before proceeding.

22.4 Directory and Path Operations

- File methods: Copy, Create, Delete, Exists, Move, Open, Replace.
- Directory methods: CreateDirectory, Delete, Exists, Move.
- Path class: GetExtension, GetFileName, ChangeExtension.

22.5 Exception Handling in File Operations

Files can be read-only, missing, or in use — wrap operations in try/catch to handle exceptions and prevent crashes.

22.6 Serial Port Communication

- SerialPort object simplifies COM port communication; create graphically or programmatically (preferred).
- Methods are blocking by default; use timeouts or the DataReceived event for responsiveness.
- Key methods: Open(), GetPortNames() (static), ReadExisting() (drains buffer to string), Write() (bytes/strings).
- Considerations: buffering (multiple Reads may be needed), keep buffer cleared by reading even unwanted bytes.

22.6.1 Programmatic vs graphical serial port setup

- You *can* drag a `SerialPort` onto the form and configure its properties in the designer; the lecturer noted that “I can’t think of a time I have ever done it graphically — I’ve always done it programmatically.”
- Reason: serial port behaviour (port name, baud rate, handshake, parity) is often set at run time based on user choice or device discovery. Hard-coding it in the designer bakes it into the build.
- Common pattern: call `SerialPort.GetPortNames()`, populate a `ComboBox` so the user picks the port, then open the port in code with that name.

22.6.2 Blocking vs the `DataReceived` event

- By default, `Read` and `ReadByte` are **blocking** — they sit there waiting for data and freeze whatever thread called them.
- Hanging the UI thread is unacceptable for a GUI, so either set a `ReadTimeout` or subscribe to the `DataReceived` event, which fires on a worker thread when bytes arrive.
- Threading subtlety: the `DataReceived` handler runs on a background thread. Any UI update inside it must be marshalled to the UI thread with `Invoke`. The Lecture 21 example shows this pattern.

22.6.3 `ReadExisting` — the buffer-flushing read

- “If you’ve got stuff in the data buffer and you haven’t processed it, when you do a `Read`, that old data will be mixed in with the data you want — which is an unwanted thing.”
- `ReadExisting` flushes everything currently in the receive buffer and returns it as a string. It does not wait for more bytes.
- Use it inside `DataReceived` to drain whatever has arrived since the last tick, or as a one-shot pre-clear before sending a new command.

22.7 Endianness in Multi-Byte Data

- Endianness is the order of bytes for multi-byte values.
- Little-Endian: LSB first — Intel/Windows PCs.
- Big-Endian: MSB first — network protocols and AVR.
- When $PC \leftrightarrow AVR$, the programmer must reverse multi-byte ordering before transmission.

22.8 Use Cases & Applications

- Logging serial data from an embedded device into CSV/binary files for offline analysis.
- Implementing simple firmware updaters that read a `.hex/.bin` file and stream it over UART.
- Building flight-data recorders / lab instruments where event-driven serial reception drives a live plot and disk log.
- Writing application configuration to disk on close and reading it back on launch.

22.9 Worked Example: Open + Read a Text File

```
var dlg = new OpenFileDialog
{
    Filter = "Text files|*.txt|All files|*.*"
};
if (dlg.ShowDialog() == DialogResult.OK)
{
    using StreamReader r = new StreamReader(dlg.FileName);
    string contents = r.ReadToEnd();
    textBox.Text = contents;
}
```

The using statement guarantees the stream closes even if an exception is thrown inside the block.

22.10 Worked Example: Event-Driven Serial Reception

```
SerialPort sp = new SerialPort("COM3", 115200);
sp.DataReceived += (s, e) =>
{
    string chunk = sp.ReadExisting();
    Invoke((Action)(() => logTBox.AppendText(chunk)));
};
sp.Open();
```

Reading inside DataReceived avoids a busy loop. Marshalling back to the UI thread with Invoke is required because DataReceived runs on a worker thread.

22.11 Endianness Conversion

If a Big-Endian AVR sends a 32-bit value as bytes [b3, b2, b1, b0], a Little-Endian PC must reverse them before BitConverter.ToInt32:

```
byte[] frame = new byte[4]; // received from serial in Big-Endian order
if (BitConverter.IsLittleEndian) Array.Reverse(frame);
int value = BitConverter.ToInt32(frame, 0);
```

22.12 FileMode Decision Table

FileMode	Behaviour
Open	Open existing file; throw if missing
OpenOrCreate	Open existing file; create if missing
Create	Always create a new file (truncates existing)
CreateNew	Create a new file; throw if it already exists
Append	Open or create; seek to end so writes append
Truncate	Open existing file and discard its contents

22.13 Image Controls — the PictureBox

- Drag a PictureBox onto the form, set its Image property (designer) or pictureBox.Image = Image.FromFile(path) (code), and the picture displays.

- The interesting property is **SizeMode**, which decides what happens when the image and the control are different sizes.

SizeMode	Effect
Normal	No scaling; image drawn from top-left; clipped if larger than the control.
StretchImage	Image stretched to fill the control; will distort if aspect ratios differ.
AutoSize	Control resizes to match the image's natural size.
CenterImage	Image centred in the control; cropped if larger, white space if smaller.
Zoom	Scales while preserving aspect ratio; letterboxed if ratios differ.

22.13.1 Picking a file format — GIF / JPEG / BMP / PNG

- **GIF** — the only one of the four that natively supports *animation*. “That’s typically the time you’d use a GIF.”
- **JPEG** — compressed photo format. Older, lossy, no transparency, but tiny file sizes for photographs.
- **BMP** — Windows-specific raw bitmap. “Literally dots” — every pixel is stored explicitly. Scales badly because it has no vector information; useful as a debug intermediate, not a final format.
- **PNG** — lossless, supports transparency, and scales better than BMP. The lecturer’s everyday default: “I basically use them all the time. Back in the day it was JPEG, and all Windows stuff was bitmaps, but again the problem with bitmaps is they don’t scale.”

22.14 2D Graphics — the System.Drawing namespace

- Three primary objects: the surface to draw on (a **Form** or **Panel**), a **Graphics** object (the “canvas”), and a **Pen** or **Brush** (the drawing tool).
- `this.CreateGraphics()` returns a **Graphics** object for the current form; alternatively, draw inside the form’s **Paint** event handler using **e.Graphics**.

22.14.1 The coordinate system: top-left is (0,0); +Y is down

This is the most common “why is my line not where I expected” trap:

- In Altium / PCB tools (and in maths), (0,0) is bottom-left and positive Y goes *up*. In WinForms graphics, (0,0) is **top-left** and positive Y goes *down*.
- `g.DrawLine(pen, 0, 0, 100, 100)` draws from the top-left corner to a point 100 pixels right and 100 pixels down — “positive in a negative direction” if you are used to mathematical axes.

22.14.2 Pens and brushes — outlines vs fills

- A **Pen** draws an outline (lines, rectangle borders). Configurable for colour, width, dash style, transparency.
- A **Brush** fills a region. Subtypes include **SolidBrush**, **LinearGradientBrush**, **HatchBrush**, and **TextureBrush**.
- Method naming follows the same Outline-vs-Fill pattern: **DrawRectangle** (outline only), **FillRectangle** (filled).

22.14.3 Paint events — persistent vs one-shot drawing

- Drawing inside a button click handler with `CreateGraphics` draws *once* — the moment the window is resized, minimised, or hidden, the drawing is lost.
- Drawing inside the form's `Paint` event handler is **persistent**: Windows calls `Paint` every time the window needs to be repainted, and your code redraws.
- Rule of thumb: anything you want to stay visible belongs in the `Paint` event; anything truly transient (a quick debug line) can go in a button click.

22.15 Debugging in Visual Studio

The lecturer spent the second half of this lecture on debugging because, in his own words, “I have yet to write a program without bugs.” Three categories of bug:

- **Syntactic errors** — the compiler refuses to build the program. Easiest to fix because the IDE points right at the line.
- **Invalid data** — the program builds but crashes at run time on bad input. Easy to defend against with `try / catch` and `TryParse`.
- **Logical errors** — the program runs cleanly but produces the *wrong answer*. The hardest category because the compiler and runtime cannot help: “the program doesn’t know that the output it’s providing isn’t what you want.”

22.15.1 Three half-serious debugging attitudes (lecturer’s slide)

- “Any sufficiently advanced bug is indistinguishable from a feature, therefore it’s good.”
- “Sometimes it pays to stay in bed on Monday rather than spend the rest of the week debugging Monday’s code.”
- “If it compiles, ship it.”

None of these handle the logic-error case, which is why the rest of this section exists.

22.15.2 Breakpoints

- Click the gutter (the blank space next to the line numbers) to set a breakpoint. Execution pauses just before that line runs.
- A breakpoint on a line that the program never reaches will appear hollow and never hit — a useful early signal that the method is an “orphan” that nothing calls.
- Run with `F5` or the green play button. They do the same thing; the lecturer uses the button.

22.15.3 Inspect: variables, locals, call stack

- Hover over any variable while paused to see its current value. Right-click → *Add Watch* keeps it visible across steps.
- The *Locals* window lists every local in the current frame, automatically. Useful for spotting one variable that has the wrong value when everything else looks fine.
- The *Call Stack* window shows the chain of method calls that led to the current line. “Particularly useful for checking that we’re calling the method from where we want to be called from” — essential when an event handler fires unexpectedly.

- You can *overwrite* a variable from the Locals window while paused. Force a value to test a specific branch without re-running.

22.15.4 Step in / step over / step out

Control	Shortcut	Behaviour
Step Into	F11	Enter a method called on the current line.
Step Over	F10	Execute the method without entering it; pause at the next line.
Step Out	Shift+F11	Run to the end of the current method and pause at the return site.
Continue	F5	Resume until the next breakpoint.
Restart	Ctrl+Shift+F5	Stop, rebuild, run from the beginning.

22.15.5 Conditional breakpoints — “best friend” for long loops

- Right-click a breakpoint, *Conditions...*, and add an expression. The breakpoint only fires when the expression is true.
- The lecturer’s killer example: “we’re doing a thousand iterations and we’re not seeing an error until iteration 800. I don’t care about the previous 800 iterations.” Set the condition to `i == 800` and step through from there.
- Other useful conditions: `user == null`, `retries >= 3`, `buffer.Count > MaxFrame`.

22.15.6 Logging to the Output window

- `Debug.WriteLine(...)` writes to the Output window (View → Output) when running under the debugger. Removed automatically in Release builds, so it is safe to leave `Debug.WriteLine` calls in checked-in code.
- For long-running diagnosis, redirect to a file: write a timestamped log line for each interesting event and inspect it after the run finishes. The lecturer’s advice: “the outputs in the Output window can sweep through too fast to read — write a log file.”
- “Don’t be afraid to refer to documentation. Pilots have checklists even though every flight is the same.” Looking things up is faster than guessing.

Key Definitions

Stream

Abstraction treating I/O as a sequential byte flow with position/length/mode.

System.IO

C# namespace containing the file/directory classes.

Binary File

File of raw bytes interpretable only with a known data format.

EOF

End-of-File marker or specified byte count terminating a stream.

FileMode

Enum for FileStream open behaviour (Open / Create / Append).

Blocking Behavior

Default serial port behaviour halting execution until I/O completes.

Endianness

Byte ordering of multi-byte data — Little-Endian or Big-Endian.

Little-Endian

Least significant byte stored first (Intel).

Big-Endian

Most significant byte stored first (AVR / network protocols).

using Statement

Block that automatically calls Dispose() on the variable at scope exit, releasing files/sockets/etc.

DataReceived Event

SerialPort event fired on a worker thread when bytes arrive; preferred over polling.

IsLittleEndian

BitConverter property indicating the host's byte order; critical for safe wire-format conversions.

Asynchronous I/O

Pattern (async/await in C#) where I/O calls return a Task and don't block the calling thread.

Sources

- Lecture 22.pdf
- In-class transcript: *File Handling and Serial Ports and More* — used for the “you can't just create a file, you need a stream” framing, the Write vs WriteLine prediction, the CD-quality / Nyquist derivation of 44.1 kHz, the programmatic-vs-graphical serial setup recommendation, the ReadExisting buffer-flush explanation, the PictureBox SizeMode table, the GIF / JPEG / BMP / PNG comparison, the top-left graphics origin trap, the Paint-event persistence rule, the three bug categories (syntactic / invalid data / logical), the “half-serious debugging attitudes” slide, breakpoints in the gutter, the Locals and Call Stack windows, step in/over/out, conditional breakpoints for long loops, and the “write a log file” advice.

Chapter 23

Week 7 — Memory Layout & OOP Fundamentals

ENEL712 Embedded Systems Design — Lecture Week 7

Summary

Wider summary of week 7 material: program memory segmentation (stack vs heap), OOP variable vernacular (value/reference, mutability), OOP philosophy, and class components in C#. This version of the chapter has been expanded with the lecturer's in-class explanations so that the narrative reasoning — *why* OOP is organised around memory in the first place — is preserved alongside the slide-level bullet points.

Topics Covered

- Program memory management
- Stack vs heap comparison
- OOP variable vernacular (value/reference, mutability)
- OOP philosophy and accessibility modifiers
- Logical vs physical organisation (namespace vs assembly)
- Class components in C#

23.1 Why Memory Is the Heart of OOP

The single line from the in-class lecture that ties this whole week together is: *“Everything in object-oriented programming is around memory.”*

- Classes, properties, constructors, get/set, access modifiers — every one of these features exists because the language is trying to manage where data lives (stack vs heap), how long it lives for, and who is allowed to touch it.
- The OOP *vocabulary* (object, class, encapsulation, namespace) looks intimidating until you map each keyword back to one of those three questions: where does it live, how long does it live, who can reach it.

- Structured/procedural programming organises code by control flow. OOP organises code by data ownership and lifetime. Both compile to the same machine instructions; the difference is at design time.
- This is also why the lecturer warns that the first exposure to OOP can feel like “a ball of clay thrown at the wall” — the chaos disappears once you anchor every concept to memory.

23.2 Program Memory Management

23.2.1 Memory segmentation

- High addresses: environment/arguments and the stack.
- Middle: shared libraries / DLLs.
- Low addresses: heap, .bss (uninitialised data), .data (initialised globals/statics), .text (code).

23.2.2 The stack

- LIFO region for function calls, locals, and return addresses.
- Grows downward from high to low memory.
- Very fast — simple pointer movement, cache-friendly.
- Direct access via CPU registers and the stack pointer.

23.2.3 The heap

- Used for dynamic allocation (objects, dynamic arrays).
- Grows upward toward high memory addresses.
- Manual management (new, malloc) or automatic via a Garbage Collector.
- Slower access via indirection; possible cache misses.

23.3 Stack vs Heap Comparison

Feature	Stack	Heap
Allocation	Automatic (compiler/runtime)	Manual (malloc / new) — or GC
Lifetime	Scope of the function	Until freed (or GC)
Size	Small & fixed (a few MB / thread)	Very large (limited by RAM)
Growth direction	Downward	Upward
Access	Direct via registers and SP	Indirect via pointers/references
Fragmentation	None	Possible (internal/external)
Thread safety	Per-thread	Shared; needs synchronisation
Risks	Stack overflow	Leaks, dangling pointers, double free, fragmentation

23.4 OOP Variable Vernacular

23.4.1 Value vs reference types

- Value types (int, bool, struct, enum) hold the actual data — usually on the stack; assignment copies the value.

- Reference types (class, string, array, List<T>) hold a pointer to data on the heap; assignment copies the reference, not the object.

In-class demonstration: “what have I just copied?”

The lecturer walked the class through two assignments to expose the difference.

```
// Value type --- I get an independent 100
int c = 100;
int d = c;           // d holds its own copy of 100
d = 200;             // c is still 100

// Reference type --- I copied the pointer, not the data
int[] a = { 1, 2, 3 };
int[] e = a;         // e and a point at the SAME heap array
e[0] = 99;           // a[0] is now 99 as well

// Decoupling: ask for a real copy
int[] f = new int[a.Length];
Array.Copy(a, f, a.Length); // f is independent of a
```

- “If I copy my entry which I’ll call **e**, what have I copied? The pointer.”
- Anything that shares a memory location is going to see the same edits — this is the source of an enormous amount of bugs and is exactly why `Array.Copy` (and friends) exist.
- The same logic applies to `string` and to every class instance. Strings are immutable, so the bug is less visible, but the assignment still copies a reference, not the characters.
- `Array.Sort(a)` sorts in place — *every* reference to that array now sees the sorted order. If you need the original order back, make a copy first.

23.4.2 Mutability

- Mutable: object can be modified after creation (e.g., List<T>); not inherently thread-safe.
- Immutable: object is fixed; modifications create new instances (e.g., string, record); naturally thread-safe.
- Mutable types “can have side effects” when there are multiple references to them — edit through one, the others see the change. Immutable types eliminate the surprise.
- Lecturer’s two-word summary: List<T> is the classic mutable example; ValueTuple (and string) are the classic immutable examples.

23.4.3 Type declaration

- Explicit: type stated, e.g., string name = "Alice".
- Implicit (var): compiler infers type, e.g., var age = 30 → int.

Choosing the right numeric type — “a double for the number 20?”

The lecturer asked the class whether using a `double` for the value 20 was worth it. The point of the question:

- A `double` is 64 bits. An `int` is 32 bits. If 20 is always an integer and never needs decimals, the double wastes half of its storage.

- This matters most in tight loops, large arrays, and on memory- constrained microcontrollers, where “a few unnecessary bits per element” multiplies fast.
- Rule of thumb: **int** for counters and indices, **double** for measurements and scientific calculations, **decimal** for money. Only widen when the algorithm actually needs the precision.
- **var** is a convenience, not a type — it asks the compiler to infer the type from the initialiser. It is most useful with long generic names and with anonymous types (where there is no spell-out-able type to write).

23.5 OOP Philosophy and Accessibility Modifiers

- OOP organises programs around data and objects — modular, reusable, encapsulated, hierarchical.
- Class = template; Object = specific instance created with new.
- Encapsulation hides internal data and methods, controlling visibility.

23.5.1 Accessibility modifiers

Modifier	Scope
public	Anywhere
private	Same class only
protected	Same class and derived classes
internal	Same assembly
protected internal	Same assembly OR derived classes
private protected	Same class + derived classes in the same assembly

23.6 Logical vs Physical Organisation

Aspect	Namespace	Assembly
Purpose	Logical organisation; prevents naming collisions	Physical packaging and deployment unit
Concept	Compile-time (logical)	Runtime (physical)
What it is	Naming container for types	Compiled file (.exe / .dll) with IL and metadata

23.7 Class Components in C#

- Properties / fields: data members (e.g., `account.Balance`); best practice is to keep them private.
- Accessors (get/set): controlled access; set uses the implicit value parameter.
- Methods: functions; only public methods are visible outside.
- Constructors: same name as class, no return type; allocate and initialise an object on new.
- Overloading: multiple methods/constructors with same name and different parameter signatures.

23.8 Use Cases & Applications

- Stack-vs-heap intuition guides daily C# decisions: prefer struct + value semantics for tiny, immutable data, class for anything large or mutable.
- Mutability rules are the difference between safe-by-default code (immutable strings, records) and code that needs locks to be thread-safe.
- Access-modifier choices document intent — protected on a base member tells future readers 'this is meant to be reused only via inheritance'.
- Namespace vs assembly thinking matters for deployment: a NuGet package is one assembly; the namespaces inside organise its API.

23.9 Worked Example: Stack Overflow Demonstration

```
// In C# (or C), unbounded recursion is the fastest way to break the stack.
void Recurse(int depth)
{
    int[] big = new int[1024];    // each call grabs ~4 KB of stack space
    Recurse(depth + 1);
}
Recurse(0);                      // crashes with StackOverflowException
```

On embedded systems the situation is worse: no exception, just silent corruption — a strong reason to budget stack and avoid recursion.

23.10 Mutability Cheat Sheet

Type	Mutable?	Where stored
string	No	Heap
int / double / bool	Yes (value)	Stack / inline
DateTime	No	Stack (struct)
List<T>	Yes	Heap
Tuple (record struct)	No	Stack

23.11 Practical Notes

- Treat fields as private by default. Promote to internal/protected only when you have a use case.
- Use readonly fields and { get; } properties to make immutability explicit.
- Don't force everything into a single namespace; group classes the way callers think about them.
- An assembly is the unit of versioning and shipping; a namespace is just a naming convenience inside it.

23.12 Lecture Notes: Memory Layout in Practice

Process memory has four logical regions. Globals split between BSS (uninitialised) and DATA (initialised). The stack grows downward from a high address; the heap grows upward from a low address. The free space between them is what either side can grow into.

Region	Direction	Lifetime	Typical contents
BSS (uninitialised data)	Fixed at link time	Whole program	Globals declared but not assigned a value
DATA (initialised data)	Fixed at link time	Whole program	Globals/static with non-zero initialiser
Stack	Grows down from high memory	Per function call	Locals, parameters, return addresses
Heap	Grows up from low memory	Until freed / GC'd	new objects, dynamically sized buffers

- When the stack and heap meet, the system fails — classic stack overflow.
- Globals also have a fixed budget; using too many of them eats space that could otherwise be free for stack/heap growth.
- Treat 'where does this live?' as a first-class design question, not an implementation detail.

23.13 Stack Mechanics — Push / Pop / Stack Pointer

- Calling a function pushes a frame onto the stack: parameters, locals, and the return address.
- Returning pops the frame automatically — you almost never pop manually in high-level code.
- The Stack Pointer (SP) always sits at the current top; pushing a new frame moves SP towards lower memory.
- Nested calls grow the stack: $\text{main} \rightarrow \text{A} \rightarrow \text{B}$ leaves three frames live at once.
- Because pop is automatic, stack memory management is essentially free — that is its single biggest advantage.

23.13.1 In-class walkthrough: how the stack actually grows

The lecturer walked the class through the same scenario step by step, emphasising that *nesting* is the only thing that makes the stack grow.

- **Initial state:** only `main` sits on the stack — nothing has been called yet.
- **From within `main`, function `A` is called.** `A`'s frame is pushed on top. Everything declared inside `A` — locals, parameters — lives in that frame because it is part of `A`.
- Each frame also carries a **return address** so the CPU knows which line of the caller to come back to.
- The **stack pointer (SP)** always tracks the current top. As you add a frame, SP moves up the conceptual stack but towards *lower* memory addresses.
- Once the function finishes, its frame is popped automatically. You do not need to think about pop in high-level code at all.
- “Anything that goes on the stack exists only while it is on the stack. As soon as the function is finished, it is gone.”

- This is exactly why **return** matters: it is your one chance to copy a result out before the frame disappears.

23.13.2 Why the area between stack and heap is shared

The stack starts at high memory and grows down; the heap starts at low memory and grows up; in between sits the *free space* that both can expand into.

- If the stack pushes too many frames, it can collide with the heap — a stack overflow.
- If the heap is over-allocated, it can collide with the stack — an out-of-memory failure.
- Globals (BSS and DATA) have a fixed budget allocated at link time: using too many of them eats space that stack and heap could otherwise expand into.
- This is the underlying reason embedded engineers think carefully about *where* variables live and *how many* of them they declare — not just whether they are correctly typed.

23.13.3 Worked example: stack frames during nested calls

Step	What happens	Stack frames (top → bottom)
1	Program starts	main
2	main calls A	A, main
3	A calls B	B, A, main
4	B returns to A	A, main
5	A returns to main	main

23.14 Recursion — Why Embedded Engineers Avoid It

A recursive function calls itself. Every call adds another frame to the stack; the deeper the recursion, the larger the stack grows. With no termination, the stack collides with the heap and the program crashes — a stack overflow.

- Tree/graph algorithms feel natural recursive — but on an MCU you almost never have the stack budget.
- Prefer explicit iteration with a heap-allocated worklist; it is bounded and inspectable.
- If you must recurse, prove an upper bound on depth and add a safety counter.

23.14.1 In-class framing: “recursion is the bee’s knees — until it isn’t”

The lecturer’s tone in class is worth keeping in mind because it matches the engineering reality of this course:

- Many *software* engineers love recursion: tree walks, parsers, and divide-and-conquer algorithms are often elegantly expressed as “a function that calls itself”.
- **Embedded engineers think differently:** “we actually care about memory, and so we try to avoid recursion because it can lead to the dangerous condition where we have run-away and our stack overflows and blows itself apart.”
- The danger is not the recursive idea itself — it is that each self-call pushes another frame, and the deeper the recursion goes the closer the stack gets to crashing into the heap.

- On a desktop with megabytes of stack, you might never notice. On a microcontroller with a few kilobytes of stack, an extra few hundred recursive calls is enough to corrupt the heap silently — no exception, just wrong behaviour.
- Conclusion the lecturer asked the class to remember: in this course, **recursion is something we are aware of but do not reach for**.

```
// Bad: unbounded recursion -- stack grows on every call
int Sum(int n) {
    if (n == 0) return 0;
    return n + Sum(n - 1);    // 10,000 deep ⇒ stack blows up
}

// Good: iterative -- single frame, constant stack usage
int Sum(int n) {
    int total = 0;
    for (int i = 1; i <= n; i++) total += i;
    return total;
}
```

23.15 Heap Behaviour — Fragmentation, GC, malloc

- The heap is unordered — allocations stay where placed until explicitly freed (C/C++) or collected (C#/Java).
- Repeated allocate/free cycles cause fragmentation: lots of small free holes but no contiguous block big enough for the next request.
- C and C++ expose malloc/free directly; C# hides this behind the .NET runtime and its garbage collector.
- The garbage collector finds heap objects no longer referenced by any live variable and reclaims them; modern GCs also compact the heap to defragment.
- Class instances in C# always live on the heap; their reference (the variable) lives on the stack.
- When the reference goes out of scope, the heap object becomes unreferenced and is eligible for collection.

23.15.1 In-class framing: “the heap is a free-for-all”

The lecturer drew a deliberate contrast between the stack and the heap:

- The stack is rigidly ordered — last in, first out, with the compiler/runtime doing all the bookkeeping.
- The heap is a **free-for-all**: wherever an object was placed is where it sits, and it stays there until it is *actively destroyed* or collected.
- Because of this freedom, you get **fragmentation**: gaps appear between long-lived objects, and even though you may still have plenty of total memory, you cannot find a contiguous block big enough to hold the next large array.
- The lecturer compared this to a fragmented hard drive: data is scattered, and large requests cannot be serviced even though the total free space looks healthy.

23.15.2 malloc vs the .NET runtime — “we don’t worry about malloc”

- In C / C++, you call `malloc` (or `new`) to request heap memory and `free` (or `delete`) to release it. The compiler gives you fine-grained control — and the responsibility that comes with it.
- In C#, the .NET runtime handles allocation and the garbage collector handles release. As the lecturer put it, “for C#, we don’t need to worry about malloc, because all the memory allocation and stuff is handled by the .NET framework.”
- This is a deliberate trade-off: you give up some low-level control in exchange for safer, faster application development. C# is designed for the application layer, not for hand-tuning memory the way C++ does.
- There is still a price to pay — the GC has to run, and managed code is generally slightly slower than equivalent native code. For desktop apps, services, and tooling that price is invisible; for hard real-time embedded firmware it is usually unacceptable, which is why C remains the embedded default.

23.15.3 The garbage collector in one paragraph

The garbage collector is an automatic process inside the .NET runtime. It periodically walks the heap, finds any object that is no longer referenced by a live variable or by another reachable object, and frees its memory. Modern GCs also *compact* the heap — they shuffle the remaining live objects together so the free space ends up as one contiguous block. That compaction step is what protects long-running C# programs from the fragmentation problem above.

23.15.4 Deconstructors — the other half of automatic cleanup

- When a class instance’s reference goes out of scope on the stack, the variable disappears. The heap-allocated object it pointed to is no longer reachable, so the GC can reclaim it.
- The lecturer described this as “a process called a deconstructor that runs and destroys all the associated data on the heap as well.”
- In C# the equivalent is the `Dispose` pattern (for deterministic cleanup) and the finalizer (for last-resort cleanup before GC). The everyday rule is: use `using` blocks for anything that owns external resources (files, sockets, GPIO handles); let the GC handle pure-managed memory.

23.16 Threading and the Stack

- Each thread has its own stack — local variables are inherently thread-safe.
- All threads share the heap — accesses to shared objects need synchronisation (lock, `Interlocked`, immutability).
- True parallelism requires multiple cores; a single core interleaves threads (often described colloquially as ‘parallel’).
- On a microcontroller, multi-threading is rare — the stack budget alone is usually prohibitive.

23.16.1 In-class clarification: “interleaving, not parallel”

A student in the lecture proposed that threads are “parallel processing”; the lecturer accepted the intuition but corrected the vocabulary:

- On a single CPU core, threads are not running at the same instant. The core executes one thread for a slice of time, switches, and executes another. The lecturer’s word for this is *interleaving*.
- True **parallel** execution needs multiple cores (or multiple CPUs). Then two threads really can advance during the same clock cycle.
- Either way, the consequence for your code is the same: as soon as two threads can touch the same heap object, you need synchronisation (**lock**, **Interlocked**, or immutability).
- Because every thread has its own stack, locals and parameters are inherently thread-safe — they are never visible to another thread.
- “With microcontrollers, having multiple threads is quite a challenge because there is just not the compute space to do it.” On a modern microprocessor running Windows or Linux, by contrast, there is plenty of room and the runtime supports it directly.

23.17 Stack vs Heap — One-Page Comparison

Aspect	Stack	Heap
Allocation	Automatic on call	Manual (or via new + GC)
Speed	Fast — pointer bump	Slower — bookkeeping, search
Lifetime	Scope of the call	Until freed / collected
Order	LIFO	Free-for-all
Direction	Grows down from high mem	Grows up from low mem
Thread safety	Per-thread, inherently safe	Shared, needs synchronisation
Fragmentation	None	Possible — GC compacts in C#
Typical contents	Locals, parameters, return addresses	Objects allocated with new, large buffers

Key Definitions

Stack

LIFO memory region for function calls, locals, and temporary data.

Heap

Region for dynamic allocation; managed manually or via a GC.

Garbage Collector

Automatic memory reclamation system for unreachable heap objects.

Value Type

Holds the actual data (int, struct); usually on the stack.

Reference Type

Holds an address to data on the heap (class, array, string).

Mutable

Object whose state can be changed after creation.

Immutable

Object whose state is fixed once created.

Class

Template for objects, including data and methods.

Object

Instance of a class created with new.

Encapsulation

Bundling data and methods and restricting external access.

Namespace

Compile-time logical container preventing naming collisions.

Assembly

Runtime physical packaging unit (.dll / .exe).

Constructor

Method invoked on creation; allocates and initialises fields.

Overloading

Multiple methods/constructors with same name and different signatures.

Accessor

get / set methods providing controlled access to private properties.

Stack Frame

Block of stack memory pushed for one function call: parameters, return address, locals, saved registers.

Memory Leak

Heap memory that remains allocated because something still references it, but is never used again.

Dangling Pointer

Pointer to memory that has been freed; classic C bug that GC eliminates in C#.

Thread-Safe

Behaves correctly when called concurrently from multiple threads.

Stack Pointer (SP)

Register that always points to the current top of the stack; updated on every push or pop.

Stack Overflow

Failure caused when the stack grows so large it collides with the heap or other memory regions.

Fragmentation

Heap state where free memory is split into many small holes, blocking large contiguous allocations.

Garbage Collector

.NET runtime service that reclaims heap objects no longer referenced by live code, optionally compacting the heap.

malloc / free

C/C++ standard-library functions for explicit heap allocation and deallocation; equivalent to new/delete in C++.

Sources

- Lecture Week 7.pdf

- In-class transcript: *Introduction to OOP* (Dr. Banner) — used for the memory-as-the-heart-of-OOP framing, the stack walkthrough, the recursion warning, the heap free-for-all analogy, the malloc / .NET contrast, the threading “interleaving not parallel” clarification, and the value-vs-reference array demonstration.

Chapter 24

Week 9 — Advanced OOP & C# Conventions

ENEL712 Embedded Systems Design — Lecture Week 9

Summary

Advanced OOP concepts and C# conventions: naming styles, static behaviour, inheritance, polymorphism (overloading + overriding), and design constraints (sealed, abstract, enums).

Topics Covered

- Naming and language conventions
- Static behaviour
- Inheritance and reuse
- Polymorphism and method overriding
- Design constraints and encapsulation
- Enumerations

24.1 Naming and Language Conventions

24.1.1 Pascal Case

- Used for classes, interfaces, structs, and public members (methods, properties, events).
- Unifies the public contract under one readable style.

24.1.2 Camel Case

- Used for private/internal fields (often `_` prefix), local variables, and delegate instances.
- Distinguishes implementation details from the public interface.

24.1.3 Identifier rules

- C# uses neither `snake_case` nor `SCREAMING_SNAKE_CASE`.
- Identifiers start with a letter or underscore; cannot contain two consecutive underscores (reserved).

24.2 Static Behaviour

- Static methods are utility functions independent of instance state — bound at compile time.
- Static data members are shared across all instances; initialised once when the class loads.
- `const` fields are implicitly static and immutable (compile-time constants).
- `readonly` is a runtime constant — set at declaration or in the constructor.

24.3 Inheritance and Reuse

- Syntax: `class DerivedClass : BaseClass` — single inheritance only.
- `base` keyword calls the base constructor and accesses hidden/overridden members.
- protected members are accessible to the class itself and derived classes.

24.4 Polymorphism and Method Overriding

- Overloading (compile-time polymorphism): same name, different parameter list.
- Virtual methods are optionally overridable; runtime decides the version called (dynamic polymorphism).
- Abstract methods have no base implementation and must be overridden by derived classes.
- Override methods must use `override` and exactly match the base signature.

24.5 Design Constraints and Encapsulation

- `public/private/protected` control visibility; C# also has `internal` and `protected internal`.
- Sealed class: no further inheritance allowed.
- Sealed (overridden) method: no further override allowed.
- Sealing helps performance optimisation and prevents inappropriate extension.

24.6 Enumerations

- Assign meaningful names to integer values (e.g., `enum Season {Spring, Summer, Autumn, Winter};`).
- Improve readability and type safety, particularly in switch-case.
- Default underlying type is `int`.

24.7 Use Cases & Applications

- Following Microsoft naming conventions makes your code blend into the rest of the .NET ecosystem and look familiar to other engineers.
- static utility classes are a clean home for pure helpers that don't keep state.
- Inheritance is best for is-a relationships (a Manager is an Employee). Use composition (has-a) when behaviour just needs to be combined.
- sealed is a deliberate design statement: 'this hierarchy is closed'. Useful in libraries to prevent breaking changes.
- enum makes finite, named state spaces explicit instead of relying on magic integers.

24.8 Naming Convention Quick Reference

Element	Convention	Example
Class / interface / struct	PascalCase	MotorController
Public member (method, property)	PascalCase	Open(), Speed
Private field	camelCase or _camelCase	_speedRpm
Local variable / parameter	camelCase	rpmTarget
Constant	PascalCase	MaxSpeed

24.9 Worked Example: Method Overriding

```
public class Animal
{
    public virtual string Speak() => "...";
}

public class Dog : Animal
{
    public override string Speak() => "Woof";
}

public class Husky : Dog
{
    // 'sealed override' freezes Speak() -- no further customisation allowed.
    public sealed override string Speak() => "Awooo";
}
```

24.10 When to Use What

Need	Use
Behaviour shared by several types	abstract base class with virtual methods
Pure utility, no instance state	static class with static methods
Multiple capabilities mixed in	interface (single inheritance limits classes)
Compile-time constant	const
Runtime constant settable in constructor	readonly
Finite, named integer set	enum

Key Definitions

Pascal Case

Capitalise the first letter of each word, e.g., MyClassName.

Camel Case

First letter lowercase, subsequent words capitalised, e.g., myLocalVariable.

Static Member

Member belonging to the class itself rather than instances.

Inheritance

Mechanism letting one class extend another's functionality.

Polymorphism

Many forms — different objects respond to the same call differently.

Method Overloading

Compile-time polymorphism via different parameter signatures.

Virtual Method

Base method that may be overridden in derived classes (runtime polymorphism).

Abstract Method

Method without base implementation that must be overridden.

Encapsulation

Bundling data and methods and restricting external access.

Sealed Class

Class explicitly prevented from being inherited.

readonly

Field that can be assigned only at declaration or in a constructor; runtime equivalent of const.

abstract Class

Class that cannot be instantiated directly; may declare abstract methods that subclasses must implement.

Sealed Override

Marks an override as final — no further class may override that method again.

Composition over Inheritance

Design preference for combining objects (has-a) instead of subclassing (is-a) when both could work.

Sources

- Lecture Week 9.pdf

Chapter 25

Week 10 — .NET & C# Concepts

ENEL712 Embedded Systems Design — Lecture Week 10

Summary

Wider .NET/C# overview: the .NET platform and CLR, automatic memory management with generational GC, namespaces, value vs reference types, structures and collections, and Windows Forms basics.

Topics Covered

- The .NET framework
- Common Language Runtime
- Automatic memory management & generational GC
- Namespaces and the class library
- C# concepts and data types
- Structures and collections
- Graphical user interfaces

25.1 The .NET Framework

- C# is compiled by Roslyn into Common Intermediate Language (CIL/MSIL) — CPU-agnostic.
- Multiple .NET languages (C#, F#, VB.NET) all compile to the same CIL.
- .NET is cross-platform, open-source, and built for high-performance apps.
- Core components: Runtime, Libraries, Compiler, SDK, app stacks (e.g., ASP.NET Core).
- Features: automatic memory management, type safety, async support.

25.1.1 .NET Framework vs Modern .NET

Edition	Notes
.NET Framework	Windows-only; first released 2002; in maintenance (v4.8.1).
Modern .NET	Cross-platform (Windows/Linux/macOS); fully open-source; actively developed (v6, v8, ...).

25.2 Common Language Runtime

- JIT compiles methods to native code on first call; subsequent calls reuse the cached native code.
- Manages memory, threads, code-safety verification, compilation, and security.
- Eliminates many memory leaks via automatic management.
- Allows multiple high-level languages to share the same CIL.

25.3 Automatic Memory Management & Generational GC

- Reference types live on a contiguous managed heap.
- GC algorithm: assume garbage → trace from application roots → free unreachable objects → compact and update pointers.
- Unsafe code (pointers) is allowed in unsafe methods compiled with `/unsafe`.

25.3.1 Generations

Generation	Role
Gen 0	Newest, smallest segment — most collections happen here.
Gen 1	Buffer for objects that survived Gen 0.
Gen 2	Largest, holds long-lived objects; collected less often, more expensive.

25.4 Namespaces and the Class Library

- Namespaces group related classes and prevent naming conflicts (e.g., `System.IO.Stream`).
- using directive imports a namespace.
- .NET Class Library covers File I/O, collections, databases, and GUIs.

25.5 C# Concepts and Data Types

- Value types (`int`, `bool`, `struct`) live directly on the stack (or inline within a heap object); passing copies the value.
- Reference types (`class`, `string`, `array`) hold a reference on the stack and the data on the managed heap; passing copies the reference.
- Common types: `int` (32), `long` (64), `float` (32), `double` (64), `decimal` (128), `bool`, `string`.
- Parameters: `ref` / `out` / `params` modifiers as in Lecture 19.

- Strings: 16-bit chars; supports concatenation (+), interpolation (\$), regular/verbatim (@)/raw (""") literals.

25.6 Structures and Collections

- Structures: value types stored on the stack; convenient for small data; copy directly when initialised.
- ArrayList: dynamic array.
- Dictionary<K,V>: key-value lookups via hash table.
- Queue (FIFO), Stack (LIFO), Hashtable.
- Generic collections (List<T>, Dictionary<K,V>) avoid boxing/unboxing and provide type safety vs older non-generic collections.

25.7 Graphical User Interfaces

- Windows Forms: event-driven; controls respond to user actions (click, change).
- Controls: Label, Button, TextBox, ComboBox.
- Containers: GroupBox, Panel, TabControl.
- Event handlers respond to messages (e.g., plusB_Click).
- Use try/catch in handlers to gracefully handle invalid input.

25.8 Use Cases & Applications

- Modern .NET (8+) is the right choice for new desktop, web, and microservice projects — cross-platform, fast JIT, and active feature work.
- .NET Framework 4.8 is still common in legacy enterprise systems; understanding both saves headaches when porting.
- JIT and generational GC are why .NET app cold-start is slower than native but warm performance is competitive.
- Generics (List<T>, Dictionary<K,V>) avoid boxing/unboxing — preferred for any new code over the legacy ArrayList/Hashtable.

25.9 Worked Example: Boxing Cost

```
ArrayList legacy = new ArrayList();    // stores object -- boxes ints!
List<int> modern = new List<int>();    // stores int directly

for (int i = 0; i < 1_000_000; i++) legacy.Add(i);    // 1M boxings
for (int i = 0; i < 1_000_000; i++) modern.Add(i);    // no boxing
```

Boxing each int allocates a tiny heap object — the legacy version pressures the GC and runs slower for no benefit.

25.10 Quick GC Mental Model

- Most allocations live and die in Gen 0 — practically free.
- Survivors get promoted to Gen 1, then Gen 2.
- Gen 2 collections are expensive — caused by long-lived caches, statics, and event-handler leaks.
- Use weak references for caches that should not pin objects in memory.

25.11 JIT vs AOT

- Default .NET ships CIL and JIT-compiles per method on first call.
- Ahead-of-Time (AOT) compilation produces native binaries — faster startup, smaller deploys, useful for short-lived CLI tools or mobile apps.
- Trade-off: AOT loses some dynamic features (Reflection.Emit, runtime code generation).

Key Definitions

CIL

Common Intermediate Language — bytecode all .NET languages compile to.

JIT Compilation

Runtime translation of CIL to native machine code; native code is cached.

Managed Heap

CLR-reserved contiguous memory for reference types.

Garbage Collection (GC)

Automatic process freeing unreachable objects on the managed heap.

Value Type

Stored directly in its memory location (e.g., int, struct).

Reference Type

Variable holds an address to data on the managed heap (e.g., class, string).

Boxing/Unboxing

Conversion between value type and reference type; avoided by generic collections.

Namespace

Hierarchical naming system grouping related types (System.IO.Stream).

Event Handler

Function executed in response to a GUI/system message.

NuGet Package

Standard .NET deployment unit containing one or more assemblies plus metadata.

Assembly

Compiled .dll or .exe holding CIL + metadata; loaded by the CLR at runtime.

AOT (Ahead-of-Time) Compilation

Compiling CIL to native machine code at build time instead of runtime.

Weak Reference

Reference that does not prevent the GC from collecting the target; useful for caches.

Sources

- Lecture Week 10.pdf

Chapter 26

Week 11 — File Handling, Serial Ports, Graphics & Debugging

ENEL712 Embedded Systems Design — Lecture Week 11

Summary

Week 11 wider summary: file handling and serial ports, GUI graphics and plotting, Visual Studio debugging tools, and a refresher on core programming concepts.

Topics Covered

- File handling and serial ports
- Graphics and plotting
- Debugging in Visual Studio
- OOP / C# review concepts

26.1 File Handling and Serial Ports

- Files are sequential byte streams ending in EOF.
- Text files (ASCII/Unicode) vs binary files (raw bytes).
- StreamReader/StreamWriter handle text + encoding.
- FileStream is the byte-level base class; requires a FileMode (Append/Create/Open).
- BinaryReader/BinaryWriter read/write structured binary formats (JPEG, WAV, PNG).
- OpenFileDialog and SaveFileDialog: standard Windows file pickers; configurable Filter property.

26.1.1 SerialPort communication

- Object simplifies COM port use; create programmatically or in the designer.
- Methods Open / Close / Read / Write are blocking by default.

- Watch buffering, keep synchronisation with remote device, handle endianness (Intel = Little-Endian, AVR = Big-Endian).

26.2 Graphics and Plotting

- PictureBox displays GIF/JPG/BMP/PNG images.
- Custom drawing requires a surface (Panel/Form), a Graphics object, and a Pen.
- Paint event keeps custom drawings persistent — they vanish on minimise/cover unless redrawn.
- Pen draws lines and shape borders (Width, DashStyle, Transparency).
- Brush fills shapes (SolidBrush, TextureBrush).
- Graphics methods: DrawRectangle, FillEllipse, DrawString, etc.
- Chart control auto-creates plots (default bar) and supports line charts via the Series Collection Editor.

26.3 Debugging in Visual Studio

- Breakpoints suspend execution at a chosen line for inspection.
- Inspection windows: Autos/Locals show current-scope variables; Watch lets you track expressions.
- Call Stack shows the function chain that led to the current breakpoint.
- Stepping: F10 (Step Over), F11 (Step Into), Shift+F11 (Step Out).
- Conditional breakpoints trigger only when a C# expression is true.
- System.Diagnostics.Debug.WriteLine() prints to the Output window without breakpoints.

26.4 OOP / C# Review Concepts

- Use the smallest data type sufficient for your data.
- Control flow: for, while, if/else.
- Functions: return type, name, args; pointers/handles allow extra outputs.
- Pointers (*x) and handles (&x) let functions modify the caller's variables.
- Arrays are 0-indexed sequences of one type.
- Dynamic memory allocation: new/delete in C/C++ (manual) and new + GC in C# (automatic).

26.5 Use Cases & Applications

- Lab tooling: a single WinForms app combining a SerialPort + Chart + log file is a standard 'desktop companion' for an embedded device.
- Async IO: when listening to multiple ports or sockets, await over Read/Write keeps the UI responsive without manual threading.

- Visual Studio's debugger is the most efficient way to learn unfamiliar libraries — set a breakpoint and inspect.
- PictureBox + Paint event is enough to render real-time waveforms without bringing in a charting library.

26.6 Worked Example: Logging Serial Data to a File

```
private SerialPort sp;
private StreamWriter logger;

void StartLogging(string port, string path)
{
    logger = new StreamWriter(path, append: true);
    sp = new SerialPort(port, 115200);
    sp.DataReceived += (s, e) =>
    {
        string chunk = sp.ReadExisting();
        logger.Write(chunk);
        logger.Flush();
        Invoke((Action)(() => logTBox.AppendText(chunk)));
    };
    sp.Open();
}
```

26.7 Plotting Live Data on a PictureBox

```
private readonly Queue<float> samples = new();

private void picBox_Paint(object sender, PaintEventArgs e)
{
    var g = e.Graphics;
    var pen = new Pen(Color.SteelBlue, 1.0f);
    int x = 0;
    float[] arr = samples.ToArray();
    for (int i = 1; i < arr.Length; i++, x++)
        g.DrawLine(pen,
                    x, picBox.Height - arr[i - 1],
                    x + 1, picBox.Height - arr[i]);
}

void AddSample(float v)
{
    samples.Enqueue(v);
    while (samples.Count > picBox.Width) samples.Dequeue();
    picBox.Invalidate(); // schedule a Paint redraw
}
```

26.8 Debugging Cheat Sheet

Action	Shortcut	Use
Toggle breakpoint	F9	Pause execution at a line
Step Over	F10	Run current line; don't enter called methods
Step Into	F11	Enter called method
Step Out	Shift+F11	Run to end of current method, return to caller
Continue	F5	Run until the next breakpoint or exception
Conditional break	F9 → right-click → Conditions	Pause only when an expression is true

Key Definitions

Stream

Sequential byte flow used for file/IO.

EOF

End-of-file marker or condition.

Text File

Human-readable file (ASCII/Unicode).

Binary File

Raw bytes interpreted with a known data format.

FileMode

Enum used by FileStream (Open/Create/Append).

Blocking Method

Function call that halts execution until done.

Little-Endian

LSB stored at the lowest address (Intel).

Big-Endian

MSB stored at the lowest address (AVR / network).

Paint Event

GUI event firing when a window must be redrawn.

Graphics Object

Provides drawing methods like DrawRectangle, FillEllipse, DrawString.

Pen Object

Tool for drawing lines and outlines (width, style, colour).

Brush Object

Tool defining the fill of shapes (SolidBrush, TextureBrush).

Breakpoint

Marker that suspends program execution for inspection.

Call Stack

Window showing function call history at a breakpoint.

Pointer (*x)

Variable holding an address; enables pass-by-reference.

Dynamic Memory Allocation

Allocating memory at runtime, not compile time.

Invalidate

Tells a control to redraw itself the next time the message loop processes paint events.

Conditional Breakpoint

Breakpoint that pauses only when a Boolean expression evaluates to true.

Output Window

Visual Studio panel that receives Debug.WriteLine messages while the program runs.

Trace.Listeners

Mechanism for routing trace messages to files, consoles, or other sinks for production diagnostics.

Sources

- Lecture Week 11.pdf